Decentralized Crypto World Bank

A Blockchain-Based Banking Platform
with AI-Enhanced Security and Assistance

by

Md. Bokhtiar Rahman Juboraz

20301138

Md. Mahir Ahnaf Ahmed

20301083

A pre-thesis 1 report submitted to the Department of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
B.Sc. in Computer Science

Department of Computer Science and Engineering
BRAC University
February 2026

Declaration

It is hereby declared that

1. The report submitted is our own original work while completing degree at BRAC University.
2. The report does not contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.
3. The report does not contain material which has been accepted, or submitted, for any other degree or diploma at a university or other institution.
4. We have acknowledged all main sources of help.

Student's Full Name & Signature:

Md. Bokhtiar Rahman Juboraz	Md. Mahir Ahnaf Ahmed
20301138	20301083

Approval

The pre-thesis 1 report of final year project titled "Decentralized Crypto World Bank: A Blockchain-Based Banking Platform with AI-Enhanced Security and Assistance" submitted by

1. Md. Bokhtiar Rahman Juboraz (20301138)
2. Md. Mahir Ahnaf Ahmed (20301083)

Of Spring, 2026 has been accepted as satisfactory in partial fulfillment of the requirement for the degree of B.Sc. in Computer Science on February 20, 2026.

Examining Committee:

Supervisor:
(Member)

Mr. Annajiat Alim Rasel
Senior Lecturer
Department of Computer Science and Engineering
BRAC University

Project Coordinator:
(Member)

Dr. Md. Golam Rabiul Alam
Professor
Department of Computer Science and Engineering
BRAC University

Head of Department:
(Chair)

Dr. Sadia Hamid Kazi
Chairperson and Associate Professor
Department of Computer Science and Engineering
BRAC University

Ethics Statement

This project operates exclusively on public blockchain test networks using test tokens with no real monetary value. No real financial transactions, personal banking data, or user financial records are involved at any stage of development or evaluation. All transaction data used for Artificial Intelligence and Machine Learning (AI/ML) model training and evaluation is either synthetically generated or sourced from publicly available anonymized datasets. The platform prototype does not process production Know Your Customer (KYC) or Anti-Money Laundering (AML) data in its current scope: architectural KYC/AML workflows are specified for future deployment, but the MVT run uses **synthetic credentials and document-hash stubs only** (Section 3.8). All wallet addresses used during testing are disposable testnet addresses. We acknowledge the ethical considerations inherent in AI-assisted financial decision-making and have incorporated explainability mechanisms (SHapley Additive exPlanations, or SHAP) to ensure that automated risk assessments remain transparent and auditable by human reviewers. Future phases plan to incorporate a privacy-preserving ZKP-based identity compliance layer in which users prove KYC status without exposing personal data on-chain; the ethical implications of this design will be evaluated in the final thesis.

Abstract

Development finance moves capital through several institutional layers, but settlement is often slow, costly, and hard to audit. Decentralized finance (DeFi) can automate lending on-chain, yet most protocols use flat pools that do not match real tiered banking. *Crypto World Bank* (CWB) is a research prototype that specifies a four-tier lending platform on EVM-compatible infrastructure: World Bank Reserve → National Bank → Local Bank → Client.

The design pairs on-chain reserve rules and role-based access with off-chain KYC and ML risk scoring (Random Forest, Isolation Forest, SHAP), committed through a commit–reveal oracle. A conversational agent (Qwen3-8B + MCP) is planned as an optional plain-language UI. At pre-thesis stage, the main output is formal specification and a four-phase build plan. Measured ML results come from a 50,000-address subsample of BCCC-DeFiFraudTrans-2025 (held-out F1 = 0.891, ROC-AUC = 0.991); full-corpus retraining and 22-dimensional CWB feature mapping are documented as evaluation extensions (Section 3.21). The graded demonstration adopts stability-first, zero-cost deployment substitutes (`MockUSDC`, Sepolia testnet, local Foundry simulation) while retaining the full production design. Final-thesis targets include testnet loan lifecycle, latency and gas tables, reserve invariant tests, and a small agent demo with a human confirmation gate.

Keywords: Blockchain, DeFi, Institutional Finance, Hierarchical Lending, Smart Contracts, Explainable AI, Group Lending

Dedication

Dedicated to our families for their unwavering support throughout our academic journey.

Acknowledgment

We would like to express our sincere gratitude to our panel members for their guidance and support throughout this project. We also thank our supervisor, Mr. Annajiat Alim Rasel, Senior Lecturer at the Department of Computer Science and Engineering, BRAC University, for his guidance and support. Finally, we thank the Department of Computer Science and Engineering at BRAC University for providing us with the resources and academic environment to pursue this work.

Contents

Declaration 2

Approval 3

Ethics Statement 5

Abstract 6

Dedication 7

Acknowledgment 8

List of Tables v

List of Figures vii

List of Formulas ix

List of Abbreviations x

1 Introduction 1
- 1.1 Background . . . 2
- 1.2 Rationale of the Study . . . 3
- 1.3 Problem Statement . . . 3
- 1.4 Objectives . . . 4
- 1.5 Research Questions . . . 4
- 1.6 Blockchain Justification . . . 4
  - 1.6.1 Blockchain Fundamentals and the EVM Execution Model . . . 5
- 1.7 Proposed Solution . . . 5
  - 1.7.1 Banking Functions of the Platform . . . 6
  - 1.7.2 Service Delivery Across Scale . . . 6
  - 1.7.3 Cross-Tier Lending System . . . 6
- 1.8 Methodology in Brief . . . 8
- 1.9 Scopes and Challenges . . . 9
  - 1.9.1 Economic Sustainability: Interest Revenue Versus Gas Costs . . . 10
  - 1.9.2 Resistance to Institutional Capture . . . 10
  - 1.9.3 Stablecoin-First Lending and Supply Scalability . . . 10

2 Literature Review 11
- 2.1 Preliminaries . . . 11
  - 2.1.1 Review Methodology (PRISMA-Style Frame) . . . 11
- 2.2 Review of Existing Research . . . 11
  - 2.2.1 DeFi Lending and the Structural Gap . . . 11
  - 2.2.2 Fraud Detection and Explainable AI . . . 12
  - 2.2.3 Hierarchical Capital Distribution . . . 12

2.2.4 Group Lending and Microfinance . . . 12
2.2.5 Smart Contract Security and Governance . . . 12
2.2.6 Supporting Infrastructure . . . 12
2.2.7 Zero-Knowledge Proofs for Compliance-Aware DeFi Identity . . . 13
2.2.8 Future Research Directions Scoped . . . 13
2.3 Summary of Key Findings . . . 13

3 System Architecture and Design 16
3.1 System Design Overview . . . 16
3.2 Specifications and Requirements . . . 16
3.3 High-Level Architecture . . . 17
3.4 Blockchain Platform Selection . . . 23
3.4.1 Transaction Verification and Consensus . . . 24
3.4.2 Oracle Architecture: Off-Chain AI to On-Chain Decision . . . 24
3.5 Data Model and Database Design . . . 25
3.5.1 Entity Summary . . . 27
3.5.2 Logical Schema versus Physical Migration Tiers . . . 28
3.5.3 EER Constructs Applied . . . 30
3.5.4 Normalization . . . 30
3.5.5 Relational Integrity Constraints . . . 32
3.6 On-Chain and Off-Chain Data Partitioning . . . 33
3.6.1 Financial Data Authority and Lifecycle . . . 34
3.7 Operating Context . . . 35
3.8 User Taxonomy and Onboarding Flows . . . 36
3.9 Banking Modules . . . 38
3.9.1 Kinked Interest Rates . . . 38
3.9.2 Liquidation Engine . . . 39
3.9.3 Savings and Fixed Deposits . . . 39
3.9.4 Credit Passport (SBT) . . . 40
3.10 Single-Chain Deployment (Design Revision) . . . 40
3.11 Multi-Entity and Cross-Tier Capital Operations . . . 42
3.11.1 InterBankLendingPool: Fully Specified Same-Tier Lending . . . 43
3.11.2 Upward Surplus Repatriation . . . 43
3.11.3 Syndicated and Club Lending . . . 43
3.11.4 Senior–Junior Tranched Lending Pools . . . 43
3.11.5 Cross-Tier Treasury FX Swap . . . 44
3.11.6 Multilateral Settlement Netting Engine (Future Work) . . . 44
3.11.7 Database, Phase, and Contract Consistency for Multi-Entity Operations . . . 44
3.12 System Modeling . . . 45
3.12.1 Use Case Diagram . . . 46
3.12.2 Activity Diagrams . . . 47
3.12.3 Data Flow Diagrams . . . 51
3.12.4 Sequence Diagrams . . . 51
3.12.5 UML Class Diagram . . . 56
3.12.6 User Interface Design . . . 58
3.12.7 Four-Tier Capital Flow . . . 58
3.13 Group Solidarity Lending . . . 59

3.14 Governance Framework . . . 60
3.15 Five-Layer Defense-in-Depth Security Architecture . . . 63
3.16 Threat Model and Security Controls . . . 64
3.17 Societal Impact . . . 64
3.18 Environmental Impact . . . 64
3.19 Ethical Issues . . . 64
3.20 Project Management and Risk . . . 65
3.21 Design Limitations and Zero-Cost Demonstration Strategy . . . 65

4 Methodology **66**
4.1 Minimum Viable Thesis (MVT) Checklist . . . 67
4.2 List of Features . . . 67
4.3 Development Methodology . . . 69
4.3.1 Process Model Selection . . . 70
4.4 Planned AI/ML Support and Risk-Score Wiring . . . 70
4.5 Conversational Agent (Phase III–IV) . . . 71
4.6 Real-Time Dashboard Pipeline and Runtime Monitoring . . . 73
4.7 Implementation Phase Plan and Deliverables . . . 73
4.7.1 Phase I: Foundation and Infrastructure (Weeks 1–4) . . . 73
4.7.2 Phase II: Core Banking and On-Chain Lifecycle (Weeks 5–9) . . . 75
4.7.3 Phase III: AI/ML Oracle Pipeline and Conversational Agent (Weeks 10–13) . . . 76
4.7.4 Phase IV: Verification, Evaluation, and Finalization (Weeks 14–16) 77
4.8 SDLC Stage Mapping . . . 80
4.9 Design Decisions and Alternatives . . . 80
4.9.1 Justification of Selected Technologies . . . 83
4.10 Implementation Feasibility and Demonstration Scope . . . 84

5 Evaluation, Testing and Feasibility **86**
5.1 Evaluation Methodology . . . 86
5.1.1 Verification and Validation . . . 86
5.2 Machine Learning Evaluation . . . 87
5.2.1 Datasets and Benchmark Results . . . 87
5.2.2 Explainability Methodology . . . 89
5.3 Foundry Invariant and Fuzz Test Suite . . . 89
5.4 Implementation Evidence and Traceability . . . 90
5.4.1 Requirements Traceability Matrix . . . 90
5.5 Discussion and Examiner Limitations . . . 91
5.6 Economic Feasibility Analysis . . . 92
5.6.1 Prototype Cost Model . . . 92
5.6.2 Gas Cost and Break-Even . . . 92
5.6.3 Net Present Value and Return on Investment . . . 93

6 Conclusion **94**

A Database Schema Reference **95**
A.1 Full Relational ERD . . . 95
A.2 Indexing Strategy . . . 97
A.3 Functional Dependencies . . . 99

A.4 Materialized Views and Agent Query Optimisation 100
A.5 Representative SQL Queries 100

B Agent Harness Reference **103**
B.1 Per-User Context and Prompt Assembly 103
B.2 Session Lineage, Compression, and Toolset Scoping 104
B.3 Lifecycle Hooks and Optional Capabilities 104

C Technology Stack **105**
C.1 In-product assistant: local large language model (LLM) integration (prototype) 105

D Smart Contract Capabilities **112**

Appendix C: Planned Testnet Deployment Manifest **114**

Appendix D: WorldBankReserve Contract Interface **115**

Appendix E: Internal Industry Research Notes **117**

Appendix F: UI Prototype Wireframes (Figma) **118**

References **120**

List of Tables

1.1 Phase–deliverable mapping . . . 9

2.1 Tier-1 literature synthesis . . . 14
2.2 Protocol comparison on eleven architectural dimensions . . . 15

3.1 Functional requirements (representative) . . . 16
3.2 Non-functional requirements . . . 17
3.3 Four-layer architecture technical detail . . . 17
3.4 Smart contract inventory . . . 22
3.5 Blockchain platform selection criteria and justification . . . 23
3.6 Blockchain Platform Selection: Comparison of EVM-Compatible Networks 23
3.7 Blockchain platform selection (continued) . . . 24
3.8 Blockchain Platform Selection (Continued): Operational and Deployment Factors . . . 24
3.9 Chainlink upgrade paths versus MVT substitutes . . . 25
3.10 Core database entities . . . 27
3.11 Deferred and extension entities . . . 28
3.12 Audit and logging table taxonomy . . . 29
3.13 EER constructs applied . . . 30
3.14 EER Constructs Applied: Specialization, Hierarchy, and Constraints . . . 30
3.15 Relational integrity constraints . . . 32
3.16 Relational Integrity Constraints: Referential and Domain Rules . . . 32
3.17 Data partitioning between on-chain and off-chain storage . . . 33
3.18 On-Chain vs. Off-Chain Data Partitioning: State, Analytics, and Privacy Boundaries . . . 33
3.19 Operational user taxonomy . . . 36
3.20 Formal actor taxonomy . . . 37
3.21 Actor permission matrix (tabular summary) . . . 37
3.22 Credit Passport tier schedule . . . 40
3.23 FK anchors for extended and multi-entity entities . . . 45
3.24 FK Anchors for Extended and Multi-Entity Entities . . . 45
3.25 Governance framework summary . . . 61
3.26 Five-layer defense-in-depth model . . . 64
3.27 Threat model (summary) . . . 64
3.28 Design reference versus demonstration path . . . 65

4.1 MVT deliverable checklist . . . 67
4.2 Complete feature inventory . . . 67
4.3 Process model comparison . . . 70
4.4 MCP tool server (MVT subset) . . . 73
4.5 Phase I task register . . . 74
4.6 SDLC stage mapping . . . 80
4.7 Technology Stack: Additional Alternative Evaluations . . . 80
4.8 Design decisions and alternatives considered . . . 81
4.9 Design Decisions: Evaluated Alternatives and Selection Rationale . . . 81

4.10 Design alternative evaluation criteria 82
4.11 Selected technology justifications 83
4.12 Alternative technology justifications 84

5.1 Datasets used for ML risk scoring 87
5.2 BCCC subsample benchmark (held-out test) 88
5.3 Pre-thesis implementation evidence 90
5.4 Requirements traceability matrix (excerpt) 91
5.5 Economic feasibility — zero-cost prototype 92
5.6 Illustrative three-year NPV and ROI (pilot scale) 93

A.1 Indexing strategy 97
A.2 Indexing Strategy: B-tree Indexes for Time-Window and High-Frequency Queries 97
A.3 Representative functional dependencies 99
A.4 Functional Dependencies in the Core Lending and Identity Schema 99

C.1 Technology stack summary 111
C.2 Local LLM Assistant Integration: Components, Configuration, and Prototype Stance 111

D.1 Planned testnet deployment manifest 114
D.2 Internal industry research corpus 117

List of Figures

1.1 Crypto World Bank system overview . . . 2
1.2 Six banking functions of the platform . . . 6
1.3 Cross-tier lending flow directions . . . 7

2.1 PRISMA-style literature screening flow . . . 11

3.1 Four-layer application architecture . . . 18
3.2 Component diagram with architecture-pattern labels . . . 19
3.3 Level-0 data-flow diagram (core lending) . . . 19
3.4 Level-1 data-flow diagram (core lending) . . . 20
3.5 System component architecture . . . 21
3.6 Layered blockchain and application stack . . . 23
3.7 Oracle architecture . . . 25
3.8 Core system entity graph . . . 26
3.9 Extended ERD: banking products . . . 26
3.10 On-chain versus off-chain data partitioning . . . 34
3.11 Financial data authority and lifecycle . . . 35
3.12 Actor permission matrix (primary actions) . . . 38
3.13 Kinked utilization-based borrowing rate . . . 39
3.14 LiquidationEngine tier models and lifecycle . . . 39
3.15 SavingsVault and FixedDeposit closed loop . . . 40
3.16 On-chain Credit Passport SBT lifecycle . . . 40
3.17 Multi-entity and cross-tier capital operations . . . 42
3.18 Use-case diagram (nine-actor taxonomy) . . . 47
3.19 USDC loan lifecycle activity flow . . . 48
3.20 Retail onboarding and identity activity flow . . . 49
3.21 Client banking session activity flow . . . 50
3.22 Loan approval sequence . . . 52
3.23 Installment and income verification . . . 53
3.24 Hierarchical banking and data sync . . . 54
3.25 Chat and AI agent sequences . . . 55
3.26 UML class diagram (core lending domain) . . . 57
3.27 Four-tier hierarchical capital flow . . . 59
3.28 Solidarity group lending lifecycle . . . 60
3.29 Governance dual-path . . . 61
3.30 SAR and AML compliance workflow . . . 62
3.31 Five-layer defense-in-depth security architecture . . . 63

4.1 ML oracle commit–reveal gate . . . 71
4.2 Planned development and verification toolchain . . . 79

5.1 RQ2 latency and gas measurement plan . . . 87
5.2 ML evaluation and explainability benchmarks . . . 88
5.3 Explainability and anomaly-detection flow . . . 89

A.1 Full relational ERD (51 entities) 96

C.1 AI banking agent request path 108
C.2 AI agent data flow 109

D.1 Retail loan application prototype (Figma) 118
D.2 Authority Brief approver dashboard prototype (Figma) 119

List of Formulas

Utilization (U): $U = \frac{L}{L+B}$

Kinked rate ($U \le U^*$): $r_b(U) = r_0 + \frac{U}{U^*} \cdot r_1$

Kinked rate ($U > U^*$): $r_b(U) = r_0 + r_1 + \frac{U-U^*}{1-U^*} \cdot r_2$

Collateral Ratio: $CR = \frac{C}{D}$

Loan-to-Value: $LTV = \frac{\text{Loan Amount}}{\text{Collateral Value}}$

APR (simple): $APR = r_{\text{period}} \times N$

Compound interest: $A = P\left(1 + \frac{r}{n}\right)^{nt}$

EMI: $EMI = \frac{P \cdot r \cdot (1+r)^n}{(1+r)^n - 1}$

Health Factor: $HF = \frac{C \times LT}{D}$

Group Health Factor: $HF_{\text{group}} = \frac{\text{total\_group\_collateral} \times LT}{\text{total\_group\_outstanding\_debt}}$

Institutional Health Factor: $HF_{\text{inst}} = \frac{\text{on\_chain\_reserve} - \text{min\_reserve}}{\text{outstanding\_borrowing}}$

Reserve Ratio: $RR = \frac{\text{Reserves}}{\text{Total Deposits}}$

Platform Solvency Ratio: $PSR = \frac{\text{TotalReserveBalance}}{\text{TotalOutstandingLoans}}$

Credit Velocity: $CV = \frac{\text{Capital Recycled per Period}}{\text{Total Reserve}}$

Interbank rate cap: $r_{\text{IB}}(U) \le r_{\text{downward}}(U) - \delta$

Upward deposit rate cap: $r_{\text{up}} < r_{\text{down}} - \delta$

FX conversion: $\text{Amount}_B = \text{Amount}_A \times \frac{P_A}{P_B}$

Treasury swap: $\text{Amount}_{\text{to}} = \text{Amount}_{\text{from}} \times \frac{P_{\text{from}}}{P_{\text{to}}} \times (1 - \text{spread})$

Oracle commit–reveal: $h = \text{keccak256}(s \,\|\, \text{nonce})$

Group loan share: $\text{Share}_i = \frac{\text{LoanTotal}}{N_{\text{members}}}$

On-chain credit score: $CS = \sum_i w_i \cdot \text{feature}_i$

Isolation Forest score: $s(x, n) = 2^{-E(h(x))/c(n)}$

Random Forest fraud probability: $P(\text{fraud} \mid x) = \frac{1}{T} \sum_{t=1}^{T} f_t(x)$

Net interest split: $\text{NetInterest} = \text{InterestCollected} - \text{DepositorYield} - \text{InsuranceAllocation}$

SHAP (Shapley value): $\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F|-|S|-1)!}{|F|!} \left[f(S \cup \{i\}) - f(S)\right]$

List of Abbreviations

AA: Account Abstraction (ERC-4337)

ALM: Asset-Liability Management

AI: Artificial Intelligence

AI/ML: Artificial Intelligence / Machine Learning

AML: Anti-Money Laundering

APR: Annual Percentage Rate

API: Application Programming Interface

BRAC: Bangladesh Rural Advancement Committee

CAR: Capital Adequacy Ratio (Basel III; see PSR for this platform)

CBDC: Central Bank Digital Currency

CCIP: Cross-Chain Interoperability Protocol (Chainlink)

CCS: ACM Computing Classification System

CR: Collateral Ratio

CVL: Certora Verification Language

DeFi: Decentralized Finance

DID: Decentralized Identifier (W3C)

DON: Decentralised Oracle Network (Chainlink)

EIP: Ethereum Improvement Proposal

EIP-7702: Ethereum Improvement Proposal 7702 — Session Key standard enabling scoped, time-bound wallet authorisations for an autonomous agent

EMI: Equated Monthly Installment

ERC: Ethereum Request for Comments (token / interface standard)

EVM: Ethereum Virtual Machine

FL: Federated Learning

FATF: Financial Action Task Force

FX: Foreign Exchange

GNN: Graph Neural Network

HF: Health Factor

KYC: Know Your Customer

LLM: Large Language Model

L2: Layer 2

LTV: Loan-to-Value Ratio

MFI: Microfinance Institution

MiCA: Markets in Crypto-Assets Regulation (EU)

ML: Machine Learning

MCP: Model Context Protocol — tool-calling interface used by the AI agent to interact with CWB banking operations

NIM: Net Interest Margin

PRISMA: Preferred Reporting Items for Systematic Reviews and Meta-Analyses

PoS: Proof of Stake

PoR: Proof of Reserve (Chainlink on-chain reserve verification)

PSR: Platform Solvency Ratio

QRC: QR code

RBAC: Role-Based Access Control

RLS: Row-Level Security (PostgreSQL policy feature enforcing append-only access on audit tables)

RR: Reserve Ratio

SAR: Suspicious Activity Report

SBT: Soulbound Token (non-transferable ERC standard)

SHAP: SHapley Additive exPlanations

SOFR: Secured Overnight Financing Rate

SSE: Server-Sent Events

SSI: Self-Sovereign Identity

SoK: Systematization of Knowledge

SWC: Smart Contract Weakness Classification

TVL: Total Value Locked

U: Utilization

UUPS: Universal Upgradeable Proxy Standard (ERC-1822)

VC: Verifiable Credential (W3C)

XAI: Explainable Artificial Intelligence

ZKP: Zero-Knowledge Proof

zkAML: Zero-Knowledge Anti-Money-Laundering proof

zkEVM: Zero-Knowledge Ethereum Virtual Machine — a ZK rollup that inherits Ethereum L1 security via cryptographic validity proofs

zk-SNARK: Zero-Knowledge Succinct Non-Interactive Argument of Knowledge

Chapter 1

Introduction

The Crypto World Bank (CWB) is a final-year research prototype for a blockchain-based institutional finance platform. Existing DeFi protocols handle single functions well—Aave for lending, Uniswap for exchange—but none models a full tiered banking stack. CWB specifies a four-tier hierarchy (World Bank → National Bank → Local Bank → Client) with deposit mobilization, credit allocation, installments, interbank liquidity, group lending, and reserve enforcement. Tier 1 contracts and the core lending workflow are specified in Solidity; multi-entity contracts are detailed in Chapter 3.3 and scheduled for later phases.

The platform targets three groups underserved by current DeFi: (1) development banks and regional institutions that face high correspondent-banking costs ($42 per transaction, 2–5 day delays) [15, 23]; (2) licensed MFIs and cooperative banks that lack programmable credit tools; and (3) semi-formal borrowers reached through MFIs rather than direct wallet onboarding. Large unbanked and MSME financing gaps [9, 13] show that the problem sits at the institutional layer, not only at the retail user. Project artifacts are in our GitHub repository (`contracts/`, `backend/`, `frontend/`, `ml-service/` at repository root).

Blockchain here is a coordination layer: shared state, tamper-evident records, and programmable rules among institutions that may not fully trust each other. Comparable hybrid models include the World Bank FundsChain programme and JPMorgan Kinexys for institutional settlement on permissioned ledgers.

Figure 1.1 shows the four tiers, application stack, oracle ML path, and compliance controls at a glance. Full specifications follow in Chapter 3.3.

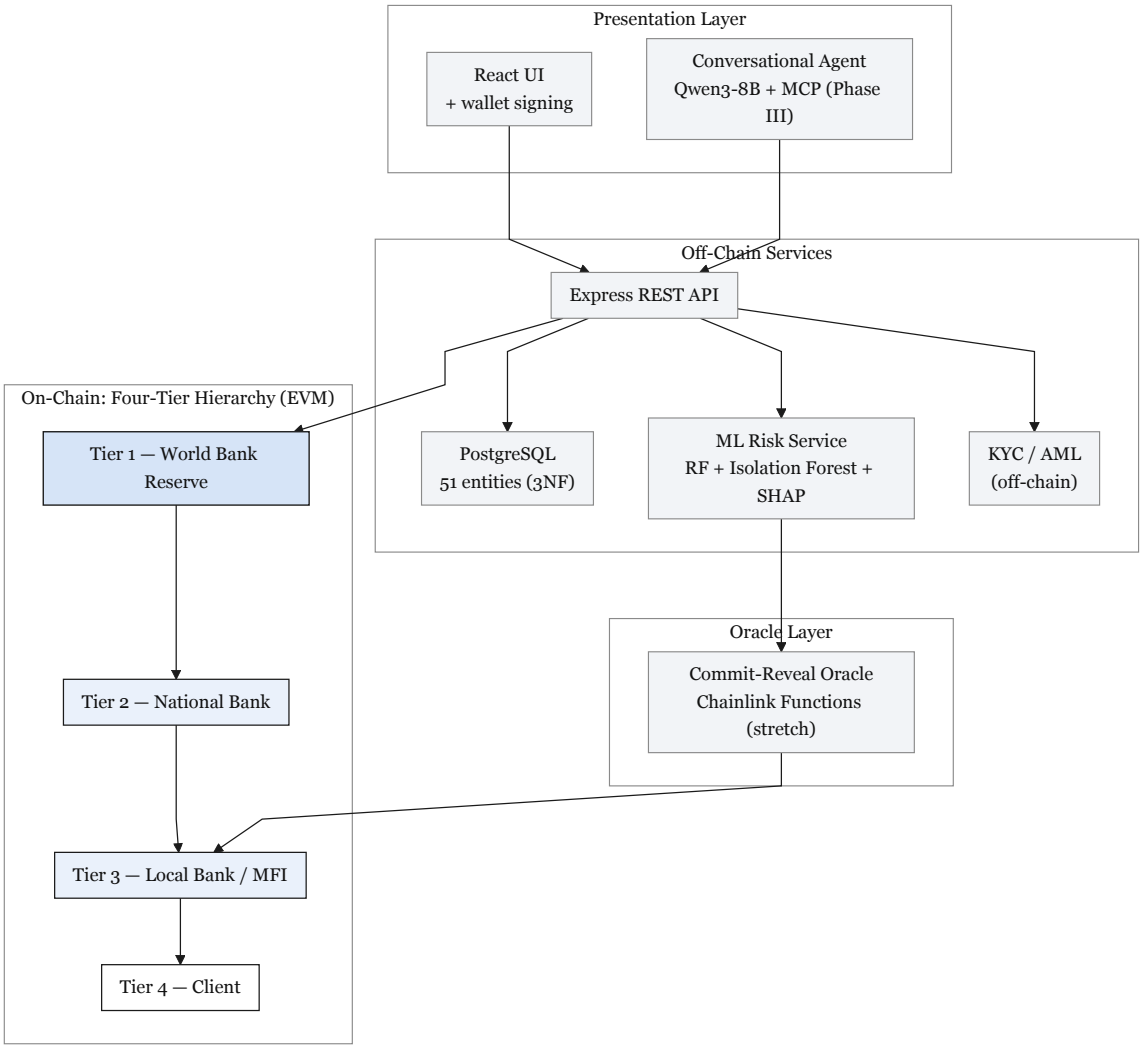


Figure 1.1: Crypto World Bank system overview

1.1 Background

Development finance moves capital through several layers—from supranational bodies to national banks, local lenders, and end borrowers. This model spreads risk but creates real operational problems. Correspondent banking requires pre-funded nostro/vostro accounts in every currency corridor, locking capital that could otherwise be lent [15]. Cross-border transfers often take 2–5 days and cost about $42 per transaction [15, 23]. Remittance fees remain high (6.49% global average in 2024, above the UN 3% target) [17]. An estimated 1.4 billion adults remain unbanked [9], and MSMEs face a $4.5 trillion financing gap [13].

DeFi has shown that lending, collateral, and interest can run on smart contracts with public audit trails. Aave v3 holds roughly $26.3 billion TVL [18], and the broader DeFi lending market exceeds $55 billion [8]. Yet these protocols use flat pools: all lenders share one pool and all borrowers draw from it, with no institutional tiers, cross-tier flows, or client-via-local-bank paths typical of development finance.

What blockchain adds here. A **smart contract** is program code on a blockchain that runs the same way on every node. For CWB, three properties matter: (1) **governed exe-**

cution—contract changes need a recorded vote and timelock (Section 3.14), reducing flash-governance attacks like the $182M Beanstalk exploit [92]; (2) **atomic settlement**—multi-step operations complete fully or revert; (3) **public audit**—every state change is logged on-chain. **Blockchain** adds a shared ledger so co-platform members see the same reserve and loan state without slow bilateral reconciliation [15]. **DeFi** applies this to finance, but CWB keeps institutions, KYC, and ML off-chain while enforcing tier rules on-chain (Section 1.6). CWB extends flat DeFi in three ways: institutional hierarchy, multi-entity operations (group loans, syndication, interbank pools), and alignment with development-finance structures absent in current protocols [1].

Optional conversational interface. Users interact mainly through the React UI and wallet signing. A planned Phase III agent (Qwen3-8B + MCP, 3–5 core tools at MVT scope) offers plain-language queries and one demonstrated write path, always behind a human confirmation gate. All write operations remain available through the web UI.

1.2 Rationale of the Study

Three forces motivate this work: (1) long-standing inefficiencies in development finance (slow settlement, weak transparency); (2) the lack of tiered banking features in current DeFi (deposits, group lending, hierarchical flows); and (3) the chance to combine on-chain auditability with practical ML support for credit decisions. CWB explores whether these can be joined in one architecture for institutional and retail participants.

1.3 Problem Statement

Development finance runs through a chain of institutions—from global bodies to national banks, local lenders, and individual borrowers. This creates several problems:

1. **Lack of transparency.** Lower tiers cannot easily verify reserve levels or lending decisions above them. Reserves are often self-reported and audited only quarterly.
2. **Slow settlement and idle capital.** Cross-border transfers take 2–5 days and cost about $42 per transaction [15, 23]. Pre-funded correspondent accounts lock capital out of lending [15].
3. **Inconsistent risk evaluation.** Fraud and credit checks rely on manual judgment. Borrowers are re-assessed at each tier with no shared, verifiable history.
4. **Access and trust barriers.** Building trust between institutions is slow and costly, disadvantaging smaller banks and borrowers in developing markets [9].
5. **No programmable reserve tools.** MFIs lack real-time, auditable reserve enforcement between audit cycles.
6. **No on-chain group credit.** Many borrowers lack individual collateral but could use solidarity groups (BRAC, Grameen). No DeFi protocol models mutual liability with MFI-appropriate controls.

DeFi automates lending at scale (Aave $26.3B TVL [18]; Compound $1.4B [19]), but remains poorly suited to institutional development finance:

- **Flat lending models**—no four-tier hierarchy or cross-tier allocation (distinct from Aave sub-markets, which are risk buckets, not institutional relationships).
- **No ML risk layer**—decisions rely on collateral and utilization curves only.
- **No tiered governance**—token-weighted voting, not institutional roles.

- **Limited institutional DeFi**—Maple and Goldfinch serve institutional credit but without hierarchical capital flows or interbank facilities [21, 22].

1.4 Objectives

1. **Design a four-tier lending architecture** (World Bank → National Bank → Local Bank → Client) on an EVM-compatible chain with on-chain hierarchy and shared ledger access. Contracts and workflows are specified in this pre-thesis; testnet deployment is planned for the final thesis.
2. **Specify an extended banking product suite**—deposits, savings, and solidarity group lending—on top of the tiered foundation.
3. **Plan an off-chain analytics layer** (fraud detection, anomaly detection, SHAP explainability) plus a minimal conversational agent (MCP, 3–5 tools, one write demo) as a Phase III deliverable.
4. **Specify transparent lending processes** with borrowing limits, installments, and role-based access in smart contracts.
5. **Plan testnet deployment** on Polygon Amoy and/or Ethereum Sepolia for stability, with Polygon zkEVM as the design-preference settlement model.
6. **Document governance, security, and rollout** toward academic validation and possible regulatory sandbox pilots.

1.5 Research Questions

The study is guided by five research questions, evaluated in Chapter 5:

1. **RQ1 (architecture encoding):** Does a four-tier smart-contract hierarchy encode development-finance-style intermediation—directional capital flow, institutional roles, and reserve enforcement—absent from flat DeFi lending pools?
2. **RQ2 (settlement and transparency):** Can on-chain settlement on public testnets achieve acceptable latency and cost for retail loan lifecycles while preserving verifiable reserve and audit state?
3. **RQ3 (analytics support):** Can an off-chain Random Forest + Isolation Forest pipeline with SHAP explainability support human approver decisions, and what baseline fraud-detection metrics are achievable on BCCC-DeFiFraudTrans-2025 before CWB-specific retraining?
4. **RQ4 (technical viability):** Can the specified contracts, APIs, and frontend complete an end-to-end loan lifecycle on a stability-first testnet with passing unit, integration, and invariant tests?
5. **RQ5 (banking extensions):** Are deposit, group-lending, and multi-entity modules sufficiently specified—and partially demonstrable—to extend flat pool lending toward a broader banking product suite?

1.6 Blockchain Justification

The core problem is **multi-party trust**: institutions must share financial state and enforce common rules without relying on one central operator or slow bilateral agreements. Traditional fixes—a central authority or heavy legal reconciliation—create either systemic risk or the 2–5 day delays and $42 costs described above.

Blockchain helps through four properties:

1. **Shared state.** All tiers read the same reserve ratios and loan states in real time, not separate ledgers reconciled by batch messaging.
2. **Programmable rules.** Reserve ratios, rate curves, limits, and approval flows run as on-chain code. Changes require governance (timelock, quorum; Section 3.14).
3. **Public audit trail.** Every disbursement, repayment, and reserve update is logged on-chain for regulators, partners, and auditors.
4. **Atomic multi-party settlement.** Group loans, syndicated deals, and netting complete fully or revert—no half-settled state.

A central database could store the same records, but it reintroduces a trusted admin. Blockchain is chosen because verifiable, programmable multi-institution coordination is the actual requirement.

Hybrid institutional architecture. CWB is **not** fully disintermediated retail DeFi. Institutions, KYC, ML inference, and APIs run off-chain (Express.js, PostgreSQL, FastAPI); the chain enforces rules, reserves, RBAC, and audit trails. This matches institutional DeFi patterns (Maple/Goldfinch-style permissioned layers plus on-chain enforcement). The chain does not remove banks; it makes tier flows verifiable.

1.6.1 Blockchain Fundamentals and the EVM Execution Model

Brief technical reference for readers new to EVM smart contracts. Familiar readers may skip to Section 1.7.

A blockchain is an append-only ledger linked by cryptographic hashes. On Proof-of-Stake networks, history stays stable as long as no majority coalition rewrites it [50]. CWB targets EVM-compatible chains: Polygon zkEVM as design preference; Amoy/Sepolia for prototype stability.

EVM execution. Smart contracts are deterministic programs whose state changes cost **gas**. Storage writes (`SSTORE`) dominate cost; a full retail loan lifecycle involves roughly 27–32 on-chain state changes versus 10–15 user-facing transactions (Section 5.6.2). Contracts behave as state machines (`PENDING` → `APPROVED` → `ACTIVE` → `REPAYING` → `COMPLETED`).

Oracle problem. Contracts cannot read off-chain data directly. ML risk scores must reach the chain via a relay, Chainlink Functions, or commit–reveal [40, 41]. CWB uses commit–reveal at MVT scope; Chainlink Functions is a stretch upgrade when supported on the chosen testnet. Chainlink price feeds supply FX rates; Proof of Reserve can verify reserve balances externally.

1.7 Proposed Solution

CWB uses a **four-tier on-chain lending model**:

- **Tier 1 — World Bank:** Holds the global reserve and allocates capital to National Banks.
- **Tier 2 — National Banks:** Borrow from Tier 1 and lend to Local Banks in their jurisdiction.
- **Tier 3 — Local Banks:** Process retail loan applications and manage the loan lifecycle.
- **Tier 4 — Clients:** Apply for loans, repay in installments, and build on-chain credit history (individual, group, or small-business).

The platform also includes risk-based borrowing limits, automatic installments above a threshold (e.g. $5,000 USDC), and planned off-chain ML (Random Forest, Isolation Forest, SHAP).

1.7.1 Banking Functions of the Platform

Six standard banking functions run across the four tiers (Figure 1.2):

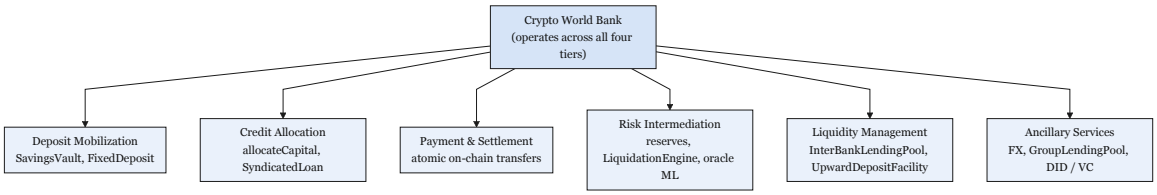


Figure 1.2: Six banking functions of the platform

- **Deposit mobilization** — `SavingsVault` with reserve checks on withdrawal.
- **Credit allocation** — downward `allocateCapital` and `SyndicatedLoan` for co-funding.
- **Payment & settlement** — atomic transfers without nostro/vostro in-transit state.
- **Risk intermediation** — reserves, approver workflows, `LiquidationEngine`, oracle ML.
- **Liquidity management** — `InterBankLendingPool` and `UpwardDepositFacility`.
- **Ancillary services** — FX oracles, `GroupLendingPool`, syndicated/tranched products, W3C DID/VC identity.

1.7.2 Service Delivery Across Scale

Global (Tier 1). The World Bank Reserve manages system-wide capital and reserve constraints with on-chain reporting.

National and regional (Tiers 2–3). National Banks allocate to Local Banks under jurisdiction-aware governance. This mirrors development-finance intermediation with faster, auditable settlement.

Retail (Tier 4). Clients use Local Bank interfaces for deposits, loans, group credit, and repayments. MFIs onboard users who may not hold wallets directly.

1.7.3 Cross-Tier Lending System

Real banking is not only top-down. Banks lend to peers (interbank markets), deposit surplus upward, and syndicate large loans. CWB encodes all three directions on-chain (Figure 1.3).

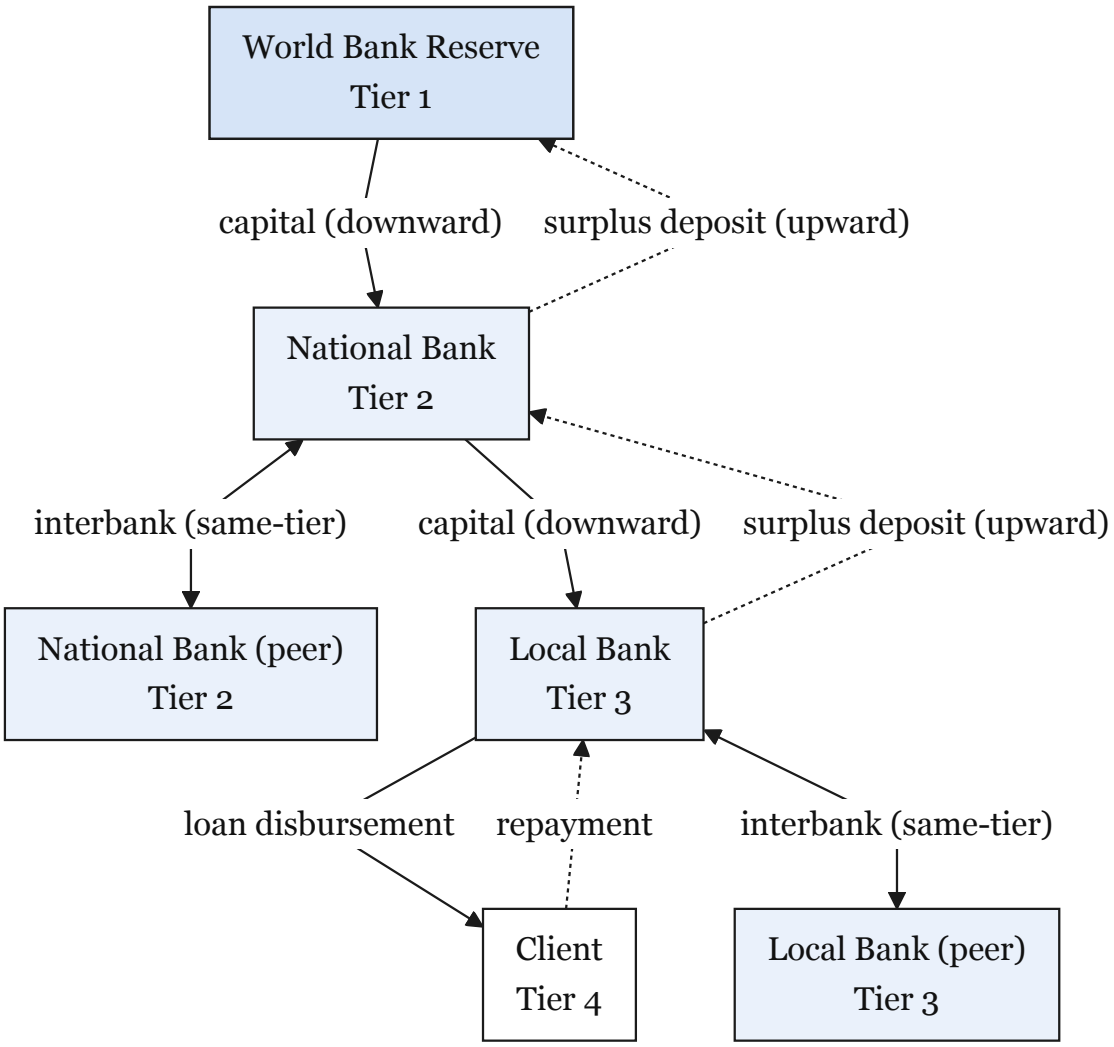


Figure 1.3: Cross-tier lending flow directions

Group co-lending and co-funding.

- **Syndicated loans** — multiple banks co-fund one large loan via `SyndicatedLoan` (Section 3.11.3).
- **Group loans** — 3–20 retail clients share collateral and mutual liability in `GroupLendingPool` (Section 3.13).
- **Tranched pools** — senior and junior tranches share one loan pool (Section 3.11.4).
- **Upward deposits** — surplus Local Bank reserves flow up via `UpwardDepositFacility` (Section 3.11.2).

Same-Tier Lending

Peer banks at the same tier share liquidity through `InterBankLendingPool`—National Bank to National Bank, or Local Bank to Local Bank—at utilization-based rates. Full specification is in Section 3.11.1; implementation is planned for Phase II.

Upward Lending

Local Banks with excess reserves deposit upward to National Banks; National Banks may deposit to the World Bank Reserve. Downward lending rates exceed upward deposit rates, creating a natural term structure across tiers.

Client Lending Path

All retail borrowers apply through a Local Bank. Clients do not access higher tiers directly. Large exposures use `SyndicatedLoan` co-funding. Downward flows and the client entry point are specified in Phases I–II; interbank and upward modules are detailed in Section 3.11.

1.8 Methodology in Brief

We combine Agile delivery with a standard SDLC frame. Pre-Thesis 1 covers requirements, architecture, and design; implementation planning runs in four final-thesis phases (Table 1.1); empirical evaluation is consolidated in Chapter 5. The full module inventory and integration paths in Chapters 3–4 are retained for research completeness; the **demonstration deployment path** documented in Sections 3.21 and 4.10 selects stability-first, zero-cost substitutes where production-grade infrastructure would add cost or schedule risk without strengthening the graded thesis claims.

Table 1.1: Four-phase implementation plan: deliverables, smart-contract scope, and deployment targets.

Phase	Primary Deliverables	Contract Scope	Deployment Target
I	Four-tier scaffold, wallet auth, frontend skeleton, PostgreSQL logical schema (51 entities; ≈13 tables migrated in Ph. I), Chainlink Proof of Reserve, off-chain KYC scaffolding	WorldBankReserve, NationalBank, LocalBank	Sepolia/Amoy (impl.); zkEVM (design pref.)
II	Loan application, approval, installment generation, borrowing-limit enforcement, deposit mobilisation, GroupLendingPool, InterBankLendingPool	SavingsVault, LoanController, GroupLendingPool, UpwardDepositFacility	Sepolia/Amoy (impl.); zkEVM (design pref.)
III	Random Forest + Isolation Forest + SHAP pipeline (commit–reveal oracle; Functions stretch), SyndicatedLoan, TranchedPool, MCP conversational agent, Foundry invariant suite, Tenderly monitoring	SyndicatedLoan, TranchedPool, FXModule, LiquidationEngine	Sepolia/Amoy (impl.); zkEVM (design pref.)
IV	Consistency pass, evaluation evidence pack, simulation outputs, testnet manifest completion	All MVT contracts (as built)	Sepolia/Amoy (impl.); zkEVM (design pref.)

Stack: Solidity 0.8.20, React 18 + TypeScript, Express.js + FastAPI, scikit-learn (ML), Circom 2.0 + snarkjs (ZKP; Future Work circuits only). Prototype *demonstration* deployments target Ethereum Sepolia as the primary testnet (Phase I) with Polygon Amoy as secondary; Polygon zkEVM remains the design-preference settlement model for production comparison (Section 3.21).

1.9 Scopes and Challenges

Scope. At pre-thesis stage, the output is formal specification and a four-phase plan. Planned testnet scope covers four-tier lending, loan approval, installments, borrowing limits, and oracle-mediated ML (RF + SHAP, Isolation Forest). Out of scope: mainnet, retail remittance, fiat ramps, production KYC/AML, and large-scale stress testing.

Challenges.

- **Limited fraud labels** — may require synthetic data or transfer learning.
- **Regulatory uncertainty** — mitigated by testnet-only deployment during the thesis.
- **Gas costs** — heavy computation stays off-chain; L2 keeps on-chain costs low.
- **Explainability** — SHAP supports audit-friendly lending decisions.
- **Economic sustainability** — interest revenue must cover gas at all tiers (Section 1.9 and Section 5.6.2).
- **Demonstration versus production gap** — mock stablecoins, trusted oracle relay, and local simulation simplify the graded prototype; Section 3.21 documents mitigations and upgrade paths without removing full-design modules from the project record.

1.9.1 Economic Sustainability: Interest Revenue Versus Gas Costs

On Polygon L2, a $25,000 loan at 8% APR yields $2,000 annual interest while full lifecycle gas stays under $0.15—well below 0.01% of interest. Oracle ML calls add $0.10–$1.00 each, negligible above $500 ticket size. Mainnet would need batching or optimization for micro-loans.

1.9.2 Resistance to Institutional Capture

On-chain design reduces capture risk through: public reserve ratios, algorithmic (not discretionary) rates, tamper-evident audit logs, open-source contracts, and programmatic borrowing caps with syndication for large exposures.

1.9.3 Stablecoin-First Lending and Supply Scalability

Retail borrowers face severe risk if loans are denominated in volatile crypto (e.g. ETH drawdowns double effective debt) [47, 48]. CWB therefore uses **stablecoin-denominated lending** at retail tiers. USDC and USDT dominate consumer DeFi usage; MiCA [58, 59] and the US GENIUS Act [60] require 1:1 reserves and issuer authorization for payment stablecoins.

Testnet may start with ETH for simplicity; loan currency is a per-pool parameter. Retail deployment will default to USDC. On public testnets, Circle's official USDC is not freely available without business verification; the demonstration path therefore deploys a `MockUSDC.sol` mintable ERC-20 (Section 3.21) while preserving USDC-denominated contract logic. BIS mBridge [61] and Project Agora [62] provide settlement rails without lending hierarchies; CWB is the lending layer that can compose on top.

Three supply measures apply: (1) ERC-20 stablecoin pools per Local Bank; (2) single-chain deployment without live bridges (after major 2022 bridge exploits); (3) four-tier reserve ratios that re-allocate existing stablecoin supply rather than create new money.

Chapter 2

Literature Review

2.1 Preliminaries

Technical terms (DeFi, smart contracts, XAI, PoS, CBDC, group lending, ZKP, Health Factor, etc.) are defined in the Glossary. Platform-specific terms (PSR, Provisional Credit Tier) appear there too. This chapter assumes basic blockchain familiarity from Section 1.6.1.

2.1.1 Review Methodology (PRISMA-Style Frame)

We reviewed literature using a PRISMA-style frame [52]. We included peer-reviewed work and formal institutional reports (2017–2026) relevant to CWB modules: DeFi lending, ML fraud detection, identity/ZKP, group lending, smart-contract security, institutional finance, regulation, or retail payments. Sources came from ACM, IEEE, Elsevier, MDPI, arXiv (venue-checked), and BIS/World Bank portals. Figure 2.1 shows screening from 892 records to 116 included entries.

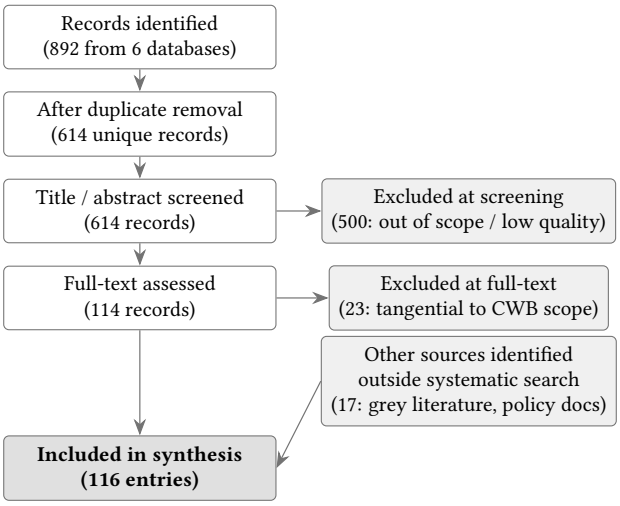


Figure 2.1: PRISMA-style literature screening flow

2.2 Review of Existing Research

Each subsection below links prior work to a CWB design choice or research gap.

2.2.1 DeFi Lending and the Structural Gap

Werner et al. [1] show that DeFi lending is flat, pool-based, and over-collateralized—with no institutional hierarchy. This is the gap Contribution 1 addresses. Bastankhah et al. [2] support dynamic rate models over static Aave/Compound curves, validating CWB's kinked-rate design (Section 3.9.1). Aspris and Svec [71] describe modular risk curators but not four-tier development finance. Yang and Cui [32] and Dao et al. [39] provide evaluation frameworks for utilization, liquidation, and credit limits in DeFi.

2.2.2 Fraud Detection and Explainable AI

Palaiokrassas et al. [3] show behavioral features raise fraud F1 to 0.76–0.85 on multichain DeFi data—informing CWB's Random Forest plan. BCCC-DeFiFraudTrans-2025 [99] is the primary training set; this pre-thesis reports a 50,000-address subsample benchmark (Section 5.2.1). Liu et al. [6] motivate Isolation Forest for unlabeled anomaly detection. Caldarelli (2025) [96] notes ML-score oracles for lending remain under-researched—supporting Contribution 3.

Adom et al. [4] favor SHAP over LIME for stable loan explanations; Yuan [38] shows SHAP credit models above 0.92 AUC. These inform the Authority Brief dashboard for approvers.

2.2.3 Hierarchical Capital Distribution

Tan [5] and BIS CBDC work [35, 80] describe two-tier central-bank-to-bank-to-user distribution—parallel to CWB's four tiers. Bindseil [80] supports asymmetric upward/downward rate spreads for `UpwardDepositFacility`. Project Pine [78] and Dong et al. [79] validate layered smart-contract factories and cross-tier capital flows in policy research, but no single DeFi spec combines them—the gap in Figure 1.3.

2.2.4 Group Lending and Microfinance

Grameen and BRAC show solidarity groups can repay above 95% without individual collateral, using weekly meetings and field officers that on-chain systems cannot replicate. CWB's `GroupLendingPool` encodes mutual liability in code with conservative default assumptions. Asaduzzaman et al. [36] report up to 60% admin savings from smart-contract microcredit. A 2025 IEEE microloan prototype [97] is the closest technical analogue. Jaffer [73] and Sherratt [74] warn that mutual liability can become coercive off-chain—a risk CWB avoids but at the cost of weaker social monitoring.

2.2.5 Smart Contract Security and Governance

Atzei et al. [7] define core EVM attack classes (reentrancy, access control), informing `ReentrancyGuard` and RBAC. Chaliasos et al. [91] find tools prevented only 8% of real exploit losses—motivating a layered Slither–Mythril–Foundry–Tenderly pipeline (Section 3.15). Flash-loan governance attacks (Beanstalk, $182M) [92] drive CWB's timelock and snapshot voting. MEV risk on liquidations is mitigated via commit–reveal triggers.

2.2.6 Supporting Infrastructure

MiCA [58, 59] and the GENIUS Act [60] require stablecoin reserves and issuer authorization—supporting USDC-first retail lending (Section 1.9.3). Beniiche [40] and Pasdar [41] frame the oracle problem as a trust, latency, and data-integrity challenge at the off-chain/on-chain boundary, supporting a single-chain prototype with commit–reveal fallback rather than a more operationally complex multi-chain oracle path.

BIS/FSB data [15, 23] document correspondent-banking delays and costs. Kim and Kim [37] and Hartmann and Hasan [33] inform gas-cost modeling. Weyl et al. [94] and MicroSave [57] motivate on-chain credit passports (Section 3.9.4). Ozili [100] shows Nigeria's eNaira had 98.5% wallet inactivity—barriers are trust-based, supporting MFI-led onboarding. Sygnum [51], mBridge [61], FundsChain [25], and Kinexys [26] show institutional blockchain adoption without retail lending hierarchies.

2.2.7 Zero-Knowledge Proofs for Compliance-Aware DeFi Identity

Piper et al. [45] and related ZKP-compliance work [46] show ZKP + DID/VC can enable privacy-preserving KYC. Together with zkAML [46], this motivates Contribution 4, deferred past MVT (Section 6).

2.2.8 Future Research Directions Scoped

GNN-based fraud detection [53, 54, 74], federated learning across banks [34, 55, 75], retail payment rails [81–90], and LLM agent risks (hallucination, prompt injection) [56] are documented in the literature but scoped to Future Work. The MVT agent uses scanning middleware and a human confirmation gate (Section 4.5).

2.3 Summary of Key Findings

Table 2.1 lists the fifteen highest-impact sources and how each maps to a CWB design decision. Table 2.2 compares CWB to major DeFi protocols. Together they support the compound gap claim for RQ1–RQ3.

Table 2.1: Tier-1 literature synthesis: core findings and design decisions.

Author & Year	Core Finding	Design Decision	Phase
Werner et al. 2022 [1]	DeFi lending uniformly flat, pool-based, overcollateralized; no institutional hierarchy	Four-tier architecture as novel design space	Contribution 1
Aspris & Svec 2025 [71]	Modular risk curator is closest prior work; lacks multilateral hierarchy	Boundary of prior work for Contribution 1	Contribution 1
Tan 2023 [5]	Two-tier CBDC distribution (central bank → banks → users)	Four-tier capital flow; client-via-Local-Bank path	Architecture
Project Pine 2025 [78]	Hierarchical smart contract factory for central-bank operations	Layered design for IBLP and SyndicatedLoan	Phase II–III
Palaiokrassas et al. 2024 [3]	Behavioral features: F1 0.76–0.85 vs. 0.08 transactional only	Random Forest feature engineering	Contribution 3
Liu et al. 2008 [6]	Isolation Forest via recursive partitioning; shorter paths = anomalies	Secondary unsupervised model when labels scarce	Contribution 3
Adom et al. 2022 [4]	SHAP deeper and more consistent than LIME on loan approval	SHAP as primary explainability method	Contribution 3
Jaffer [73] / IEEEMicroloan 2025 [97]	Solidarity mechanics + most recent blockchain microloan prototype	GroupLendingPool mutual liability design	Contribution 2
Atzei et al. 2017 [7]	Foundational EVM attack taxonomy	ReentrancyGuard, CEI, RBAC, Solidity 0.8.20	Security
Chaliasos et al. 2024 [91]	Tools prevented only 8% of real-world losses; all reentrancy	Layered pipeline—no single tool sufficient	Security
Qin et al. [92]	Flash-loan governance passes malicious proposals atomically ($182M)	Snapshot voting, 48-h timelock, 60% quorum	Governance
Piper et al. 2025 [45] + ZKP-KYC [45, 46]	ZKP + SSI + DID enables privacy-preserving KYC	Groth16 KYC circuit, W3C DID/VC choice	Contribution 4
MiCA [58, 59] + GENIUS [60]	Stablecoin issuers require 1:1 reserves, authorization, audits	USDC-first denomination as regulatory requirement	Denomination
Beniiche [40] + Pasdar [41]	Oracle boundary introduces trust, latency, and integrity trade-offs	Single-chain prototype with commit–reveal fallback and deferred DON upgrade	Infrastructure
Gudgeon et al. [42]	Utilization above 90% causes liquidity crises empirically	Kink point at $U^* = 80\%$ in rate model	Rate model

Feature	Aave	Comp.	Maker	Maple	Gfinch	CWB
Institutional hierarchy	✗	✗	✗	✗	✗	●
Cross-tier capital flow	✗	✗	✗	✗	✗	⧗
Same-tier interbank lending	✗	✗	✗	✗	✗	⧗
Syndicated / co-lending	✗	✗	✗	Part.	Part.	⧗
Upward capital repatriation	✗	✗	✗	✗	✗	⧗
Solidarity group lending	✗	✗	✗	✗	Part.	●
AI/ML fraud detection	✗	✗	✗	Man.	Man.	●
SHAP explainability	✗	✗	✗	✗	✗	●
ZKP KYC compliance	✗	✗	✗	✗	✗	●
Kinked interest rate curve	✓	✓	Gov.	✓	✗	⧗
Role-based access control	Part.	Part.	Gov.	✓	Part.	●
Institutional / L2 focus	✗	✗	✗	✓	Part.	●
TVL / Status (2026)	$26B	$1.4B	$10B	$2.6B	$680M	Planning

Table 2.2: Protocol comparison on eleven architectural dimensions. Competitor columns reflect publicly documented protocol capabilities (2026); CWB columns reflect pre-thesis specification status.

Chapter 3

System Architecture and Design

3.1 System Design Overview

This chapter documents the technical dependencies and logical design based on in-depth system analysis of the project. The chapter demonstrates a four-tier banking system with on-chain lending flows and off-chain data and AI/ML layer integration and optional conversational AI agent support. All technical specifications and logical explanations of their interconnectivity, and how they work as a complete system, have been documented here sequentially and structurally. The tables contain concise information and the figures are provided for visualization of each complicated section.

3.2 Specifications and Requirements

Table 3.1 and Table 3.2 consolidate the functional and non-functional requirements that drive the architecture in this chapter and the implementation plan in Chapter 4. The full feature inventory and MVT checklist are in Chapter 4 (Table 4.2 and Table 4.1).

Table 3.1: Functional requirements (representative subset). Full feature inventory in Chapter 4 (Table 4.2).

ID	Requirement	Description
FR-01	Four-tier hierarchy	World, National, and Local Bank contracts with role-gated downward capital allocation and client onboarding at Local Bank tier.
FR-02	Loan lifecycle	Request → ML score commit–reveal → approver decision → disbursement → installment schedule → repayment.
FR-03	Reserve enforcement	Each tier retains a minimum reserve ratio before allocating capital downward; on-chain revert on breach.
FR-04	Borrowing limits	Rolling six-month and one-year limits enforced on-chain; off-chain projection in `BORROWING_LIMIT`.
FR-05	Risk scoring	Off-chain RF + IF + stacking; SHAP explainability; immutable `LOAN_RISK_ASSESSMENT` audit row per inference.
FR-06	Group lending	`GroupLendingPool` with mutual liability, consent, and per-member share disbursement (Section 3.13).
FR-07	Event synchronisation	Blockchain event listener projects on-chain state into PostgreSQL for dashboards and agent context.
FR-08	Dual client interface	React dashboard and MCP conversational agent call the same Express.js API and contracts.

Table 3.2: Non-functional requirements.

ID	Category	Constraint / target
NFR-01	Performance	ML inference $\leq$50 ms on laptop hardware; API p95 $\leq$500 ms for read endpoints at MVT scale.
NFR-02	Security	RBAC on contracts and API; `ReentrancyGuard`; human approver gate before disbursement; agent write-path confirmation.
NFR-03	Scalability	Modular contract deployment; PostgreSQL indexing (Appendix A); L2/testnet gas target sub-cent per retail lifecycle step.
NFR-04	Maintainability	TypeScript/React/Express monorepo at repository root; OpenZeppelin primitives; documented phase gates G1–G4.
NFR-05	Usability	Mobile-first React UI; five-stage onboarding funnel; Authority Brief for approvers (Section 3.12.6).
NFR-06	Compliance (design)	Tiered KYC ladder; GDPR data minimisation (hash-only on-chain); EU AI Act Art. 86 explainability artifacts.
NFR-07	Reliability	Pause mechanism on contracts; zero-cost demo substitutes with documented upgrade path (Section 3.21).
NFR-08	Interoperability	EVM-standard contracts; WalletConnect; ERC-20/ERC-4626 patterns; MCP tool schema for agent integration.

3.3 High-Level Architecture

The system uses a 4-layer architecture: React for frontend, application services by Express.js and FastAPI, persistence by PostgreSQL, and Solidity for on-chain contracts. The layers are summarized in Table 3.3 and visually demonstrated in Figure 3.1, including how they interact.

Table 3.3: Four-layer architecture: technical components and responsibilities.

Layer	Technology	Key components	Responsibility
Presentation	React, TypeScript, WalletConnect / RainbowKit	Web UI, wallet signing, approver dashboard	User interaction and transaction initiation
Application services	Express.js, FastAPI	REST API, ML scoring service, MCP agent server, event listener	Business logic, AI/ML inference, agent tool calls, API orchestration
Persistence	PostgreSQL, Redis (cache)	Lending projections, audit logs, session and agent tables	Off-chain data storage and read models synced from chain events
On-chain	Solidity 0.8.20 (EVM)	World / National / Local Bank contracts, `LoanController`, deposit and pool modules	Authoritative banking state, RBAC, loan lifecycle, reserve rules

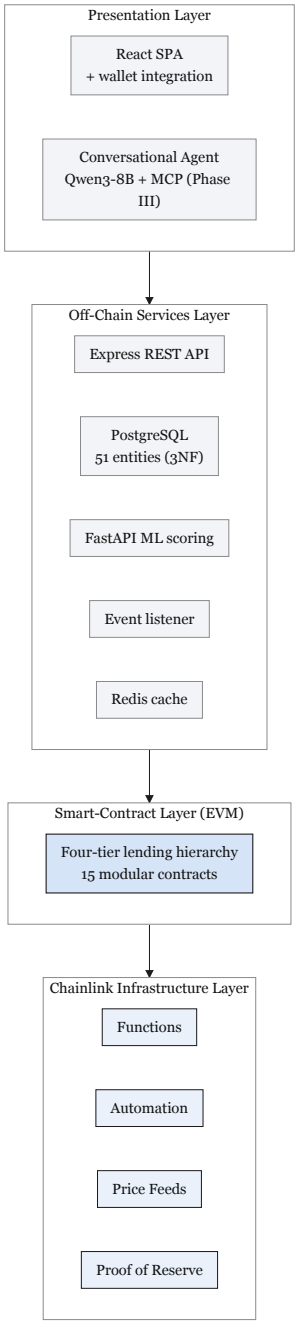


Figure 3.1: Four-layer application architecture

Figure 3.2 shows component-level interactions. Core lending contracts are specified in Phase I; extended modules are documented in Appendix B.

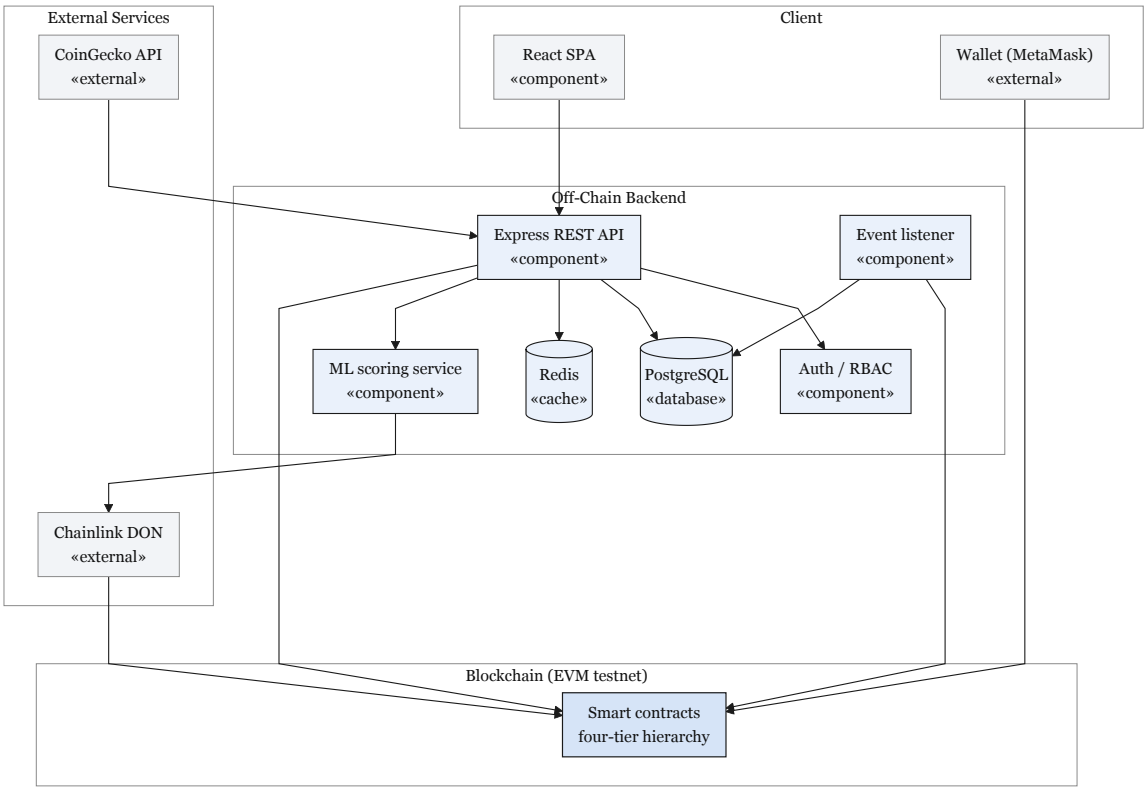


Figure 3.2: Component diagram (planning phase): layered client–server topology with PostgreSQL repository and external oracle feeds.

Figures 3.3 and 3.4 model the core lending subsystem in Gane-Sarson notation: Level-0 (Figure 3.3) shows boundary flows to five external entities; Level-1 (Figure 3.4) decomposes Process 0 into Processes 1.0–5.0 with data stores D1–D6.

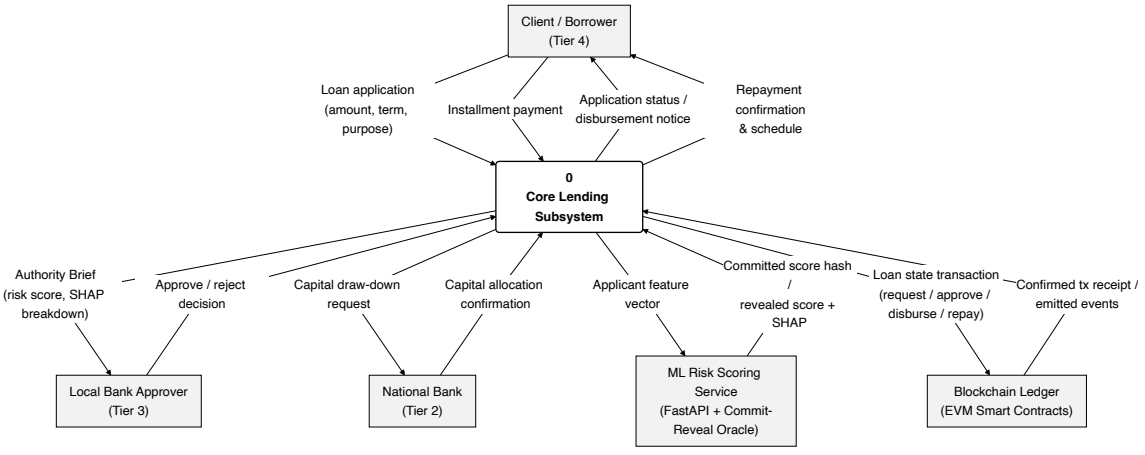


Figure 3.3: Level-0 data-flow diagram (Gane-Sarson notation): context view of the core lending subsystem and its external entities.

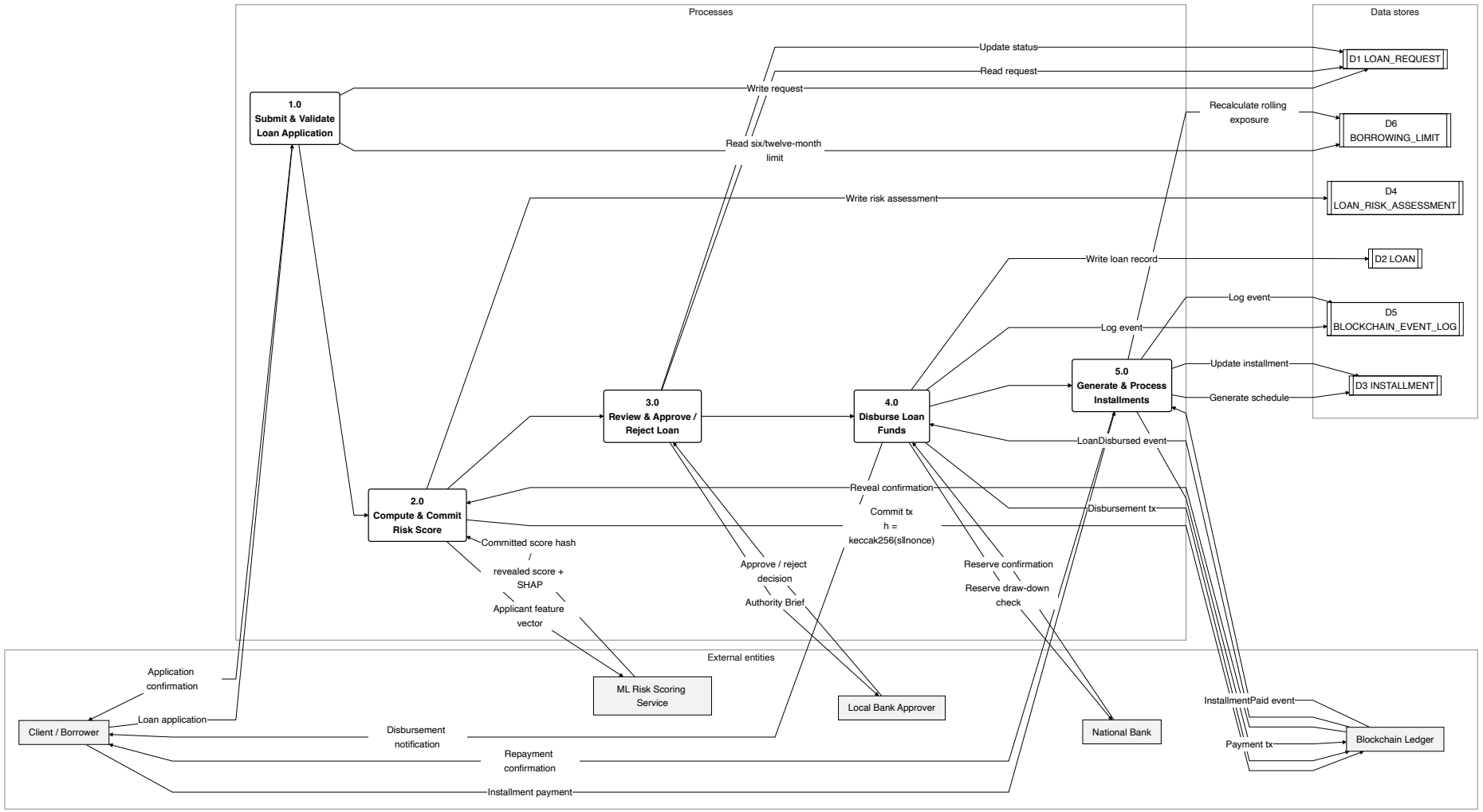


Figure 3.4: Level-1 data-flow diagram (Gane-Sarson notation): decomposition of Process 0 into Processes 1.0–5.0 with data stores D1–D6.

Figure 3.5 is the detailed UML-style component diagram: four layers, required interfaces (lollipop notation), and delegation between Express.js, FastAPI, MCP agent, persistence stores, and on-chain contract modules.

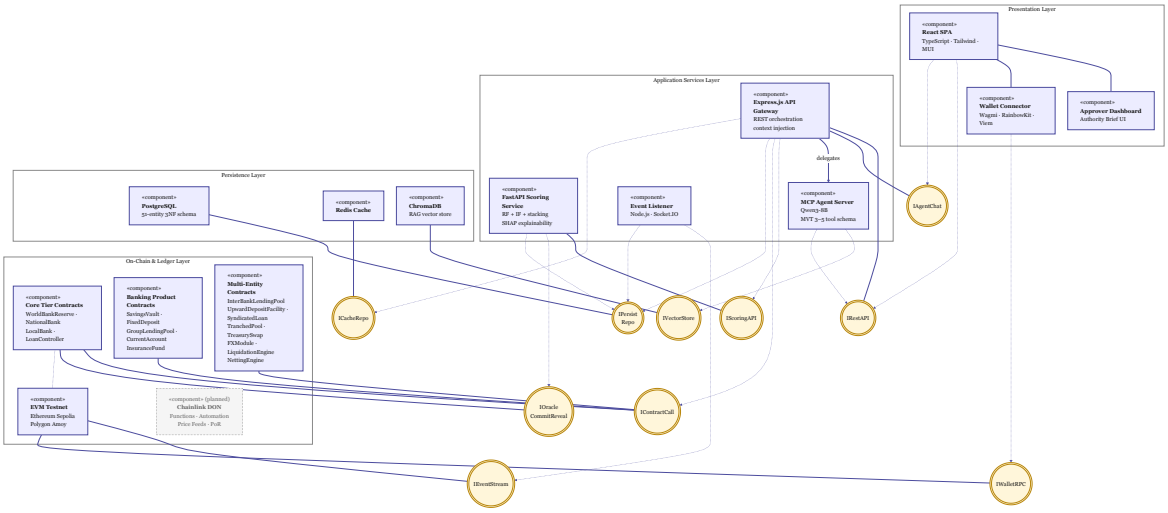


Figure 3.5: System component architecture: presentation, application services, persistence, and on-chain layers with interface dependencies and contract groupings.

Table 3.4: Smart contract inventory: seventeen modular contracts across core, product, and multi-entity modules.

Contract	Module	Status	Primary responsibility
WorldBankReserve	Tier 1 core	Specified (Phase I)	Global reserve custody, national-bank registration, downward capital allocation.
NationalBank	Tier 2 core	Specified (Phase I)	Local-bank registration, upward borrowing, downward allocation.
LocalBank	Tier 3 core	Specified (Phase I)	Client onboarding, approver management, owns `LoanController`; `freezeAccount` (Phase II+).
LoanController	Lending state machine	Ph. I shell; Ph. II	Loan request, approval, disbursement, installment state machine (DT-II.01–03); deployed by `LocalBank`.
SavingsVault	Deposits	Specified (Phase II)	Variable-yield savings, reserve-gated withdrawals.
FixedDeposit	Deposits	Stub (Phase II+)	Term deposits with deterministic early-withdrawal penalty.
GroupLendingPool	Group credit	Specified (Phase II)	Solidarity-group formation, consent, mutual liability.
FXModule	Treasury / FX	Planned (Phase III)	Oracle-priced retail FX with disclosed spread.
InsuranceFund	Risk buffer	Ph. I (often II)	Default coverage buffer; Chainlink Proof of Reserve.
CurrentAccount	Payments	Stub (Phase II+)	Transactional accounts and atomic peer transfers.
InterBankLending-Pool	Multi-entity	Specified (Phase II)	Same-tier interbank liquidity pools per tier.
UpwardDeposit-Facility	Multi-entity	Specified (Phase II)	Upward surplus repatriation with yield spread.
SyndicatedLoan	Multi-entity	Planned (Phase III)	Multi-lender syndicate consent and disbursement.
TranchedPool	Multi-entity	Planned (Phase III)	Senior/junior tranching and waterfall recovery.
TreasurySwap	Multi-entity	Planned (Phase III)	Institutional oracle-priced asset swaps.
LiquidationEngine	Risk / liquidation	Planned (Phase III)	Health-factor monitoring; third-party liquidator bonus when $HF < 1.0$ (Section 3.9.2).
NettingEngine	Multi-entity	Future Work	ERD stubs only; Section 6

3.4 Blockchain Platform Selection

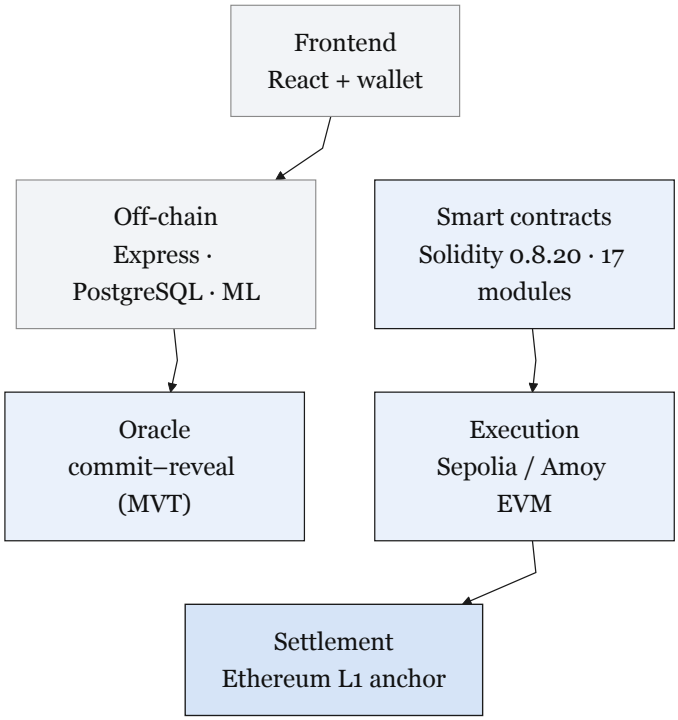


Figure 3.6: Layered blockchain and application stack

Table 3.5: Blockchain platform selection criteria and justification.

Criterion	Selection	Justification
Platform	Ethereum Virtual Machine (EVM)	Largest EVM ecosystem; mature tooling.
Network	Ethereum Sepolia (primary) and/or Polygon Amoy PoS (secondary prototype implementation)	Free public testnets; zkEVM evaluated, not default prototype chain.
Consensus	Public-testnet consensus (PoS/L1 testnet)	Stable testnet consensus; ZK settlement is a design preference only.
Smart contract language	Solidity 0.8.20	Industry standard; overflow-safe compiler; rich libraries.

Table 3.7: Blockchain platform selection (continued): operational and deployment factors.

Criterion	Selection	Justification
Gas cost	$0.001–$0.01 per transaction	Far below mainnet; viable for micro-loans.
Block finality	≈2 seconds	Fast retail UX; L1-anchored security.
Developer tooling	Hardhat + OpenZeppelin	Hardhat tests; audited OpenZeppelin libs.
Testnet availability	Amoy (Polygon PoS), Sepolia (Ethereum)	Free faucets; EVM-identical; no real funds.
Security model	Public-testnet security model	Testnet PoS; ZK settlement evaluated separately.
Migration path	EVM-compatible	Portable Solidity across EVM chains.

3.4.1 Transaction Verification and Consensus

The prototype deploys on **Ethereum Sepolia** (primary) with **Polygon Amoy** as a secondary fallback (Tables 3.5). Both are public PoS testnets with free faucets and no real-asset exposure. Polygon zkEVM Cardona remains a **design-preference** target for future ZK-anchored settlement but is not the default MVP chain. The retail loan lifecycle involves multiple on-chain state transitions per client; projected gas cost stays well below 0.01% of interest on a representative micro-loan and is validated in Phase IV (Section 5.6.2).

3.4.2 Oracle Architecture: Off-Chain AI to On-Chain Decision

Off-chain ML risk scores must reach the on-chain loan approval workflow without trusting a single party at decision time (the oracle problem [40, 41]). The graded MVT path is a **commit–reveal relay**: the FastAPI scoring service commits $h = \text{keccak256}(s \,\|\, \text{nonce})$ to `LoanController`, then reveals the score s and nonce within the decision window. The contract verifies the hash and stores s immutably, giving an audit trail and preventing post-commit score changes. A trusted backend relay may assist integration in Phases I–II only.

Figure 3.7 summarises the authoritative MVT path and the documented Chainlink upgrade routes. Table 3.9 maps each Chainlink service to its production role and the zero-cost demonstration substitute (Section 3.21; Table 3.28).

Table 3.9: Chainlink upgrade paths versus MVT demonstration substitutes.

Service	Production role	MVT / demo substitute
Chainlink Functions	DON-signed fetch of ML risk score from the off-chain service	Commit–reveal relay (authoritative at MVT scope)
Chainlink Automation	On-chain upkeep for overdue installments and interest accrual	Node.js cron calling `checkOverdueLoans()`
Price feeds	`FXModule` and fiat-equivalent UI via `AggregatorV3Interface`	`MockPriceFeed.sol` with fixed testnet answers
Proof of Reserve	External attestation of `WorldBankReserve` solvency	On-chain `getReserveSummary()` view; PoR job is Future Work

Chainlink Functions adds roughly 30–60 s latency and a per-call fee, acceptable at loan origination but unnecessary for the graded prototype. Production contract interfaces for reserve reporting and FX conversion are specified in Appendix D; integration code remains in the repository rather than in this chapter.

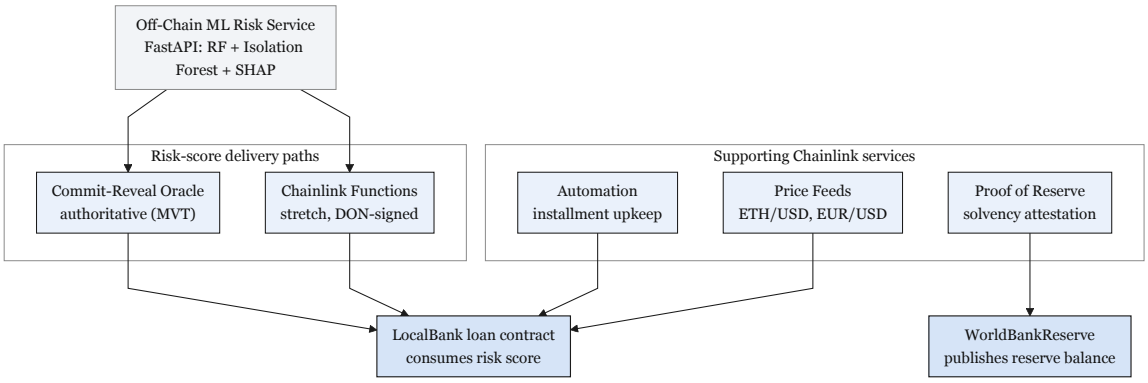


Figure 3.7: Oracle architecture

3.5 Data Model and Database Design

This section describes the PostgreSQL data model for Crypto World Bank. The complete logical schema defines **51 entities** in Third Normal Form (3NF); Table 3.10 summarises the **16 core entities** that drive the prototype, and Table 3.11 groups the remaining extension and stub relations by phase.

The graded prototype **migrates about 32 tables** across Phases I–III (Section 3.5.2); Gate G1 requires 13 tables only. The full 51-entity relational ERD is in Appendix A (Figure A.1) because of its size; Figures 3.8 and 3.9 below provide readable focused views of the core lending graph and banking-product extensions.

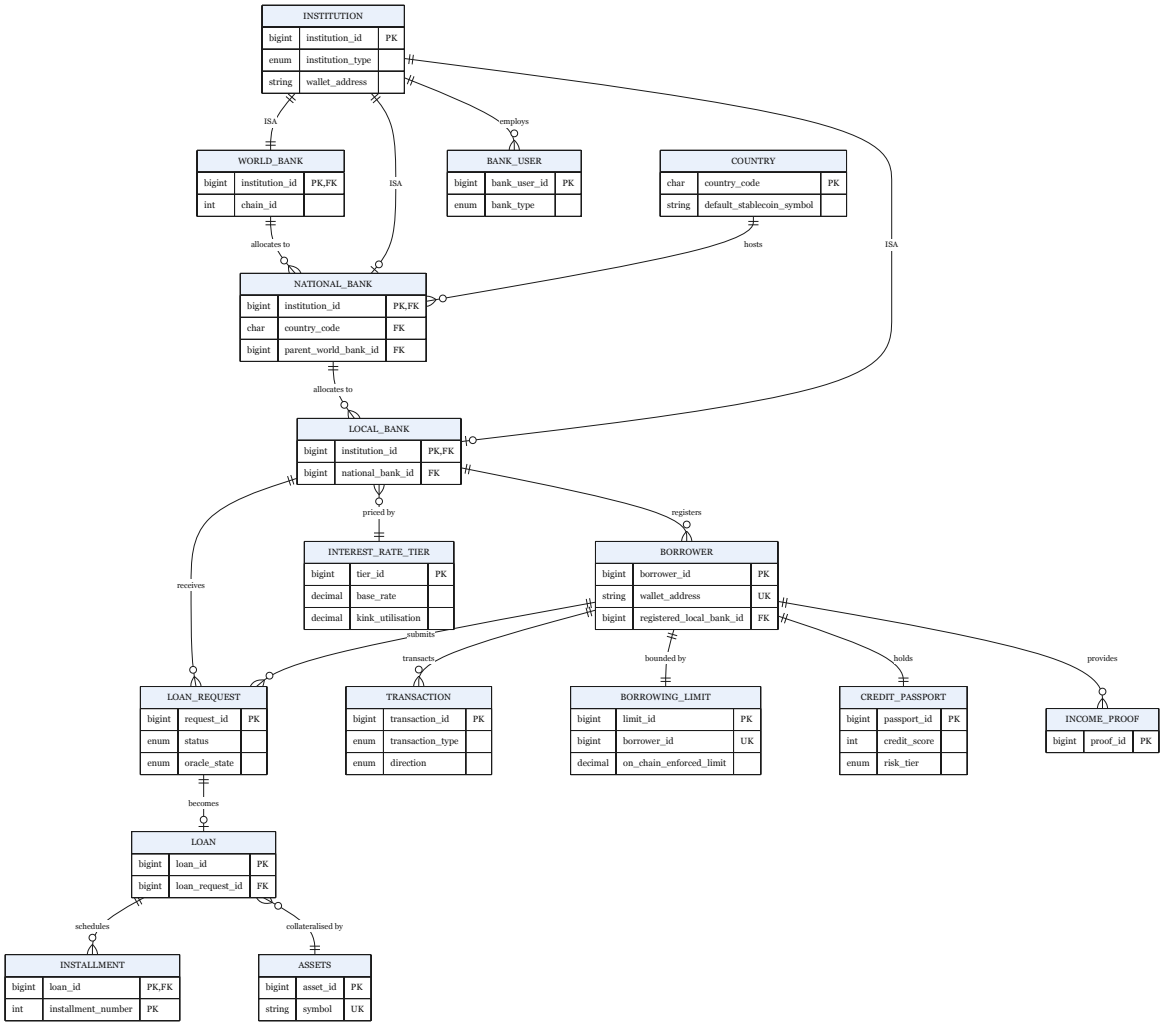


Figure 3.8: Core system entity graph

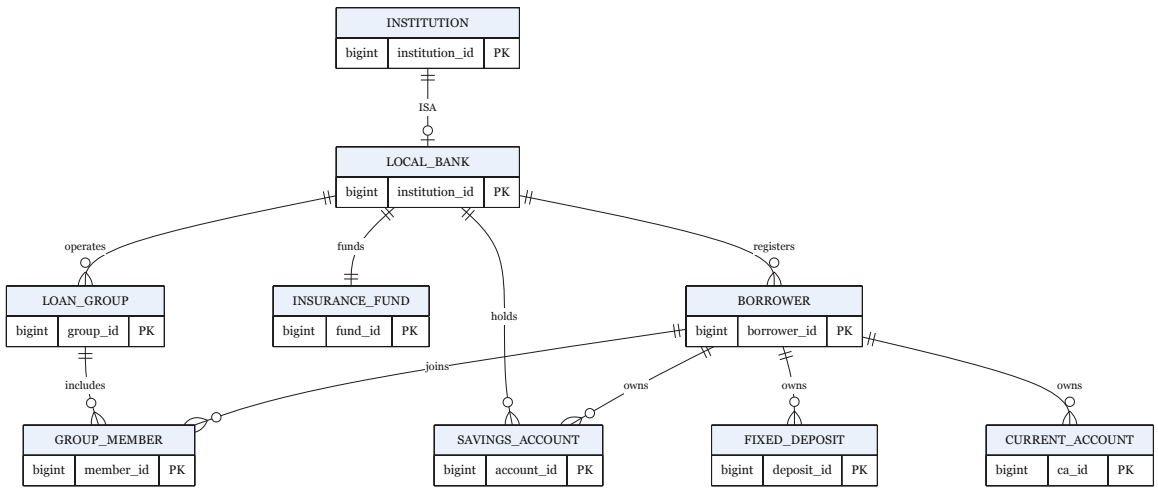


Figure 3.9: Extended ERD: banking-product entities (crow's-foot; PK shown; entity roles in Table 3.11).

Figure 3.9 documents banking-product extensions; FK anchors into core lending entities are in Table 3.23 (Section 3.11.7).

3.5.1 Entity Summary

Table 3.10: Core database entities: primary keys and roles (Phase I–III prototype scope).

Entity	Primary key	Role
Institution hierarchy (M1)		
INSTITUTION	`institution_id`	Shared supertype for World, National, and Local bank identities; enables single FK target for cross-tier operations.
COUNTRY	`country_code`	ISO 3166-1 reference; one `NATIONAL_BANK` per country (`UNIQUE`).
WORLD_BANK	`institution_id`	Global reserve subtype; singleton (`CHECK institution_id = 1`).
NATIONAL_BANK	`institution_id`	National reserve subtype; FK `country_code`, `parent_world_bank_id`.
LOCAL_BANK	`institution_id`	City-level lending institution; FK `national_bank_id`.
Actors and lending lifecycle (M1)		
BANK_USER	`bank_user_id`	Bank staff; exactly one parent institution enforced by CHECK (Table 3.15).
BORROWER	`borrower_id`	Retail client (UK: `wallet_address`); home bank via `registered_local_bank_id`; hash-only KYC fields.
LOAN_REQUEST	`request_id`	Application workflow (`status`, `oracle_state`); links borrower to local bank.
LOAN	`loan_id`	Active loan after disbursement; FK `loan_request_id`; on-chain linkage fields.
INSTALLMENT	`loan_id`, `installment_number`	Repayment schedule (weak entity on `LOAN`).
Sync, assets, and audit (M1)		
BLOCKCHAIN_EVENT_LOG	`event_id`	Canonical on-chain event cache (event-listener owned); FK target for lending tables.
ASSETS	`asset_id`	Collateral and loan asset registry (UK: `symbol`).
AUDIT_LOGS	`audit_id`	Append-only governance and compliance log (Express-owned).
Phase II–III extensions (selected)		
TRANSACTION	`transaction_id`	Financial ledger for retail and institutional flows.
BORROWING_LIMIT	`limit_id`	Off-chain projection of on-chain rolling borrow limits.
CREDIT_PASSPORT	`passport_id`	Event-listener projection of CreditPassport SBT state.
LOAN_RISK_ASSESSMENT	`assessment_id`	Per-loan ML inference record; FK `model_id` → `MODEL_REGISTRY`.
SESSIONS	`session_id`	Retail agent wallet session (Phase III); FK `borrower_id`.

Table 3.11: Deferred and extension entities (35 relations): grouped by implementation phase. Full attribute lists are omitted; see ERD figures and Section 3.5.2.

Tier	Entities	Purpose
M2	CREDIT_PASSPORT_HISTORY, INCOME_PROOF, INTEREST_RATE_TIER	SBT history, Level 2+ document hashes, normalised kinked rates
M2	GROUP_CONSENT, LOAN_GROUP, GROUP_MEMBER	Group lending (if GroupLendingPool demo chosen)
M2	INTERBANK_LOAN, UPWARD_DEPOSIT, SAVINGS_ACCOUNT	One multi-entity PoC path
M3	MODEL_REGISTRY, SECURITY_EVENT_LOG	ML lineage and system-wide security events
M3	AGENT_ACTION_LOG, AGENT_CONVERSATION_TURN, SESSION_ANCESTOR	Agent audit and session lineage
Stub	FIXED_DEPOSIT, CURRENT_ACCOUNT, INSURANCE_FUND	Deposit and risk-buffer products
Stub	SYNDICATE, SYNDICATE_MEMBER, TRANCHED_POOL, TRANCHED_POOL_SUBSCRIPTION	Syndication and tranched lending
Stub	TREASURY_SWAP, NETTING_BATCH, NETTING_ENTRY	Treasury swap and netting (Future Work)
Future	CHAT_MESSAGE, INSTITUTION_MESSAGE, PROFILE_SETTING	UX messaging and preferences
Future	MARKET_DATA, CHAINLINK_PRICE_ROUND, ORACLE_HEARTBEAT_MONITOR	Oracle caches (mock at MVT)
Future	COLLATERAL_POSITION, KYC_VERIFICATION_REQUEST, NETTING_COORDINATOR_STATE	Liquidation, ZKP KYC, settlement coordinator
Future	CHAIN_REGISTRY, SESSION_KEY_PERMISSION	Phase IV platform configuration

3.5.2 Logical Schema versus Physical Migration Tiers

The 51-entity count describes the **complete logical data model** for architectural consistency with the seventeen-contract inventory. The graded prototype does *not* require 51 physical PostgreSQL tables at Gate G1. **Phase I (M1, 13 tables)** covers the institution hierarchy, actors, lending lifecycle, and event sync—the Gate G1 minimum. **Phase II (∼12 tables)** adds ledger, credit passport, and rate-tier tables plus one optional product path. **Phase III (∼7 tables)** adds ML, agent session, and security-log tables. Remaining entities stay as ERD stubs or Future Work (Table 3.11).

Platform-configuration relations (outside the 51-entity count). CHAIN_REGISTRY (deployment chain IDs, RPC endpoints, EIP-7702 delegation contract addresses) and SESSION_KEY_PERMISSION (normalised MCP tool allowlists per session) are Phase IV platform-configuration tables. They are referenced in Sections 3.8 and 4.5 but are *not* included in the 51-entity lending schema because they hold deployment metadata rather than banking-domain state.

Table 3.12: Audit and logging table taxonomy: separate relations for on-chain events, governance actions, ML inference, and system security.

Table	Writer	Scope	Typical contents
BLOCKCHAIN_-EVENT_LOG	Event listener	On-chain receipts	tx_hash, event_name, raw_data; FK target for LOAN_REQUEST, LOAN, INSTALLMENT
LOAN_RISK_-ASSESSMENT	FastAPI ML service	Per-loan inference	fraud_probability, shap_vector, FK model_id → MODEL_REGISTRY
AUDIT_LOGS	Express API	Governance / compliance	Travel Rule packets, role grants, manual approver actions (Section 3.14)
SECURITY_EVENT_-LOG	ML / monitoring	System-wide security	AML alerts, anomaly spikes, agent injection blocks
AGENT_ACTION_LOG	Express agent hooks	Agent write tools	tool_name, confirmation_turn_id, onchain_tx_hash

3.5.3 EER Constructs Applied

Table 3.13: EER constructs applied: specialisation, hierarchy, and constraints.

EER Construct	Applied To	Notes
Specialisation (disjoint)	BANK_USER → WorldBankUser, NationalBankUser, LocalBankUser	A bank user belongs to exactly one bank type (`world`, `national`, or `local`); enforced by CHECK constraint (Table 3.15).
Generalisation / specialisation	INSTITUTION → WORLD_BANK, NATIONAL_BANK, LOCAL_BANK	ID-based table-per-type inheritance: subtype PK = FK to `INSTITUTION.institution_id`; shared identity for cross-tier FKs (Table 3.10).
Weak entity	INSTALLMENT depends on LOAN	Existence and identity determined by parent loan.
Multi-valued attribute	INCOME_PROOF per BORROWER	Modelled as separate entity to maintain 1NF.
Aggregation	LOAN_RISK_ASSESSMENT ← LOAN_REQUEST; SECURITY_EVENT_LOG ← runtime monitors	Per-loan ML scores and system-wide security events are separate append-only relations, not a conflated aggregate.
Association entity	LOAN_REQUEST links BORROWER + LOCAL_BANK	Captures many-to-many borrowing relationships; distinct from `BORROWER.registered_local_bank_id` (home bank at onboarding).
Participation constraint	BORROWER.`registe` `local_bank_id` NOT NULL at onboarding	Home Local Bank is mandatory; `LOAN_REQUEST` participation for lending services enforced at application layer.
Append-only table (policy)	AGENT_ACTION_LOG, AUDIT_LOGS, SECURITY_EVENT_LOG	DB-level INSERT-only role + Row-Level Security policy enforces that no UPDATE or DELETE is permitted on audit records.

3.5.4 Normalization

The schema has been normalized to Third Normal Form (3NF) and checked against Boyce-Codd Normal Form (BCNF):

- **1NF:** There are no repeating groups in any of the attributes. Separate entities store multi-valued attributes, such as proof of income.
- **2NF:** There are no partial dependencies. The full composite key (`loan_id`, `installment_number`) determines the non-key attributes of INSTALLMENT.

- **3NF:** No dependencies that go through other dependencies. Instead of storing redundant values like `total_borrowed` in bank entities, they are calculated at query time.
- **BCNF:** All non-trivial functional dependencies have a superkey as their determinant. For example, in `INTEREST_RATE_TIER`, `tier_id` $\rightarrow$ {`base_rate`, `kink_utilisation`, `rate_above_kink`, `max_rate`}: the determinant `tier_id` is the primary key, satisfying BCNF. Prior to the normalisation fix in this section, interest-rate parameters were columns on the three bank entities, determined by `bank_tier_id` (a non-key attribute), violating BCNF.

A transitive dependency was identified during schema review: `interest_rate_parameters` $\rightarrow$ `bank_tier_id` (interest rate values were stored as columns in the World Bank, National Bank, and Local Bank entities, making them dependent on a non-key attribute via the tier relationship). This violates 3NF. The fix is to extract these parameters into a dedicated `INTEREST_RATE_TIER` table keyed by `tier_id`, which each bank entity references via FK. This normalisation change ensures that updating a base rate does not require touching multiple bank-tier rows.

3.5.5 Relational Integrity Constraints

Table 3.15: Relational integrity constraints.

Constraint Type	Examples
Primary Key	One surrogate or composite key per entity (Tables 3.10, 3.11): e.g. `institution_id`, `borrower_id`, `request_id`, (`loan_id`, `installment_number`), …
Foreign Key	Tier subtypes → `INSTITUTION`; `BORROWER.registered_local_bank_id` → `LOCAL_BANK`; `LOAN_RISK_ASSESSMENT` → `LOAN_REQUEST`, `MODEL_REGISTRY`; multi-entity stubs → `INSTITUTION.institution_id`.
UNIQUE	`BORROWER.wallet_address`; `NATIONAL_BANK(country_code)`; `INSTITUTION.contract_address`; `LOAN_REQUEST.blockchain_tx_hash`; `BORROWING_LIMIT.borrower_id`; `ASSETS.symbol`; `BLOCKCHAIN_EVENT_LOG.tx_hash`; partial unique on active `INTEREST_RATE_TIER(bank_tier_type)`.
CHECK	`WORLD_BANK`: singleton (`institution_id = 1`). `BANK_USER`: exactly one parent FK matching `bank_type`. `TRANSACTION`: at least one of `borrower_id`, `origin_institution_id`, `counterparty_institution_id` non-null.
NOT NULL	`INSTITUTION.name`, `INSTITUTION.institution_type`; `BORROWER.wallet_address`, `registered_local_bank_id`, `kyc_level`; `status`; `oracle_state` (default `NONE`).
APPEND-ONLY	`AGENT_ACTION_LOG`, `CREDIT_PASSPORT_HISTORY`, `SECURITY_EVENT_LOG`, `AUDIT_LOGS`: INSERT-only role + RLS; no UPDATE/DELETE.
PROJECTION (RLS)	`CREDIT_PASSPORT`, `BORROWING_LIMIT`, `COLLATERAL_POSITION`: `event_listener` role only may INSERT/UPDATE; Express API SELECT-only.
FOREIGN KEY	`LOAN` → `ASSETS` (collateral, loan asset); `AGENT_ACTION_LOG` → `SESSIONS`, `AGENT_CONVERSATION_TURN`; `LOAN.disbursement_event_log_id` → `BLOCKCHAIN_EVENT_LOG`.

This integrity constraints table summarizes referential and domain constraints that keep loan, repayment, and identity records consistent. These constraints reduce operational risk by preventing orphaned records and invalid lifecycle states, which is critical for auditability in financial systems.

3.6 On-Chain and Off-Chain Data Partitioning

Table 3.17: Data partitioning between on-chain and off-chain storage.

Data Category	Storage	Rationale
Reserve balances, loan requests, approval/rejection events, repayment transactions	On-chain	Immutability, public auditability, trustless verification.
User profiles, income verification documents, chat messages, loan risk assessments, security event logs	Off-chain (database)	Data privacy, query flexibility, storage cost optimisation.
Borrowing limit computations	Off-chain with on-chain enforcement	Complex temporal aggregation; results committed as on-chain constraints via `LocalBank.updateBorrowingLimit(borrowerId, newLimit)` (see sync protocol below).
Cryptocurrency market data (non-Chainlink)	Off-chain (`MARKET_DATA`)	CoinGecko and other fallback feeds; high-frequency updates; external API dependency.
Chainlink oracle price data	Off-chain (`CHAINLINK_PRICE_ROUND`)	Each `latestRoundData()` round cached with `answered_in_round` for staleness detection; on-chain `AggregatorV3Interface` is authoritative. `MARKET_DATA` does *not* store Chainlink rounds.
Agent action execution log	Off-chain (append-only DB)	Immutable audit trail of every agent write-tool call, linked to on-chain tx hash; not stored on-chain to save gas.
Session key material	Off-chain (`sessions` table)	EIP-7702 session key hash, scope JSON, and TTL stored off-chain; the scope restrictions are enforced on-chain by the session key contract at signing time.
Session lineage chains (SESSIONS, message history, compression summaries)	Off-chain (PostgreSQL)	Full transcript history is too large for on-chain storage. The lineage chain enables searchable long-term memory without gas cost. `AGENT_ACTION_LOG` provides the on-chain audit link via confirmed transaction hashes.

Borrowing limit sync protocol. Borrowing limits are enforced on-chain, but the rolling exposure that feeds those limits is derived from off-chain transaction history. When a `LoanDisbursed` or `LoanRepaid` event arrives, the event listener first records it in

TRANSACTION and recalculates the borrower's six-month and one-year windows. If the new figure no longer matches on_chain_enforced_limit stored in BORROWING_LIMIT, the Express API submits an on-chain update through LocalBank.updateBorrowingLimit. Once the chain emits BorrowingLimitUpdated, the listener refreshes the projection row and clears any sync_discrepancy_flag, keeping the database aligned with the authoritative contract state.

Projection vs. application tables. The database splits cleanly between tables the application writes and tables the event listener maintains as read models. Projection tables—CREDIT_PASSPORT, CREDIT_PASSPORT_HISTORY, BORROWING_LIMIT, and COLLATERAL_POSITION—are populated only from chain events; Express.js may read them for dashboards and API responses but cannot insert or update them directly, a rule enforced by PostgreSQL row-level security (Table 3.15). Workflow tables such as LOAN_REQUEST, CHAT_MESSAGE, and PROFILE_SETTING remain Express-owned. LOAN_REQUEST is the one hybrid case: most columns are application-writable, but oracle_state is reserved for the listener, with column-level privileges preventing the API role from modifying it while still allowing the listener to track commit–reveal oracle progress independently of user-facing application state.

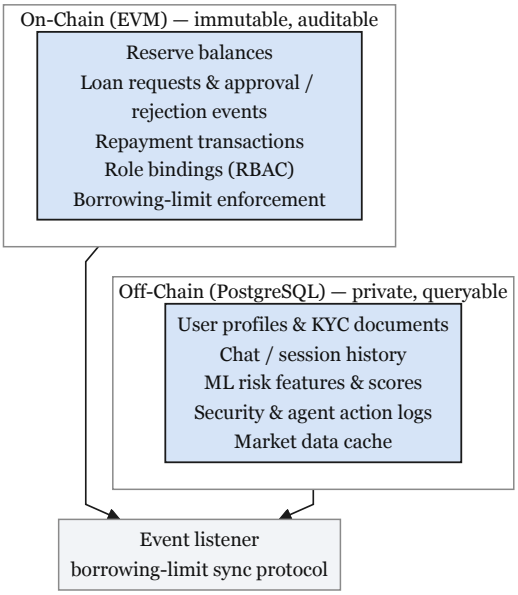


Figure 3.10: On-chain versus off-chain data partitioning

3.6.1 Financial Data Authority and Lifecycle

On-chain contract state is the **authoritative source of truth** for settlement: reserve balances, loan lifecycle transitions, role bindings, and enforced borrowing limits. PostgreSQL holds workflow metadata, PII, ML inference records, and **read-only projections** synchronised from chain events by the event listener (FR-07; Section 4.6). Where the two layers disagree after a confirmed transaction, the on-chain record prevails; projection tables exist for fast queries and dashboards, not for independent financial ledgering.

Retail loan write path. The end-to-end flow for a standard retail loan is: (1) the client submits an application (wallet-signed requestLoan on LocalBank, with optional off-chain

document hash in `LOAN_REQUEST`); (2) the ML service scores features off-chain and the commit–reveal oracle records the risk score on-chain (Section 3.4.2); (3) the Local Bank approver signs `approveLoan`, which atomically updates loan state and disburses funds; (4) the event listener ingests `LoanRequested`, `LoanApproved`, and `LoanDisbursed` events into `BLOCKCHAIN_EVENT_LOG` and updates `LOAN`, `INSTALLMENT`, and projection tables (Figure 3.22). Repayments follow the same pattern: `payInstallment` on-chain, then listener projection.

At MVT scope, loan denomination may use `MockUSDC` or native ETH on the demonstration testnet (Table 3.28); the authority rules above are unchanged—only the settlement asset differs.

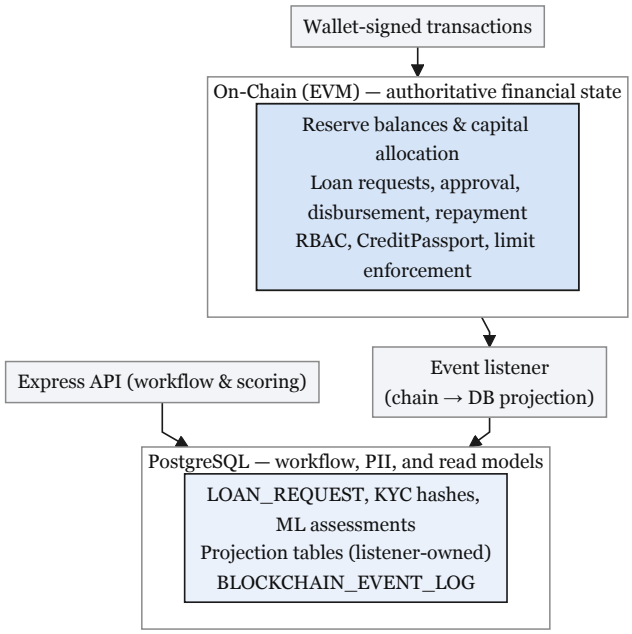


Figure 3.11: Financial data authority and lifecycle: on-chain settlement state, off-chain workflow and projections, and the event-listener bridge.

3.7 Operating Context

The Crypto World Bank is specified as a **blockchain-based institutional finance platform** on EVM-compatible infrastructure, not as a geographic pilot for a single rural market. The primary contribution is programmable hierarchical banking—reserve enforcement, tiered capital flow, on-chain auditability, and modular lending products—deployable on a stability-first prototype testnet (Ethereum Sepolia as primary, Polygon Amoy PoS as secondary). Polygon zkEVM remains an evaluated option in the platform comparison tables rather than the default prototype deployment chain narrative.

Three design choices anchor the operating context. First, **stablecoin-denominated retail lending** (Section 1.9.3) limits liability-side volatility when collateral and loans use different asset types. Second, **low per-transaction gas cost** on zkEVM L2 makes high-frequency installment and reserve operations economically viable relative to Ethereum mainnet. Third, wallet-based onboarding (Section 3.8) keeps retail access simple without changing contract role logic. Solidarity group lending (Section 3.13) is a programmable product module informed by microfinance literature (BRAC, Grameen); it is not scoped to a specific coun-

try deployment. Institutional onboarding is planned through academic pilots, open-source replication, and regulatory sandbox partnerships (Section 3.14).

3.8 User Taxonomy and Onboarding Flows

On-chain roles are bound to wallet addresses; KYC document hashes stay off-chain. Tables 3.19–3.21 and Figure 3.12 define actors and permissions. Groth16 zkKYC is Future Work (Section 6).

Table 3.19: Operational user taxonomy: KYC level, wallet type, and data residence by user type.

User Type	Tier	KYC Level	Wallet Type	Data Residence
Anonymous Visitor	—	None	None	Session only (Redis)
Registered Retail User	Tier 4	Level 1 (gov. ID + selfie)	MetaMask or optional ERC-4337	Off-chain primary; on-chain role only
Group Client	Tier 4	Level 1 + group verification	ERC-4337 abstracted	Off-chain + on-chain group contract
Local Bank Operator	Tier 3	Full institutional KYC	MetaMask (institutional)	Off-chain + on-chain role binding
Local Bank Approver	Tier 3	Full institutional + background check	MetaMask + Safe co-signer	Off-chain + on-chain role binding
National Bank Admin	Tier 2	Full institutional + regulatory license	Safe multisig mandatory	Off-chain + on-chain role binding
World Bank Admin (Governance)	Tier 1	Founding team + supervisor attestation	Safe 3-of-5 multisig	On-chain governance only

Table 3.20: Formal actor taxonomy (nine actors).

A1a	Retail Client (pre-KYC)	Browsing only; no banking transactions.
A1b	Retail Client (KYC Tier 1)	Bronze/Silver credit tier; small loans up to the KYC-1 cap.
A1c	Retail Client (KYC Tier 2)	Gold/Platinum/Diamond; larger limits, group lending.
A2	Local Bank Admin (Approver)	Loan approver; risk officer; reviews AML alerts.
A3	National Bank Admin	Capital allocator; compliance officer; SAR review; freeze authority.
A4	World Bank Admin (Governance)	Parameter governor; system operator via multi-sig.
A5	AI Agent (retail)	Phase III Must deliverable; read tools always permitted; write tools require explicit human confirmation gate before execution.
Secondary Actors (external systems or authorities)		
A6	Regulatory Authority	Read-only audit access via encrypted data package.
A7	Chainlink DON	Oracle, price feeds, automation.
A8	Blockchain Validator	Polygon zkEVM validity proof generation.
A9	External Auditor	Chainlink PoR verification.

Table 3.21: Actor permission matrix (tabular summary of primary actions).

Action	**Client**	**LB Admin**	**NB Admin**	**WB Admin**	**AI Agent**
View own loan status	YES	YES	YES	YES	READ
Submit loan application	YES	NO	NO	NO	WRITE
Pay installment	YES	NO	NO	NO	WRITE
Submit KYC documents	YES	NO	NO	NO	NO
Approve loan application	NO	YES	NO	NO	NO
Reject loan application	NO	YES	NO	NO	NO
Freeze client account	NO	YES	YES	YES	NO
Set borrowing rate	NO	NO	YES	YES	NO
Change reserve ratio	NO	NO	NO	YES	NO
Upgrade LB borrowing cap	NO	NO	YES	YES	NO
View audit logs (all)	NO	YES	YES	YES	NO
Generate SAR report	NO	YES	YES	YES	NO
View reserve summary (public)	YES	YES	YES	YES	READ

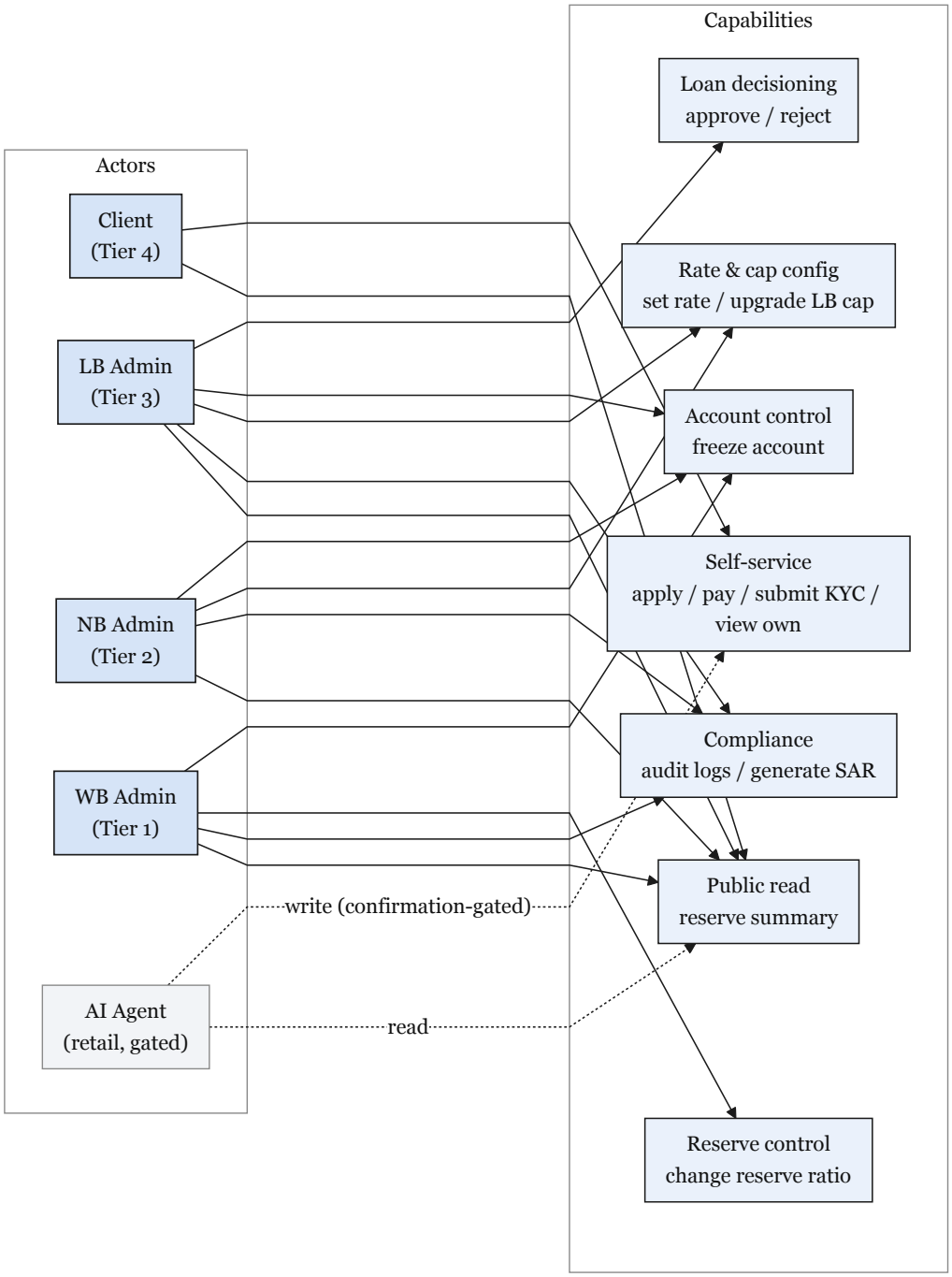


Figure 3.12: Actor permission matrix (primary actions)

3.9 Banking Modules

3.9.1 Kinked Interest Rates

Borrowing cost follows a kinked utilization model (Compound/Aave pattern [42]) with optimal utilization $U^* = 80\%$. Below the kink the rate rises gently; above it the rate jumps to incentivise repayment (Formulas 18–19, List of Formulas). Figure 3.13 shows the curve.

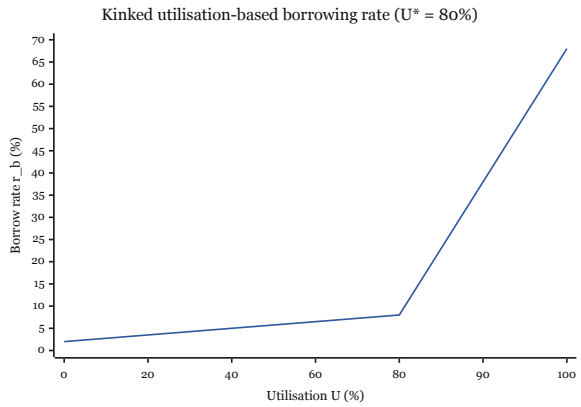


Figure 3.13: Kinked utilization-based borrowing rate

3.9.2 Liquidation Engine

`LiquidationEngine` monitors health factor (HF) and pays third-party liquidators a 5–8% bonus when $HF < 1.0$. Retail over-collateralised loans use $HF = (C \times LT)/D$; group pools use pool-level HF; credit-based loans trigger SBT downgrade instead of collateral seizure. Figure 3.14 summarises tier models.

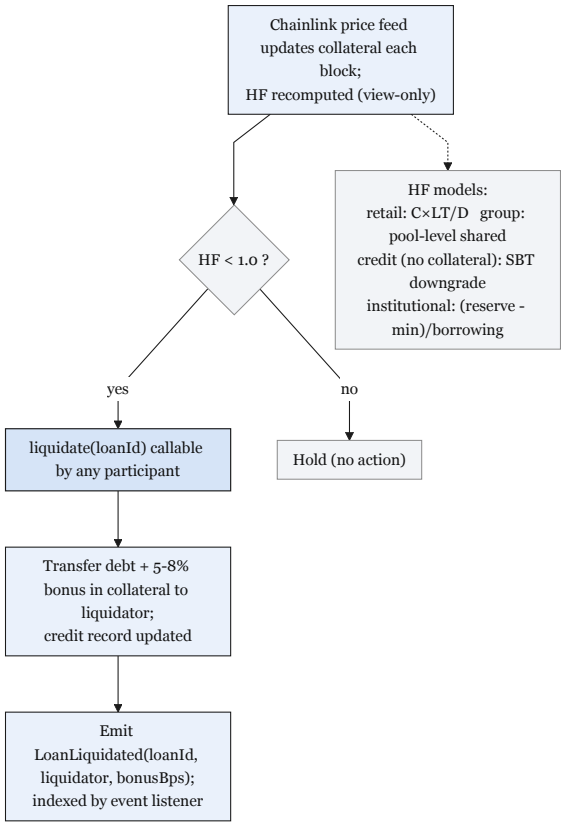


Figure 3.14: LiquidationEngine tier models and lifecycle

3.9.3 Savings and Fixed Deposits

`SavingsVault` accepts USDC deposits with utilization-linked yield (ERC-4626). `FixedDeposit` locks principal for a fixed term (ERC-7540). Interest splits among

depositor yield, `InsuranceFund`, and protocol revenue (Formula NetInterest).

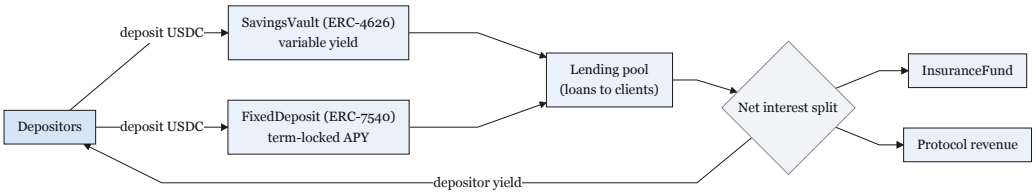


Figure 3.15: SavingsVault and FixedDeposit closed loop

3.9.4 Credit Passport (SBT)

Each KYC-verified client receives a non-transferable SBT [94] encoding repayment history (`creditScore`, `riskTier`). Table 3.22 governs loan limits by tier; the token upgrades on full repayment and downgrades on default.

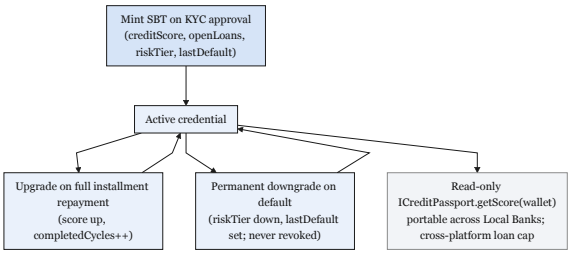


Figure 3.16: On-chain Credit Passport SBT lifecycle

Table 3.22: Credit Passport tier schedule governing maximum loan limits and interest modifiers.

Tier	Label	Score Range	Max Loan (USDC)	Interest Modifier
1	Bronze	0–299	50	Base rate
2	Silver	300–549	250	Base − 0.5%
3	Gold	550–749	1,000	Base − 1.0%
4	Platinum	750–899	5,000	Base − 1.5%
5	Diamond	900–1000	25,000	Base − 2.0%

3.10 Single-Chain Deployment (Design Revision)

An initial design considered multi-chain deployment bridging Polygon zkEVM and Ethereum Sepolia to distribute retail and institutional traffic across separate networks. This approach was revised after security analysis identified cross-chain bridges as the highest-risk attack surface in DeFi—accounting for over $1.1 billion in documented losses through Ronin ($625M), Wormhole ($320M), and Nomad ($190M) exploits in 2022 alone. The revised architecture consolidates **all live contract deployment on a single EVM chain**. Polygon zkEVM remains the design-preference target (ZK validity proofs verified on Ethereum L1), while the prototype implementation may deploy to stable public EVM

testnets such as Ethereum Sepolia (primary) and/or Polygon Amoy PoS (secondary) for operational stability, without changing the contract logic or the single-chain assumption.

Ethereum Sepolia is retained in the testing plan as a stable public EVM testnet option for prototype deployment when required, with no live asset bridge between networks. This simplification reduces attack surface and keeps Phases I–II deliverable within the project timeline. Chen et al. [31] validate multi-chain lending feasibility in the literature; the CWB adopts their throughput lessons without deploying a live CCIP or LayerZero bridge. Credit Passport SBT state, borrowing-limit enforcement, and loan lifecycle data remain single-chain on the chosen prototype testnet.

3.11 Multi-Entity and Cross-Tier Capital Operations

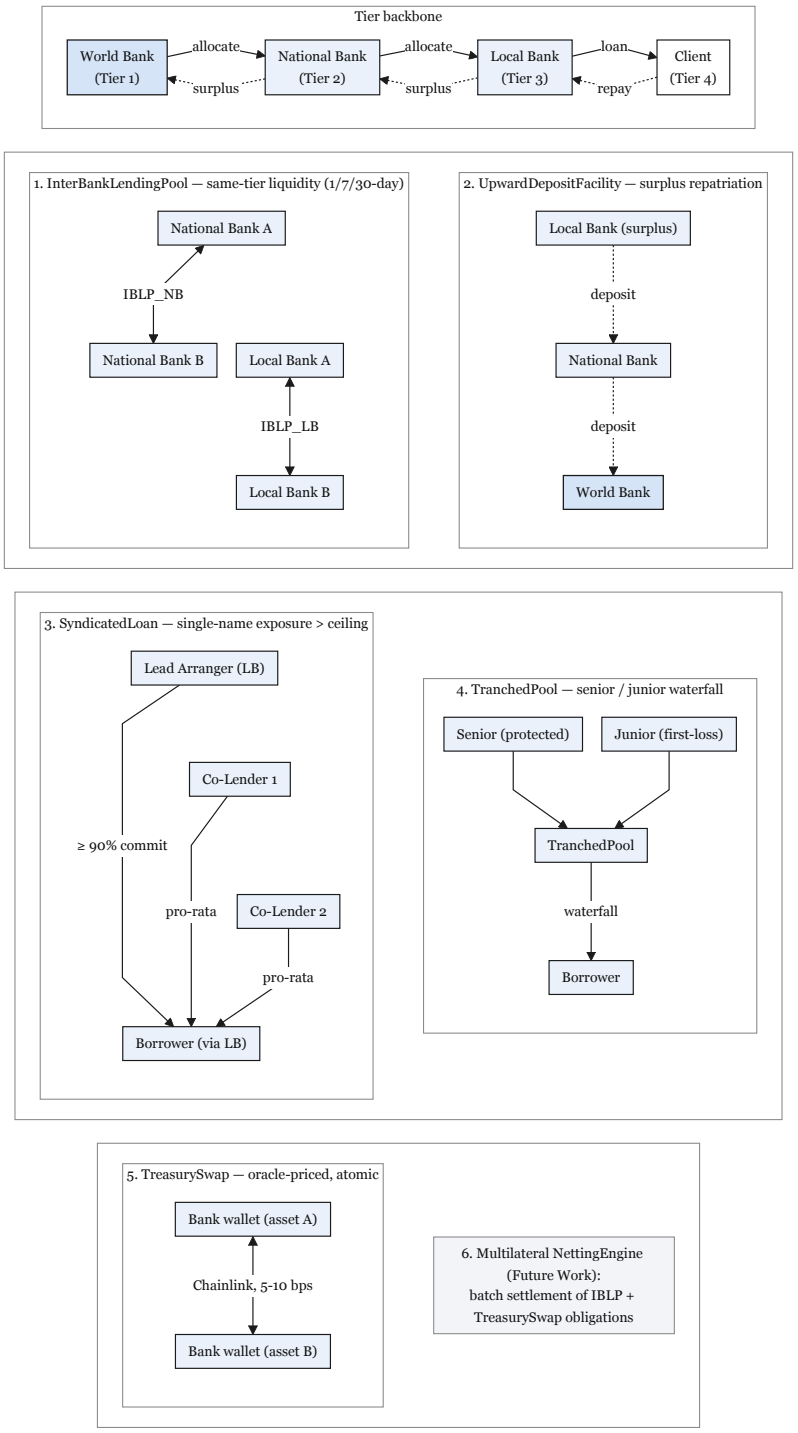


Figure 3.17: Multi-entity and cross-tier capital operations

The architecture framing of multi-directional flows promises downward, same-tier, and upward lending plus bank-level syndication for large exposures. Earlier specifications left same-tier and upward flows at design-level only. A paper that claims a *complete* banking system must specify these flows—and the genuinely multi-entity operations they enable—at the same depth as the downward flow. This section closes that gap with five concrete

contract-and-flow specifications: a fully-specified InterBankLendingPool (Section 3.11.1), upward surplus repatriation (Section 3.11.2), syndicated / club lending (Section 3.11.3), senior–junior tranched lending pools (Section 3.11.4), and cross-tier treasury FX swap (Section 3.11.5). Multilateral settlement netting (`NettingEngine`) is scoped to Future Work (Section 6). The corresponding ERD entities, contract list, implementation phase plan, and Future Work are updated downstream so the database design, software-engineering stages, and architecture remain a single consistent system.

3.11.1 InterBankLendingPool: Fully Specified Same-Tier Lending

One `InterBankLendingPool` (IBLP) per tier (`IBLP_NB`, `IBLP_LB`) supplies same-tier short-tenor (1/7/30-day) USDC liquidity among peer banks, gated by active tier role (Section 3.15).

Rate model. $r_{\text{IB}}(U)$ follows the kinked curve of Section 3.9.1 (Formulas 18–19) with kink $U^*_{\text{IB}} = 0.90$. The interbank rate is capped by the parent-tier downward rate:

$$r_{\text{IB}}(U) \leq r_{\text{downward}}(U) - \delta \tag{3.1}$$

δ is the governance-set spread that blocks arbitrage against the tier above; checked live via `getCurrentBorrowRate()` at each rate update.

Default cascade. On maturity default: (1) reserve buffer debited, (2) `InsuranceFund` drawn, (3) parent tier absorbs residual and queues role suspension via TimeLock.

3.11.2 Upward Surplus Repatriation

Banks with reserves above the minimum by 20% may deposit excess into the parent tier via `UpwardDepositFacility`, earning parent deposit yield. The upward rate is capped below the downward rate:

$$r_{\text{up}} < r_{\text{down}} - \delta \tag{3.2}$$

Withdrawals revert if they breach the depositor's minimum reserve ratio; daily withdrawals are capped at 20% of outstanding upward liabilities; net upward exposure is capped at 30% of on-chain assets.

3.11.3 Syndicated and Club Lending

When single-name exposure exceeds a bank's lending ceiling, multiple banks co-fund one loan through `SyndicatedLoan`: Lead Arranger, Co-Lenders, and Borrower (client via Local Bank only). Funding requires aggregate commitments $\geq 90\%$ of `totalAmount` and Co-Lender confirmation $\geq 75\%$ by capital share. Each lender's share is recorded as `shareBps[lender][syndicateId]`; principal and interest (and default recovery) distribute pro-rata by that share.

3.11.4 Senior–Junior Tranched Lending Pools

`TranchedPool` splits capital into senior (lower yield, protected) and junior (higher yield, first-loss) tranches. Repayment waterfall: senior interest → junior interest → senior principal → junior principal; losses reverse (junior written down first). Subordination ratio (junior/senior) is fixed at deposit between 10/90 and 30/70.

3.11.5 Cross-Tier Treasury FX Swap

`TreasurySwap` executes oracle-priced atomic cross-asset swaps for banking-role wallets:

$$\text{Amount}_{\text{to}} = \text{Amount}_{\text{from}} \times \frac{P_{\text{from}}}{P_{\text{to}}} \times (1 - \text{spread}) \tag{3.3}$$

P is the Chainlink feed price; treasury spread is 5–10 bps. Post-swap reserve ratio must remain above the tier minimum; swaps above $1M require Safe two-signature confirmation.

3.11.6 Multilateral Settlement Netting Engine (Future Work)

Future Work. A `NettingEngine` contract to compress bilateral IBLP and `TreasurySwap` obligations into batch settlement—including partial settlement (`settlePartial`) and challenge-period dispute resolution—is listed in Section 6. ERD stubs (`NETTING_BATCH`, `NETTING_ENTRY`) remain in the schema for forward compatibility but are not specified at implementation depth in this report.

3.11.7 Database, Phase, and Contract Consistency for Multi-Entity Operations

Eight ERD entities (`INTERBANK_LOAN`, `UPWARD_DEPOSIT`, `SYNDICATE`, `SYNDICATE_MEMBER`, `TRANCHED_POOL`, `TREASURY_SWAP`, `NETTING_BATCH`, `NETTING_ENTRY`) extend Figure 3.9, together with `GROUP_CONSENT` (Phase II, required before `GroupLendingPool`) and `TRANCHED_POOL_SUBSCRIPTION` (Phase II). All institution-participant FKs anchor to `INSTITUTION.institution_id` (Table 3.11). Five contracts are specified at implementation depth in this report: `InterBankLendingPool`, `UpwardDepositFacility`, `SyndicatedLoan`, `TranchedPool`, and `TreasurySwap`; `NettingEngine` is Future Work (Section 6). Extended entities include planned `on_chain_address` and `last_synced_block` columns for event-listener mapping.

Table 3.23: FK anchors connecting extended entities to core schema.

Entity	Foreign Keys into Core Schema
SAVINGS_ACCOUNT, FIXED_DEPOSIT, CURRENT_ACCOUNT	borrower_id → BORROWER; local_bank_id → LOCAL_BANK(institution_id)
LOAN_GROUP	local_bank_id → LOCAL_BANK(institution_id)
GROUP_MEMBER	group_id → LOAN_GROUP; borrower_id → BORROWER
INSURANCE_FUND	local_bank_id → LOCAL_BANK(institution_id)
INTERBANK_LOAN	lender_institution_id, borrower_institution_id → INSTITUTION; tier-type CHECK per IBLP instance
UPWARD_DEPOSIT	depositor_institution_id, parent_institution_id → INSTITUTION; adjacent-tier CHECK
SYNDICATE	lead_institution_id → INSTITUTION; borrower_id → BORROWER
SYNDICATE_MEMBER	syndicate_id → SYNDICATE; member_institution_id → INSTITUTION
TRANCHED_POOL	sponsor_institution_id → INSTITUTION
TRANCHED_POOL_SUBSCRIPTION	pool_id → TRANCHED_POOL; subscriber_id → BANK_USER
TREASURY_SWAP	initiator_institution_id → INSTITUTION; from_asset_id, to_asset_id → ASSETS
NETTING_BATCH	coordinator_id → BANK_USER
NETTING_ENTRY	batch_id → NETTING_BATCH; participant_institution_id → INSTITUTION
INSTITUTION_MESSAGE	sender_institution_id, receiver_institution_id → INSTITUTION
LOAN_RISK_ASSESSMENT	loan_request_id → LOAN_REQUEST; model_id → MODEL_REGISTRY
GROUP_CONSENT	loan_request_id → LOAN_REQUEST; member_id → GROUP_MEMBER
COLLATERAL_POSITION	borrower_id → BORROWER; loan_id → LOAN; asset_id → ASSETS
BLOCKCHAIN_EVENT_LOG	Referenced by: LOAN_REQUEST.submission_event_log_id, approval_event_log_id; LOAN.disbursement_event_log_id; INSTALLMENT.payment_event_log_id

3.12 System Modeling

This section presents the system analysis diagrams that model the platform's structure, behavior, and data flow. Loan state transitions are formalised in Section 3.4.2; the diagrams

below show actor interactions and operational flows at the use-case and sequence level.

3.12.1 Use Case Diagram

The system involves nine actors across eleven representative use cases (Figure 3.18). Four **core primary** actors initiate banking operations: Retail Client (A1, three KYC sub-types), Local Bank Admin (A2), National Bank Admin (A3), and World Bank Admin (A4). A5 (AI Agent) is a **planned Phase III primary** actor with restricted write permissions gated by the human confirmation protocol (Table 4.1, item 11). Four secondary actors are external systems or authorities: Regulatory Authority (A6), Chainlink DON (A7), Blockchain Validator (A8), and External Auditor (A9). Note that the primary actor classes in the use-case diagram are a condensation of the formal nine-actor taxonomy defined in Table 3.20: Visitor and Registered Retail User are pre-conditions for the CLIENT (BORROWER) role rather than independent diagram actors, and Local Bank Operator / Approver are merged into the Bank Approver role at the diagram level.

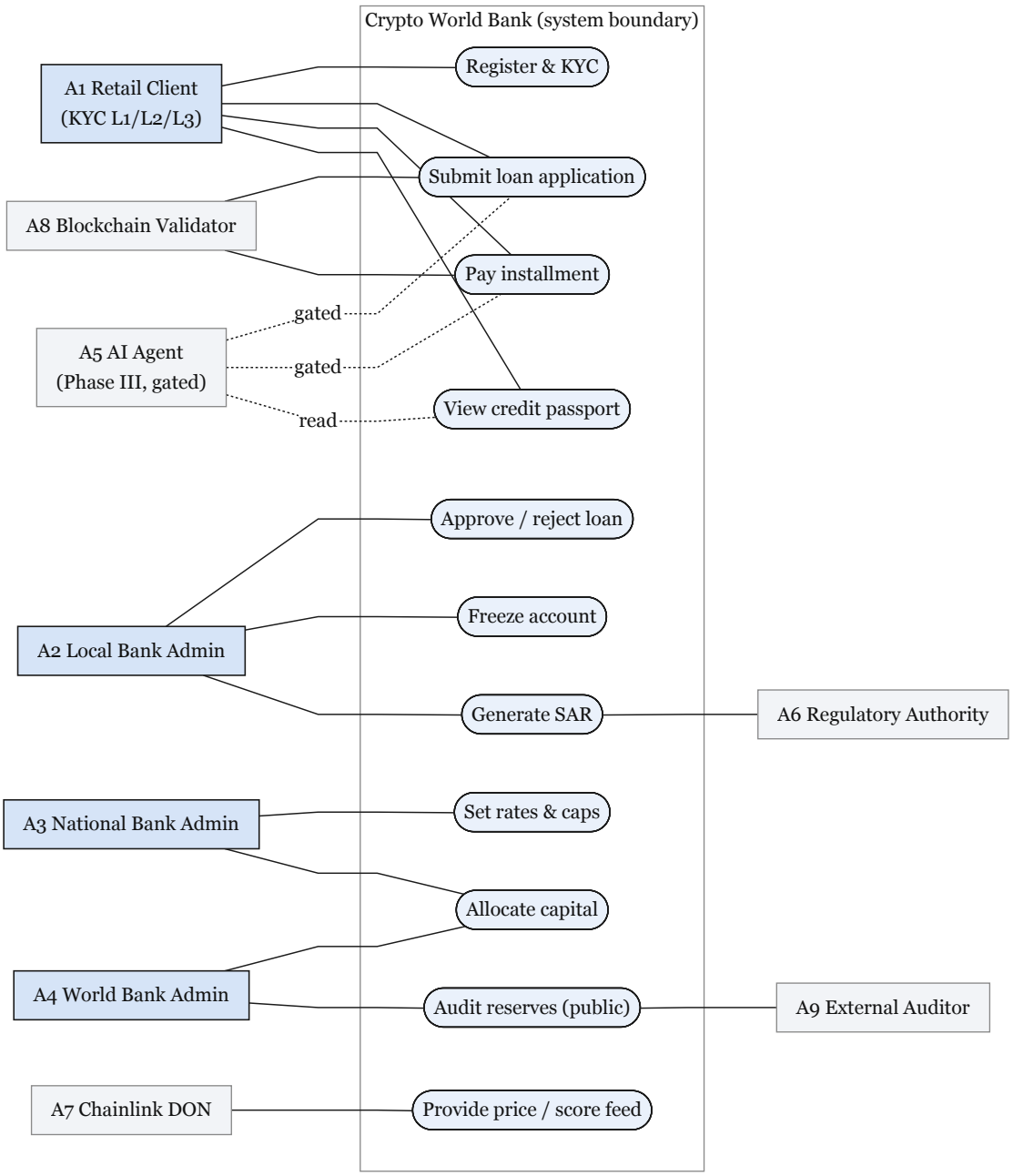


Figure 3.18: Use-case diagram (nine-actor taxonomy)

3.12.2 Activity Diagrams

Figures 3.19–3.21 present three end-to-end activity flows. Each diagram has a single start and a single end node; decision diamonds route exceptional paths (limit exceeded, rejection, verification retry) back to the terminal state without spawning parallel lifelines.

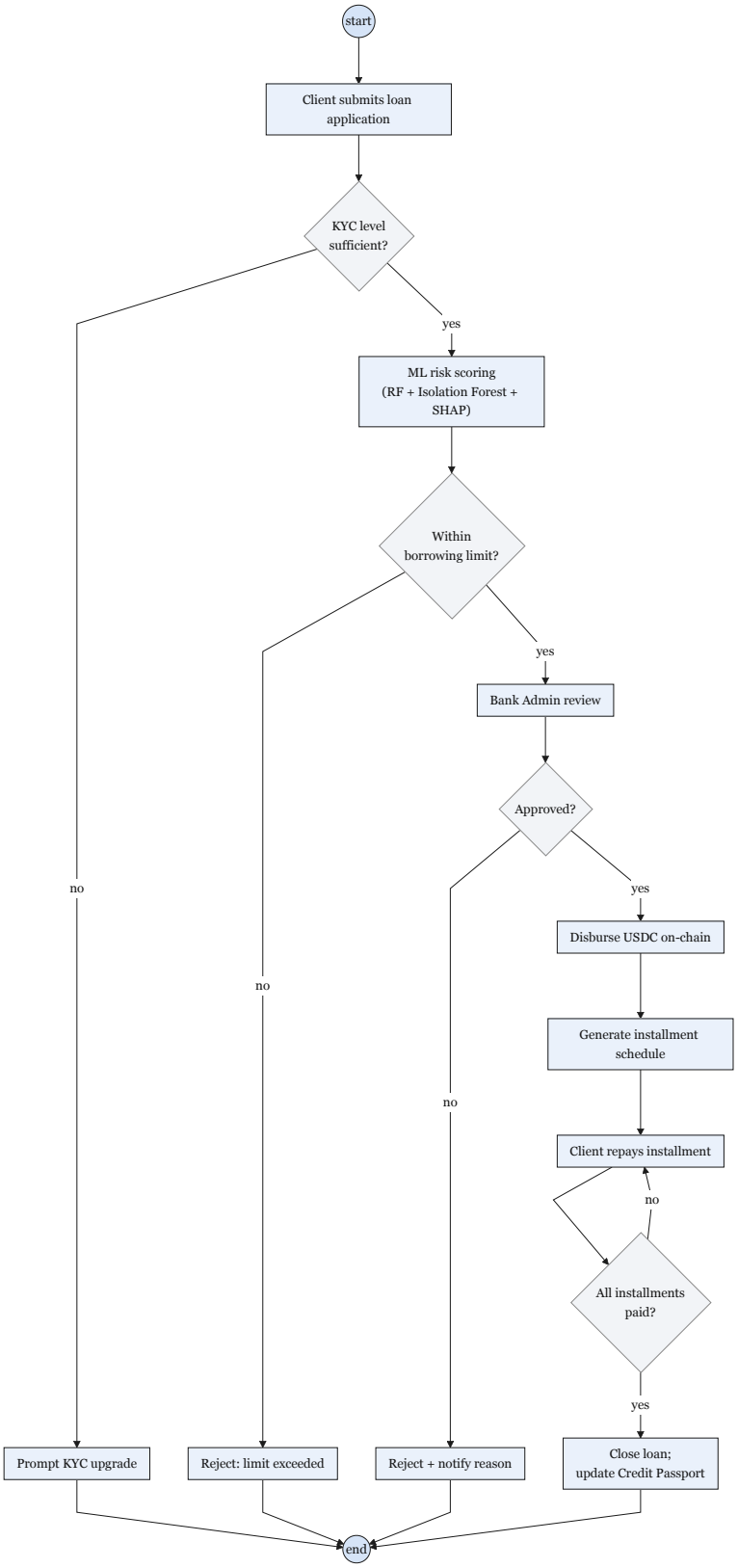


Figure 3.19: USDC loan lifecycle activity flow

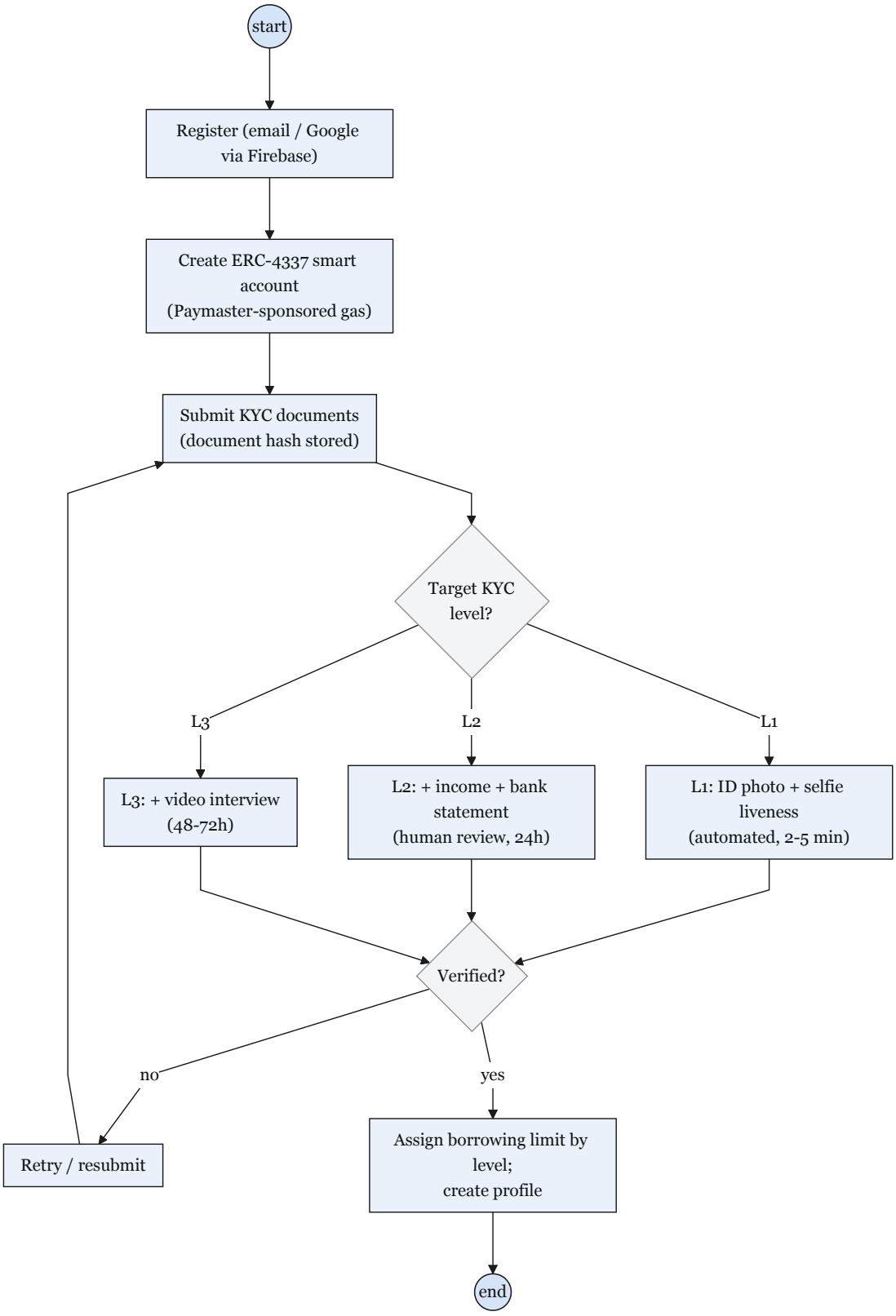


Figure 3.20: Retail onboarding and identity activity flow

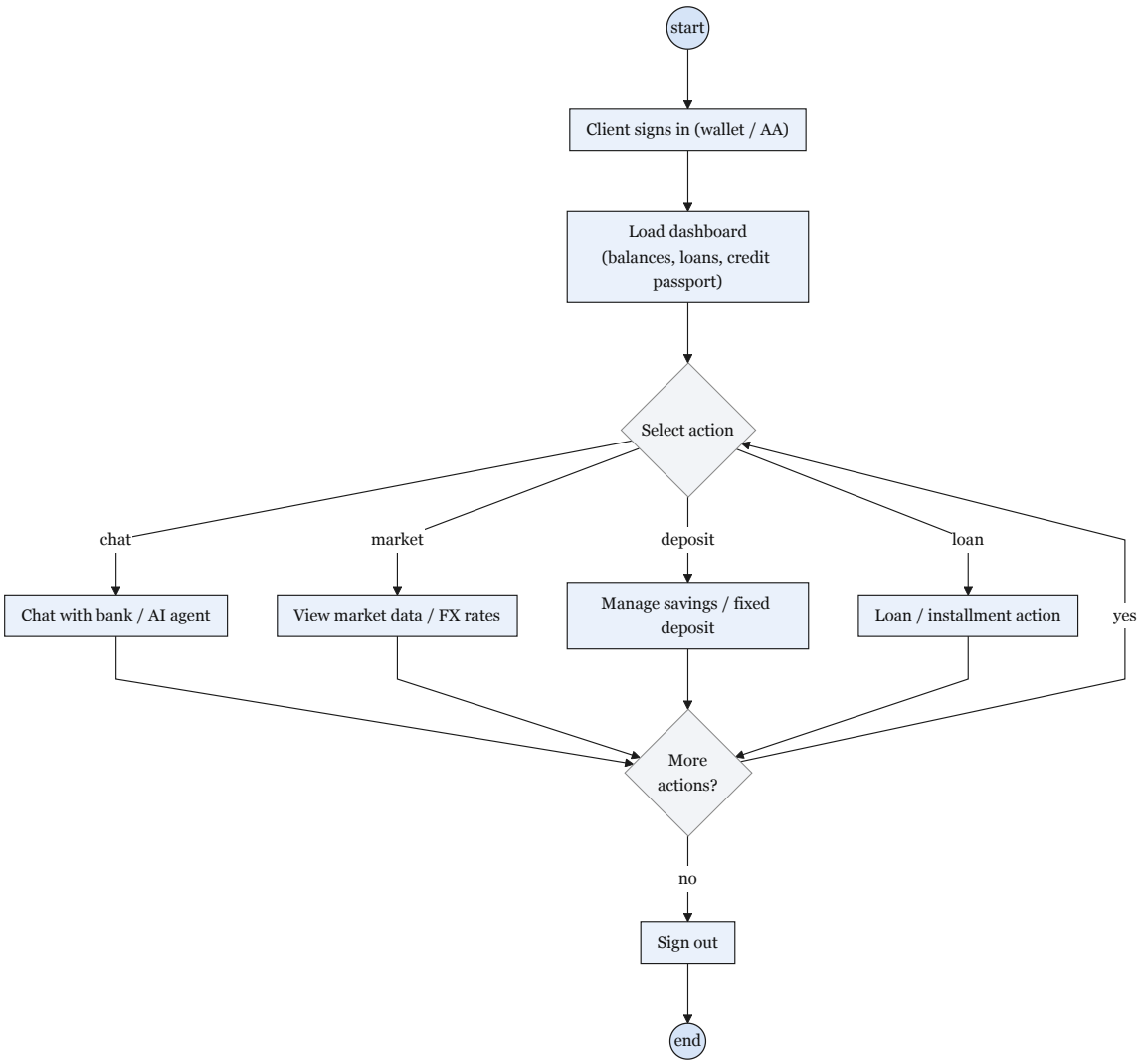


Figure 3.21: Client banking session activity flow

SAR activity flow. The Suspicious Activity Report (SAR) workflow is triggered when the Isolation Forest anomaly detector (Section 3.16) flags a wallet with an anomaly score above the 0.75 threshold. The five-step flow is:

1. Isolation Forest flags the wallet: `anomaly_score > 0.75`.
2. `LOAN_RISK_ASSESSMENT` records the per-loan detection (FK `model_id` → `MODEL_REGISTRY`); `SECURITY_EVENT_LOG` records the system-wide AML alert event.
3. Express.js backend emits a planned event-bus message (`aml-alert` topic); the compliance dashboard consumes it.
4. Bank officer reviews the alert in the admin compliance queue:
   - **False positive:** Dismiss and document reason (audit trail in `audit_logs`).
   - **Confirmed:** Generate SAR record in `audit_logs`; insert `INSTITUTION_MESSAGE` (`message_type='SAR_ESCALATION'`) to notify the tier above (National Bank); freeze wallet via `LocalBank.freezeAccount(clientId)`.
5. The `freezeAccount` function is gated by the `onlyApprover` modifier and is Foundry-tested (Phase III invariant suite). A frozen wallet cannot initiate new loan disbursements or installment payments until the freeze is lifted by the tier above.

3.12.3 Data Flow Diagrams

The core lending data flows are specified in Figures 3.3 and 3.4 (Section 3.3). Level-0 defines subsystem boundary flows to the Client, Local Bank Approver, National Bank, ML Risk Scoring Service, and Blockchain Ledger. Level-1 expands Process 0 into submit/validate (1.0), risk scoring with commit–reveal oracle (2.0), approver review (3.0), disbursement (4.0), and installment processing (5.0), reading and writing stores D1–D6 (`LOAN_REQUEST`, `LOAN`, `INSTALLMENT`, `LOAN_RISK_ASSESSMENT`, `BLOCKCHAIN_EVENT_LOG`, `BORROWING_LIMIT`).

3.12.4 Sequence Diagrams

Figures 3.22–3.25 consolidate the platform's nine interaction flows into four panels: loan approval with reject alternative, installment and income, hierarchical banking with market data and borrowing-limit, and chat with AI-chatbot.

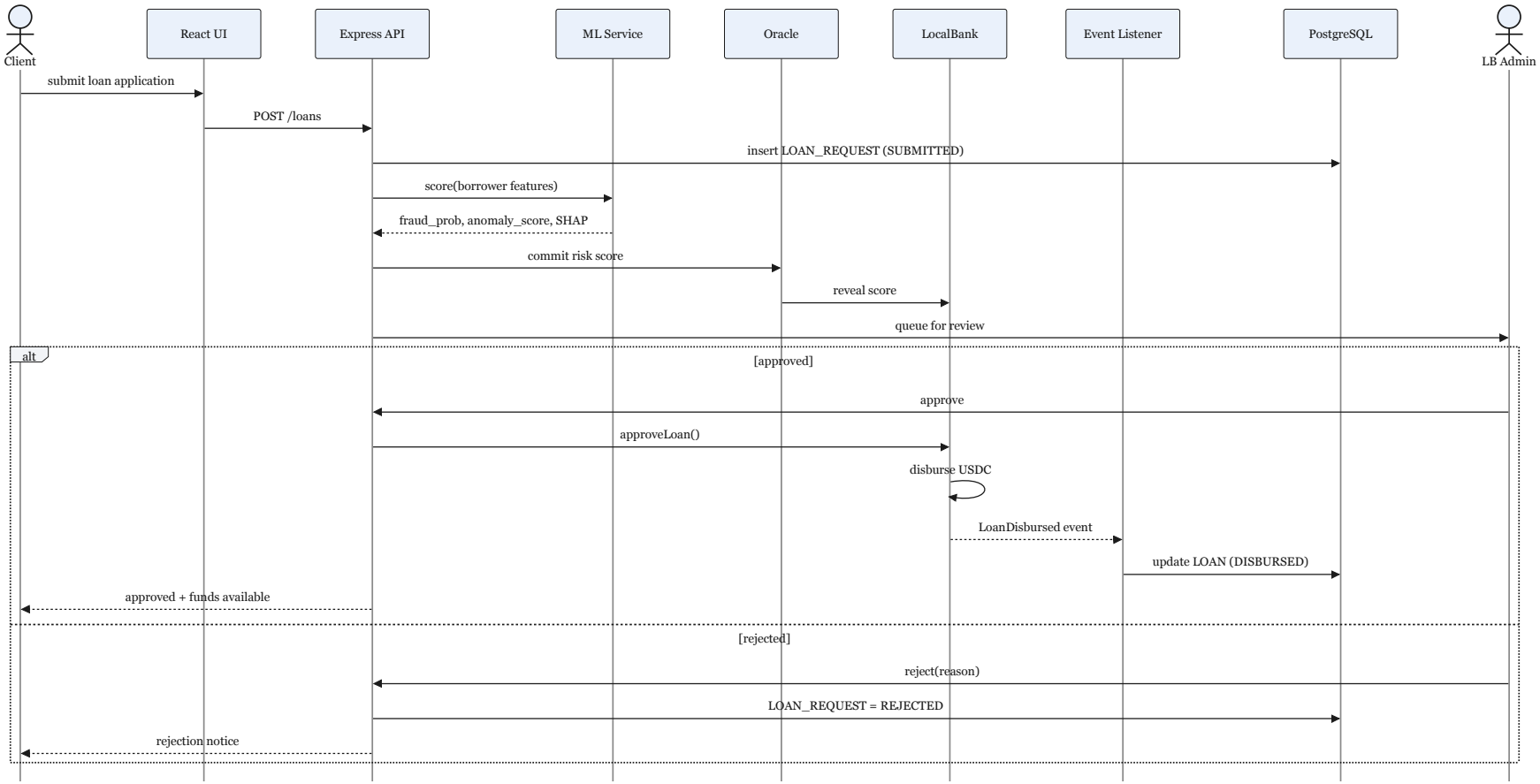


Figure 3.22: Loan approval sequence: application submission, ML scoring, commit–reveal oracle, approver review, and on-chain disbursement or rejection.

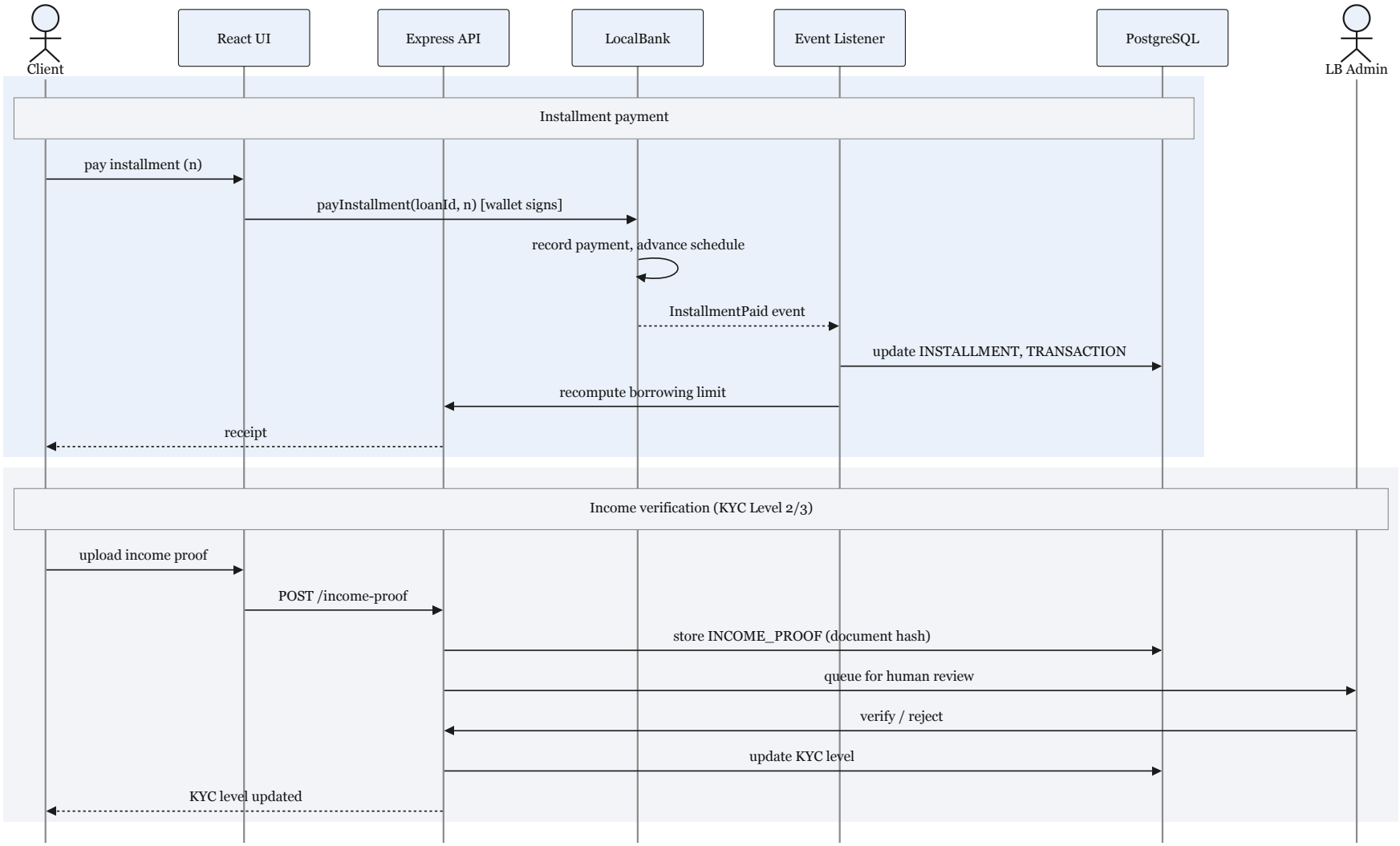


Figure 3.23: Installment payment via wallet-signed `payInstallment` with listener projection, and income-proof upload for KYC Level 2/3 officer review.

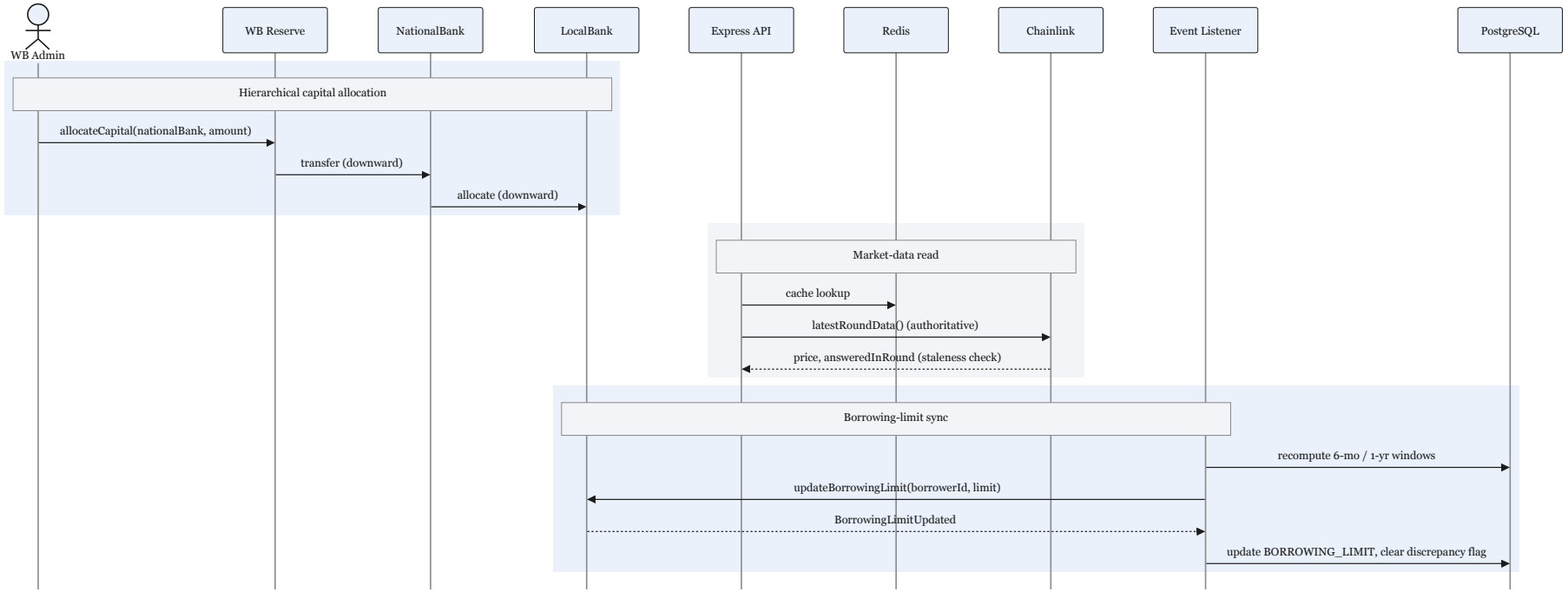


Figure 3.24: Downward capital allocation across tiers, Chainlink price read with Redis cache, and event-driven borrowing-limit synchronisation to PostgreSQL.

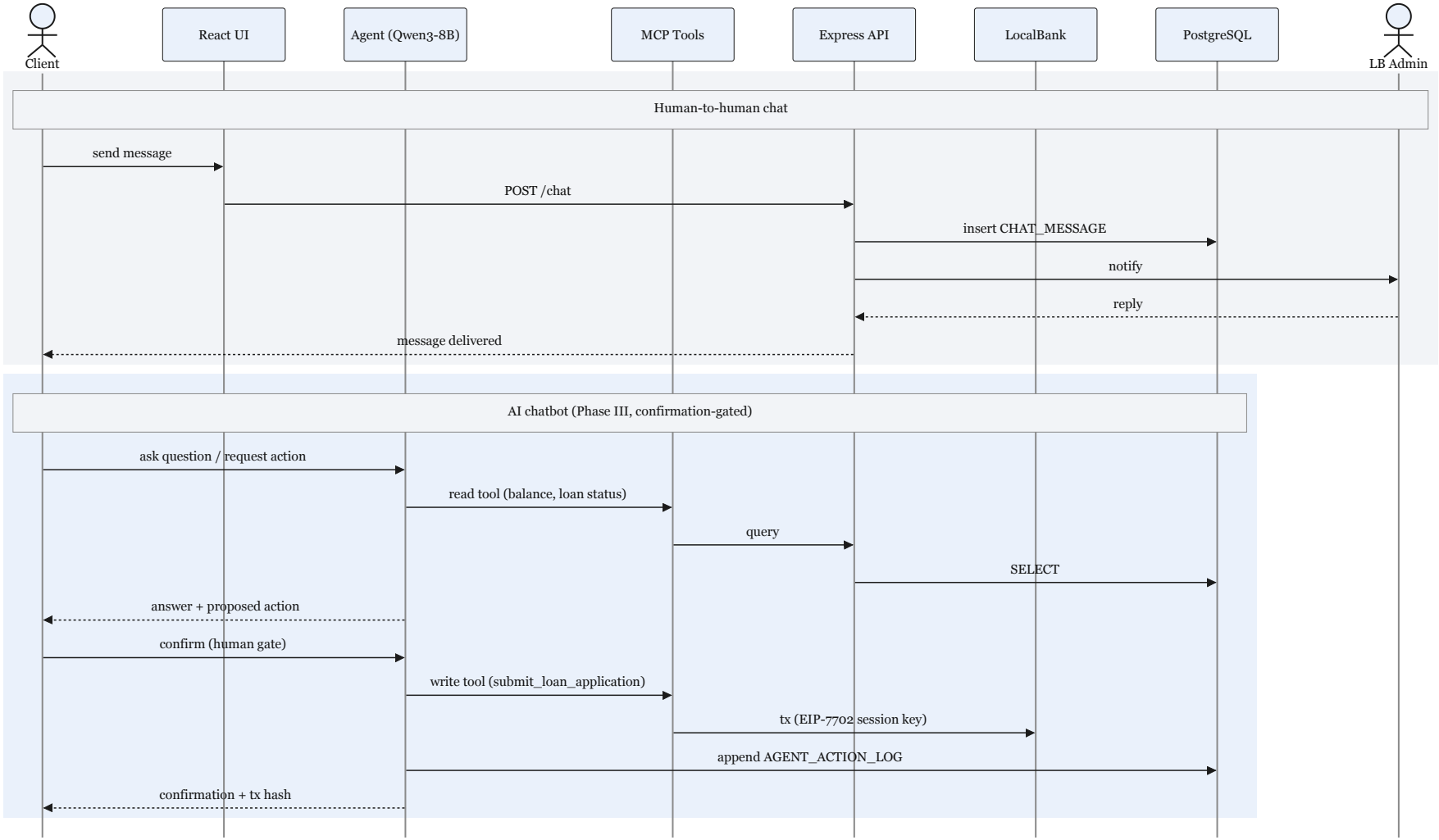


Figure 3.25: Human client–officer messaging through the Express API, and Phase III confirmation-gated conversational agent with MCP read/write tools.

3.12.5 UML Class Diagram

Figure 3.26 is a **Software Engineering** UML class diagram for the *core lending domain*. It is not another system-architecture drawing (those are Figures 3.1 and 3.2). Instead it answers: *which software classes exist, what data they hold, and how they relate*—the same question CSE470 asks when moving from requirements to implementation.

The diagram has three layers of classes:

- **Institution hierarchy** — `Institution` is an abstract supertype; `WorldBankReserve`, `NationalBank`, and `LocalBank` are its subtypes (generalisation), matching Figure A.1 and the Solidity contracts in `contracts/`.
- **Retail lending** — `Borrower` and `LoanRequest` mirror PostgreSQL tables; `LoanController` is the on-chain state machine that enforces the oracle gate and disbursement rules.
- **Off-chain services** — `LoanService` (Express.js) and `ScoringService` (FastAPI) are controller/service classes that the React UI calls; they delegate to contracts and the ML pipeline without duplicating business logic in the frontend.

Why include it? Use-case and sequence diagrams (above) show *who talks to whom over time*; the class diagram shows *the static object model* that developers implement. Examiners assessing System Analysis and Design and Software Engineering coursework can trace each class back to a requirement in Table 3.1.

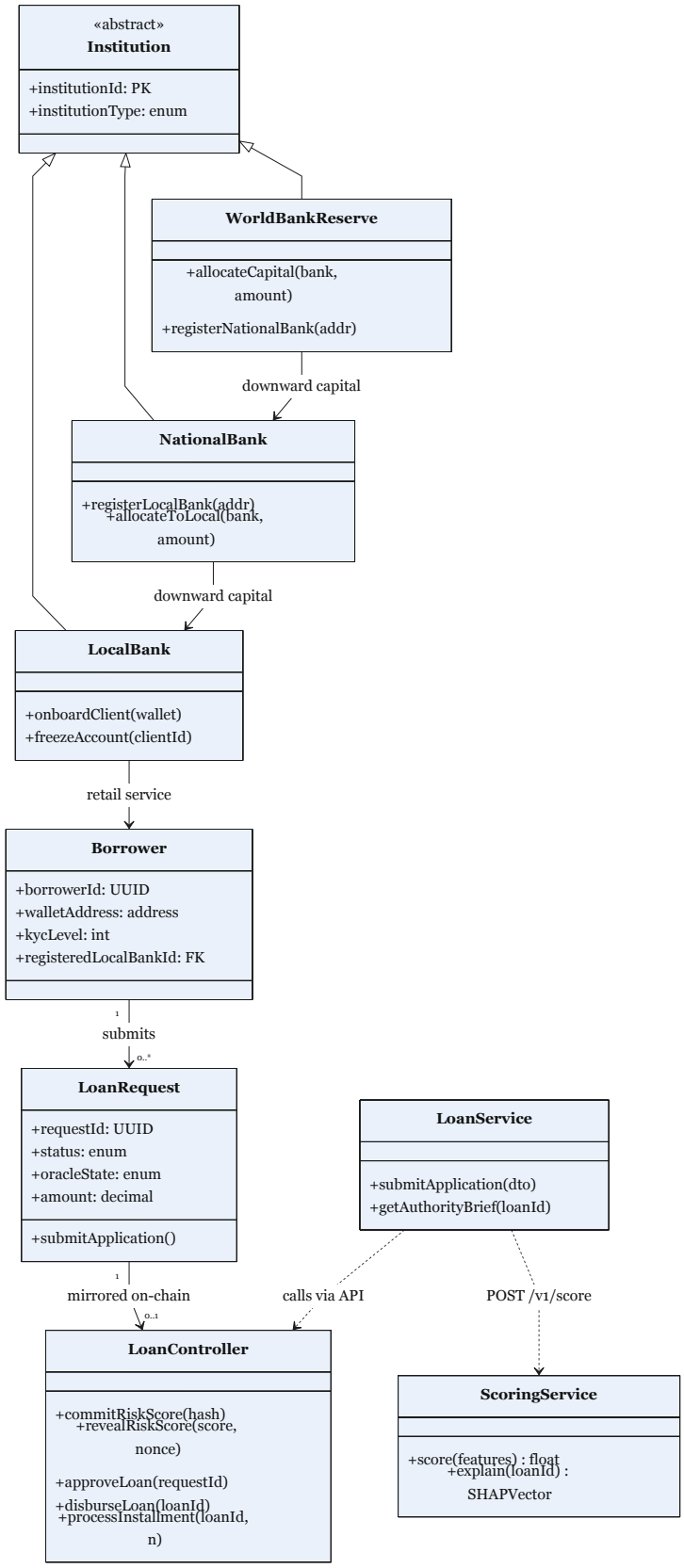


Figure 3.26: UML class diagram (core lending domain): institution hierarchy, loan entities, on-chain controller, and off-chain services.

3.12.6 User Interface Design

The retail and approver interfaces follow a mobile-first Material Design 3 layout implemented in `frontend/` (React 18, TypeScript, Tailwind, MUI). Two primary surfaces are specified:

- **Retail loan flow** — wallet connect, KYC status banner, loan application form with plain-language APR and installment preview, and post-submission status tracking.
- **Authority Brief dashboard** — approver view with composite risk score s, SHAP waterfall chart, anomaly deviation table, and approve/decline actions gated on oracle `SCORE_REVEALED` state.

High-fidelity Figma prototype exports (screen flows, component layout, and approver dashboard) are collected in **Appendix F** (Section D) so the main chapters stay specification-focused; Figure D.1 and Figure D.2 will be inserted there before final submission.

3.12.7 Four-Tier Capital Flow

Figure 3.27 illustrates the hierarchical capital flow and cascading repayment structure.

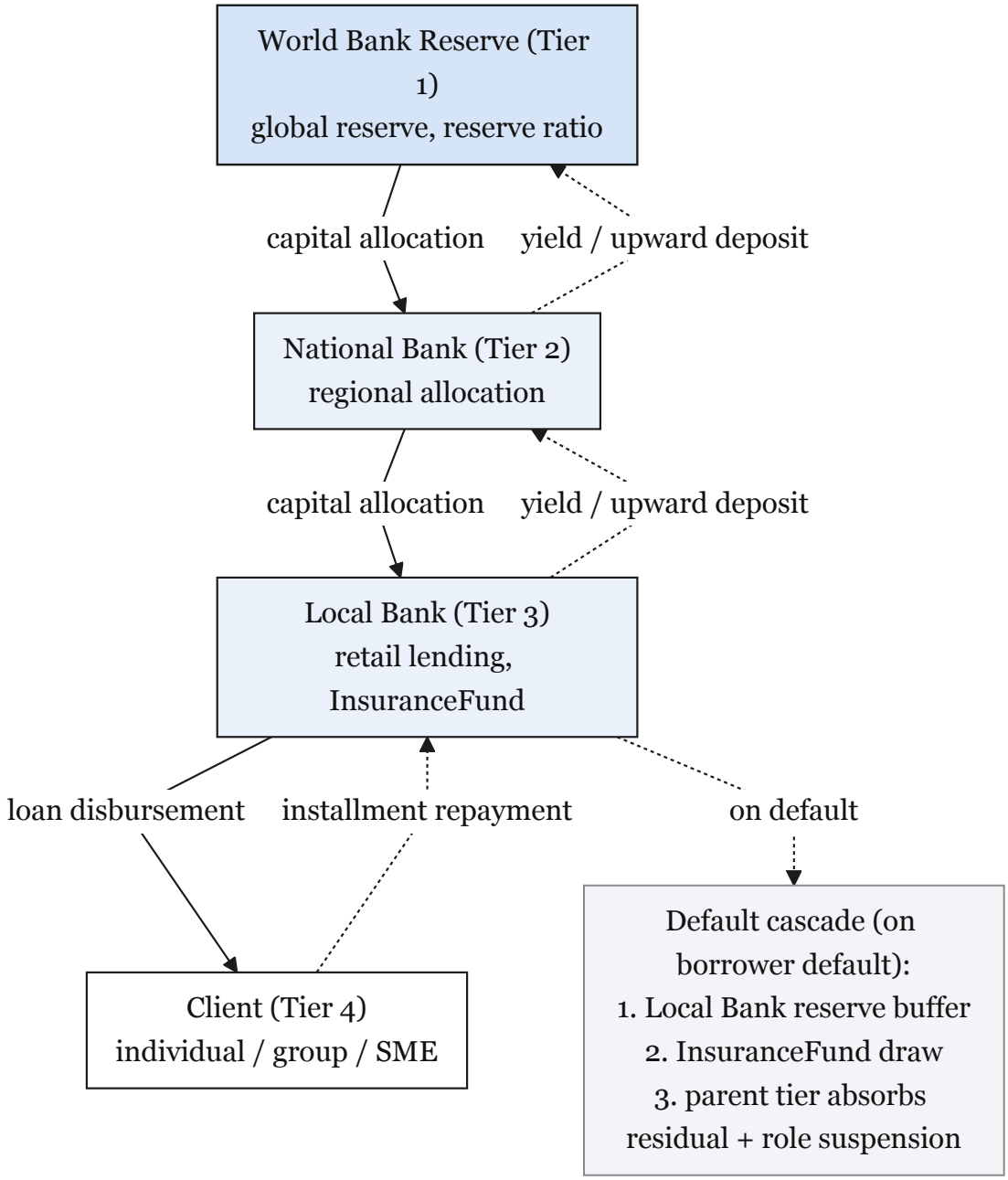


Figure 3.27: Four-tier hierarchical capital flow

3.13 Group Solidarity Lending

Groups of 3–20 clients borrow via `GroupLendingPool` with shared collateral. Per-member share is LoanTotal/N_{members}. Default uses collateral first, then SBT `riskTier` downgrade [94].

Over-indebtedness risk control. Max two simultaneous group loans, 30-day cooling-off, and cross-bank check via `ICreditPassport.getScore()`. Reject if debt-to-income DTI > 0.40 (Formula in List of Formulas).

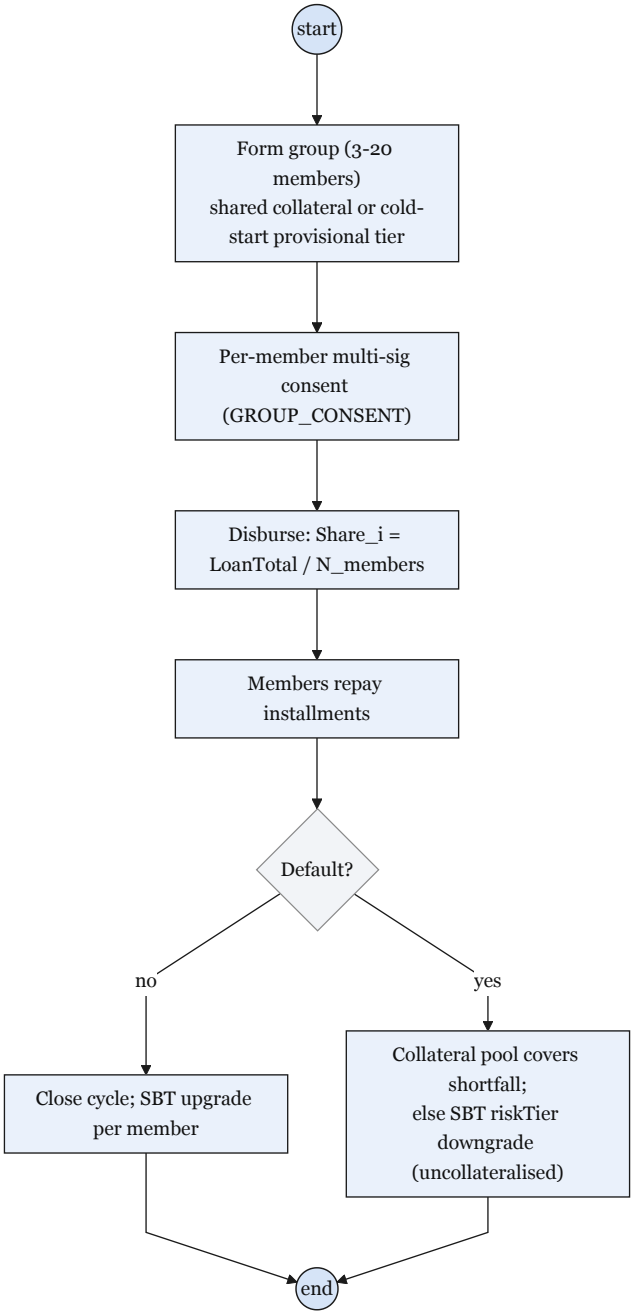


Figure 3.28: Solidarity group lending lifecycle

3.14 Governance Framework

Parameter changes route through a 48-hour `TimelockController` with snapshot voting and 60% quorum. Emergency pause uses a 2-of-3 multisig; production admin keys use Safe multisigs (Table 3.25). Prototype runs on public testnets; SAR workflow: Isolation Forest → compliance queue → `freezeAccount` (Section 3.12).

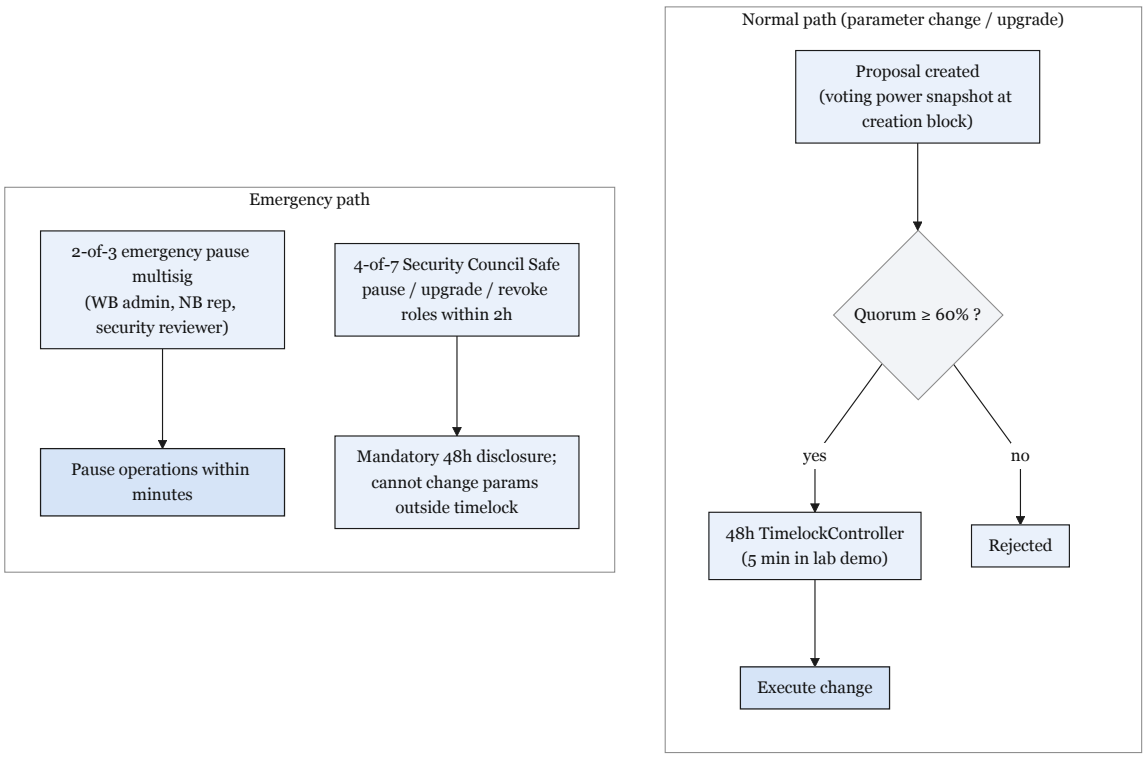


Figure 3.29: Governance dual-path

Table 3.25: Governance framework summary.

Area	Implementation
Network membership	World Bank registers National Banks; National Banks register Local Banks; hierarchical RBAC on-chain
Business operations	Reserve management, loan lifecycle, event notifications; testnet best-effort SLA
Technology	EVM contracts (decentralised); Express API (centralised); event listener syncs PostgreSQL projections
Emergency controls	Per-function pause registry; 2-of-3 emergency multisig; upgrade via UUPS + Safe
Regulatory	Audit logs via contract events; sandbox pilots for production; zkKYC Future Work

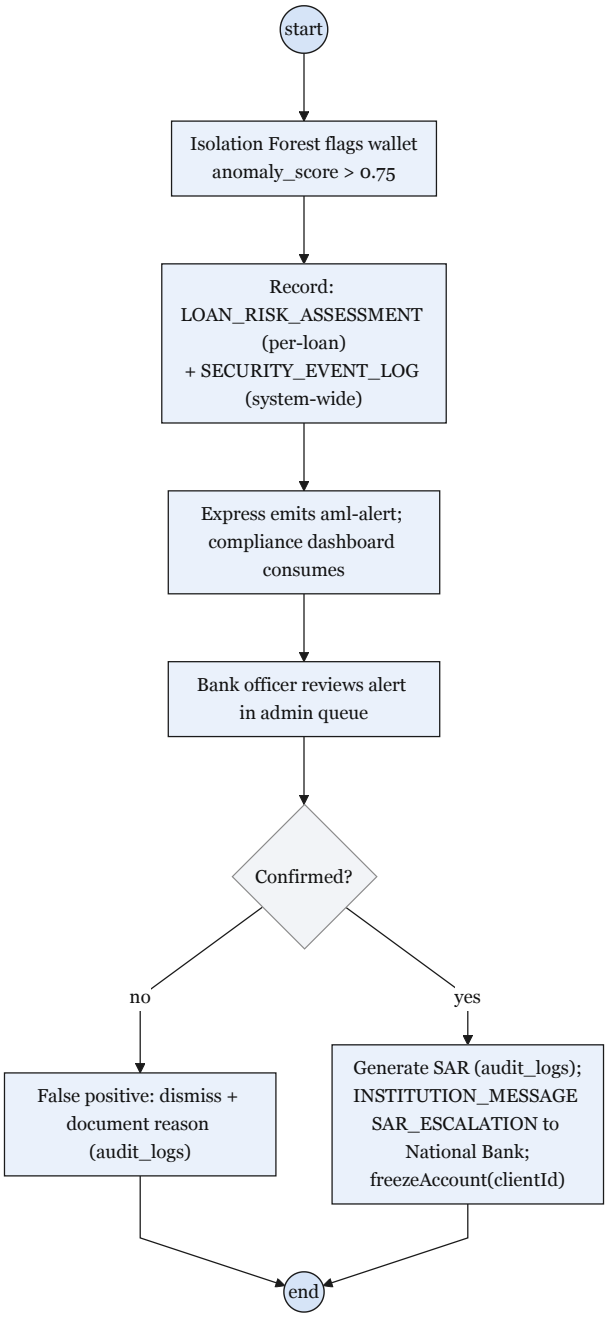


Figure 3.30: SAR and AML compliance workflow

3.15 Five-Layer Defense-in-Depth Security Architecture

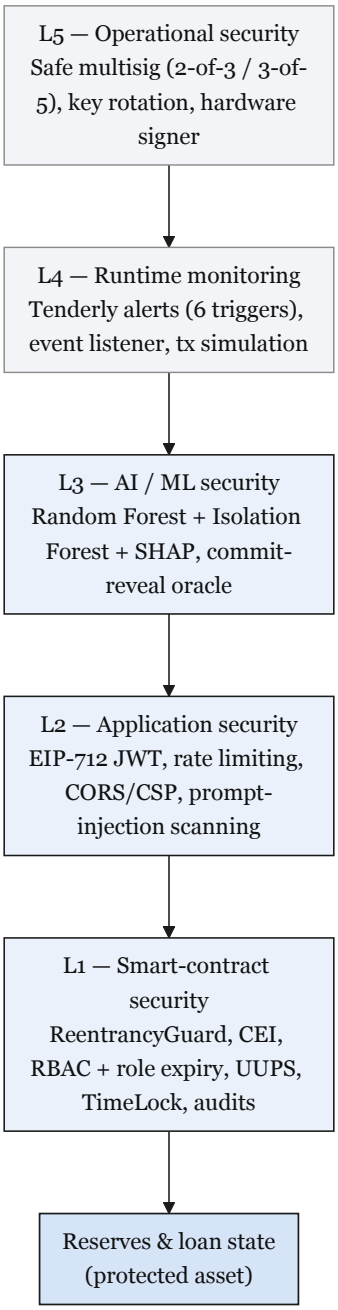


Figure 3.31: Five-layer defense-in-depth security architecture

Security spans five layers (Table 3.26): L5 operational keys (Safe multisigs), L4 runtime monitoring (Tenderly alerts), L3 ML fraud scoring with human review, L2 API auth and rate limits, L1 contract RBAC + `ReentrancyGuard` + Foundry fuzz tests. Each layer fails closed: if ML is unreachable, loans route to manual review rather than auto-approval.

Table 3.26: Five-layer defense-in-depth model.

Layer	Component	CWB control (MVT)
L5	Operational	Safe 2-of-3 for `WorldBankAdmin`
L4	Monitoring	Tenderly alerts on large disbursement, low reserve, pause
L3	AI/ML	RF + IF + SHAP; commit–reveal oracle; human approver gate
L2	Application	JWT auth, rate limits, prompt-injection scan on agent input
L1	Smart contract	OpenZeppelin RBAC, CEI, pause, Hardhat + Foundry tests

3.16 Threat Model and Security Controls

Table 3.27 maps primary threats to mitigations across the five layers.

Table 3.27: Threat model and mitigations (summary).

Threat	Layer	Vector	Mitigation
Reentrancy	L1	Recursive external call	`ReentrancyGuard`, CEI, Foundry tests
Oracle manipulation	L1	Bad price feed	Chainlink feeds; mock feed on testnet
Governance capture	L1+L5	Privileged role abuse	Safe multisig, TimeLock, role expiry
JWT / API abuse	L2	Stolen token, DDoS	Short JWT TTL, `slowapi` limits
ML evasion	L3	Adversarial features	SHAP review, human approver gate
Admin-key theft	L5	Compromised EOA	Safe multisig; no single-key admin

3.17 Societal Impact

The platform targets financial inclusion for underbanked adults through sub-cent Layer-2 transaction costs, mobile-first onboarding, and micro-loan denominations (Section 3.17). Societal benefits are *design intent*, not empirically validated deployment outcomes at pre-thesis stage: graded evidence is limited to architecture specification, BCCC fraud-detection benchmarks, and economic feasibility analysis (Chapter 5).

3.18 Environmental Impact

Settlement is planned on Proof-of-Stake EVM networks (Sepolia for demonstration; Polygon zkEVM as design-preference ZK-anchored target). Compared with proof-of-work alternatives, PoS consensus reduces per-transaction energy intensity by orders of magnitude [50]. Off-chain ML inference runs on commodity hardware; no continuous on-chain model execution is required. Environmental claims at pre-thesis stage are qualitative; quantitative carbon accounting is reserved for Future Work deployment studies.

3.19 Ethical Issues

Ethical constraints are stated in the Ethics Statement and operationalised through: tiered KYC with synthetic document hashes at MVT scope (Section 3.8); ML as **decision support** with mandatory human approver gates (Section 5.2.2); immutable `LOAN_RISK_ASSESSMENT`

audit rows; and agent write-path human confirmation (Section 4.5). BCCC training data contains public on-chain behaviour only; no private borrower PII is ingested at pre-thesis stage.

3.20 Project Management and Risk

Development follows a four-phase, 16-week plan with supervisor gates G1–G4 (Chapter 4), MoSCoW prioritisation, and explicit zero-cost risk registers (Table 3.28). Primary schedule risks—testnet instability, faucet scarcity, and scope creep—are mitigated by Sepolia-first deployment, `MockUSDC`, local Anvil/Hardhat simulation, and MVT pruning (Section 3.21).

3.21 Design Limitations and Zero-Cost Demonstration Strategy

The full architecture remains specified for examination; the graded prototype uses zero-cost testnet substitutes (Table 3.28). Key risks—no official USDC on testnets, faucet limits, RPC rate limits—are mitigated by `MockUSDC`, local Anvil simulation, and Sepolia-first deployment.

Table 3.28: Design reference versus demonstration path and key mitigations.

Area	Full design	Demo path (Ph. I–IV)	Risk mitigated
Network	Polygon zkEVM Cardona	Sepolia primary; Amoy secondary	Testnet instability
Stablecoin	USDC / USDT	`MockUSDC.sol`	No official testnet USDC
Oracle	Chainlink DON	Commit–reveal relay	Oracle cost / latency
ML delivery	Chainlink Functions	FastAPI + local relay	Integration complexity
Simulation	300 wallets live	Foundry/Anvil + 5–10 live demo	Faucet funding
Hosting	Cloud API + ML	Local stack + Vercel frontend	Cold-start / cost

Acknowledged simplifications: trusted commit–reveal relay, mock stablecoin/feeds, centralised installment cron, BCCC 21-feature benchmark (not full 22-dim CWB vector), and local simulation instead of 300 live testnet wallets. Production upgrade paths remain in Future Work.

Chapter 4

Methodology

This project paper is constructed to match with the final year project evaluation rubrics. Full development work has been distributed to 4 phases, and priority of all the capability sections has been divided to Must, Should and Future work. The Must and Should priority sections are to be completed by the end of the development and testing phase, Future Work sections have been specified only, to show the Potential of the project by extending its functionality and development scopes.

Capability	Priority	Primary interface / phase
Four-tier smart contracts	Must (Ph. I–II)	React UI + wallet; Hardhat/Foundry tests
On-chain loan lifecycle	Must (Ph. II)	React forms; `LoanController` state machine
Chainlink Functions oracle	Future Work	FastAPI → DON → on-chain score commit; commit–reveal is MVT demonstration path
RF + Isolation Forest + SHAP	Must (Ph. III)	Approver Authority Brief; client decline reasons
Authority Brief UI	Should (Ph. II–III)	Bank approver dashboard (React) — *not* agent-only
MCP conversational agent	Must (Ph. III)	Qwen3-8B + 3–5-tool MCP server; one write demo E2E
LLM evaluation protocol	Should (Ph. IV)	Red-team + task accuracy (Section 4.5; deferred)

4.1 Minimum Viable Thesis (MVT) Checklist

Table 4.1: MVT deliverable checklist.

#	Deliverable	Priority	Phase
1	Four-tier bank contracts on testnet	Must	I
2	Loan lifecycle E2E (request, approve, disburse, repay)	Must	II
3	On-chain reserve and borrowing limits	Must	II
4	ML risk scoring (RF + IF + stacking)	Must	III
5	SHAP explainability and Authority Brief UI	Must	III
6	Risk assessment audit row per inference	Must	III
7	Commit–reveal score gate before approval	Must	III
8	BCCC fraud-detection benchmark	Must	III
9	300-client simulation or gas-cost table	Must	IV
10	README with reproduction instructions	Must	IV
11	MCP agent (3–5 tools + confirmation gate)	Must	III
12	LLM evaluation protocol	Should	IV
13	Contract audit reports (Slither + Mythril)	Must	IV
14	Foundry fuzz and invariant tests	Must	IV
15	Safe multisig for World Bank admin	Must	I
16	Tenderly monitoring alerts	Should	IV

4.2 List of Features

Table 4.2 lists the complete list of features of the full project. The Must features will be completed during the development phases, Should and Stretch features are extensions of the main Must features, and Specified features are documented but not yet implemented, Future Work features are to show the potential improvements of the project and some of these features might be deliverable by the end of the project as special features.

Table 4.2: Complete feature inventory by category, priority, and implementation phase.

#	Feature	Priority	Phase
Core platform and hierarchy			
1	Four-tier banking hierarchy (World → National → Local → Client)	Must	I
2	Role-based access control (RBAC) on contracts and API	Must	I
3	World Bank Reserve — global capital custody	Must	I
4	National Bank — jurisdiction-level allocation	Must	I
5	Local Bank — retail operations hub	Must	I
6	Downward capital allocation (`allocateCapital`)	Must	I
7	Hierarchical bank registration (National → Local)	Must	II
8	Reserve-ratio enforcement before downward allocation	Must	II
9	MockUSDC / testnet stablecoin for lending	Must	I
10	2-of-3 Safe multisig for `WorldBankAdmin`	Must	I
11	Contract pause / unpause (emergency controls)	Must	I
12	Single-chain deployment (Sepolia / Amoy)	Must	I–IV
Retail lending lifecycle			

Table 4.2 (continued on next page)

Table 4.2 (continued)

#	Feature	Priority	Phase
13	Loan request submission (on-chain + UI)	Must	II
14	Approver review workflow (approve / reject / request info)	Must	II
15	Loan disbursement to client wallet	Must	II
16	Installment schedule generation	Must	II
17	Installment payment processing	Must	II
18	On-chain loan state machine (`PENDING_SCORE` → `ACTIVE`)	Must	II
19	Borrowing limits (six-month / one-year rolling caps)	Must	II
20	Collateral-based lending (LTV)	Must	II
21	Credit-based lending (no collateral, SBT-gated)	Should	II
22	Kinked interest rate model (utilization-based)	Should	II
23	Credit tier schedule (Bronze → Diamond)	Should	II
24	Overdue installment checks (cron / Chainlink Automation)	Should	II
25	Loan history and transaction tracking UI	Should	II
26	Income verification document upload and review	Should	II
Deposits and savings products			
27	SavingsVault — variable-yield savings (ERC-4626)	Should	II
28	FixedDeposit — term deposits (30/90/180/365 days)	Specified	II+
29	Current / checking account (atomic transfers)	Specified	II+
30	Deposit-to-lending pool integration	Should	II
31	Interest split (depositor / InsuranceFund / protocol)	Should	II
32	Withdrawal gated by reserve ratio	Must	II
33	InsuranceFund (5% interest capture, default coverage)	Should	II
Group and solidarity lending			
34	Group formation (3–20 members)	Must (C2)	II
35	Multi-signature group consent	Must (C2)	II
36	Per-member disbursement share	Must (C2)	II
37	Shared collateral / mutual liability	Must (C2)	II
38	Over-indebtedness controls (DTI, cooling-off, max two group loans)	Should	II
39	Group credit history on SBT	Should	II
Multi-entity and cross-tier operations			
40	InterBankLendingPool — same-tier peer lending (1/7/30 day)	Should	II
41	UpwardDepositFacility — surplus repatriation to parent tier	Should	II
42	SyndicatedLoan — multi-bank co-funding	Stretch	III
43	TranchedPool — senior/junior tranches	Stretch	III
44	TreasurySwap — cross-tier treasury FX	Should	III
45	FXModule — oracle-priced retail FX	Should	III
46	Dual-currency facility (fiat ↔ crypto)	Specified	III
47	Cascading repayment / interest across tiers	Should	II
48	NettingEngine — multilateral settlement	Future Work	—
49	Trade finance / escrow letters of credit	Future Work	—
Credit passport and identity			
50	On-chain Credit Passport (Soulbound Token)	Must	II
51	Credit score, tier, and repayment history on SBT	Must	II
52	Tier-based max loan limits and rate modifiers	Should	II
53	Wallet authentication (MetaMask / WalletConnect)	Must	I
54	EIP-712 typed-data login + JWT session	Must	I–II
55	Five-stage retail onboarding funnel	Should	I–II
56	Tiered KYC (Level 1 / Level 2)	Must	I–II
57	Off-chain KYC document storage (hash-linked)	Must	I–II
58	ERC-4337 account abstraction (gasless onboarding)	Stretch	III
59	Groth16 zkKYC / zkAML circuits	Future Work	—
AI/ML risk and explainability			
60	Random Forest fraud probability scoring	Must (C3)	III
61	Isolation Forest anomaly detection	Must (C3)	III
62	Stacking meta-learner (calibrated composite score)	Must (C3)	III
63	SHAP explainability (top-k features)	Must (C3)	III
64	Authority Brief UI for approvers	Must (C3)	II–III
65	Commit–reveal oracle score gate before approval	Must	III

Table 4.2 (continued on next page)

Table 4.2 (continued)

#	Feature	Priority	Phase
66	`LOAN_RISK_ASSESSMENT` audit row per inference	Must	III
67	BCCC-DeFiFraudTrans benchmark evaluation	Must	II–III
68	Chainlink Functions DON oracle	Future Work	—
69	GNN (GraphSAGE) ablation	Future Work	—
70	Federated learning across banks	Future Work	—
Conversational agent			
71	Qwen3-8B local LLM chat interface	Must	III
72	MCP tool server (3–5 tools at MVT)	Must	III
73	Human confirmation gate for all write tools	Must	III
74	Read tools: account state, loan status, requirements	Must	III
75	Write tools: loan application, installment payment	Must	III
76	RAG over policy documents (Q&A path)	Must	III
77	Live on-chain context injection per turn	Must	III
78	Three-tier prompt assembly (Stable / Context / Volatile)	Must	III
79	Prompt injection scanning	Must	III
80	`AGENT_ACTION_LOG` audit trail	Must	III
81	Full 17-tool MCP suite	Future Work	—
82	EIP-7702 scoped session keys for agent	Stretch	III
Frontend, API, and data layer			
83	React + TypeScript dashboard (mobile-first)	Must	I–II
84	Reserve transparency dashboard	Should	III
85	Five-state transaction UX machine (idle → success/error)	Should	II
86	PostgreSQL event listener (on-chain → DB sync)	Must	I–II
87	Real-time notifications (WebSocket / Socket.IO)	Should	III
88	Client–bank chat	Should	II
89	Express.js REST API with RBAC	Must	I–II
90	FastAPI ML inference service	Must	II–III
91	API rate limiting and correlation-ID tracing	Should	II
92	The Graph subgraph (optional query layer)	Future Work	—
Security, governance, and compliance			
93	Five-layer defense-in-depth architecture	Specified	I–IV
94	ReentrancyGuard + CEI pattern on contracts	Must	I–II
95	LiquidationEngine (health factor, liquidator bonus)	Should	III
96	SAR workflow (anomaly → freeze account)	Should	III
97	Governance timelock + snapshot voting	Specified	III+
98	Slither + Mythril static analysis	Must	IV
99	Foundry fuzz + invariant test suite	Must	II–IV
100	Tenderly monitoring alerts	Should	III–IV
101	Regulatory read-only audit access	Specified	III+
102	GDPR-style data minimisation (hash-only on-chain)	Specified	I–IV
Evaluation and operations			
103	300-client Foundry / Hardhat simulation	Must	IV
104	LLM red-team evaluation protocol	Should	IV
105	Open-source README with reproduction steps	Must	IV

4.3 Development Methodology

We have developed a plan to adapt lightweight Agile/Scrum Methodology for the prototype build with an estimated development window of 16 weeks. This iterative approach enables us to explore and improve the demonstrable features that include the minimum requirements and advanced research oriented features. These feature development phases are organised in four incremental stages (Section 4.7).

Agile Methodology application process:

- **Phase duration:** each phase is designed for 3–5 weeks, total 4 phases, total estimated time — 16 weeks.

- **Team Size:** 2 developers
- **Weekly Sync:** One weekly meeting will be conducted on Tuesday. Completion of each feature, testing and next development plans will be discussed in these meetings, based on the opinion and suggestions received from the supervisor.
- **Phase Planning and review:** After completion of each phase, a long reflective phase meeting will be arranged, to organize and plan upcoming weeks.

4.3.1 Process Model Selection

Based on the Software engineering process for a blockchain project, a development plan of Agile/Scrum with incremental phases has been chosen. Other models such as Waterfall, Spiral, V-Model have some flaws that do not meet the requirements to successfully build all the features.

Table 4.3: Process model comparison and selection rationale.

Model	Fit for CWB	Limitation	Decision
Waterfall	Clear phase documents	Cannot absorb evolving smart-contract and ML scope mid-project	Rejected as primary
Incremental	Matches four phases + gates G1–G4	Still needs within-phase flexibility	**Adopted** (phase structure)
Iterative / Agile	Weekly sync; phase retrospectives	Requires discipline on scope	**Adopted** (within-phase)
Spiral	Strong for high-risk R&D	Overhead excessive for 16-week student project	Not adopted
V-Model	Maps to test levels in Ch. 5	Too rigid for research prototyping	Used for *verification planning* only

The project is built in the main branch from the root section. It follows a conventional **Model–View–Controller (MVC)** architecture for separation in root. **Model** — `contracts/`: on-chain state machines and PostgreSQL (off-chain projections), **View** — `frontend/` (React components), **Controller** — `backend/`. For version control the repository is managed with git command, with consistency and compatibility checks, validated by `npm run test:contracts` (Hardhat) before each merge into main branch.

4.4 Planned AI/ML Support and Risk-Score Wiring

The purpose of the AI/ML layer is to provide a risk score per loan approval that will be delivered on-chain through the commit–reveal oracle (Section 3.4.2). The oracle gate design has been shown in Figure 4.1. ML scoring and explainability outputs have been shown in Section 5.2.2, and the rest of the benchmarks and evaluations have been reported in Chapter 5.

Risk Scoring: A sequential step combines Random Forest classifier and Isolation Forest to provide fraud probability and anomaly scores, and these scores are evaluated within $s \in [0, 1]$.

Commit–reveal oracle: This commits a hash of (s, nonce) to `LoanController`, then each transaction comes with a risk score.

Decision threshold: A minimum and maximum threshold of the value of score (s) is maintained for loan approval decisions: approval consideration ($s < 0.4$), manual review ($0.4 \leq s < 0.7$), or rejection ($s \geq 0.7$).

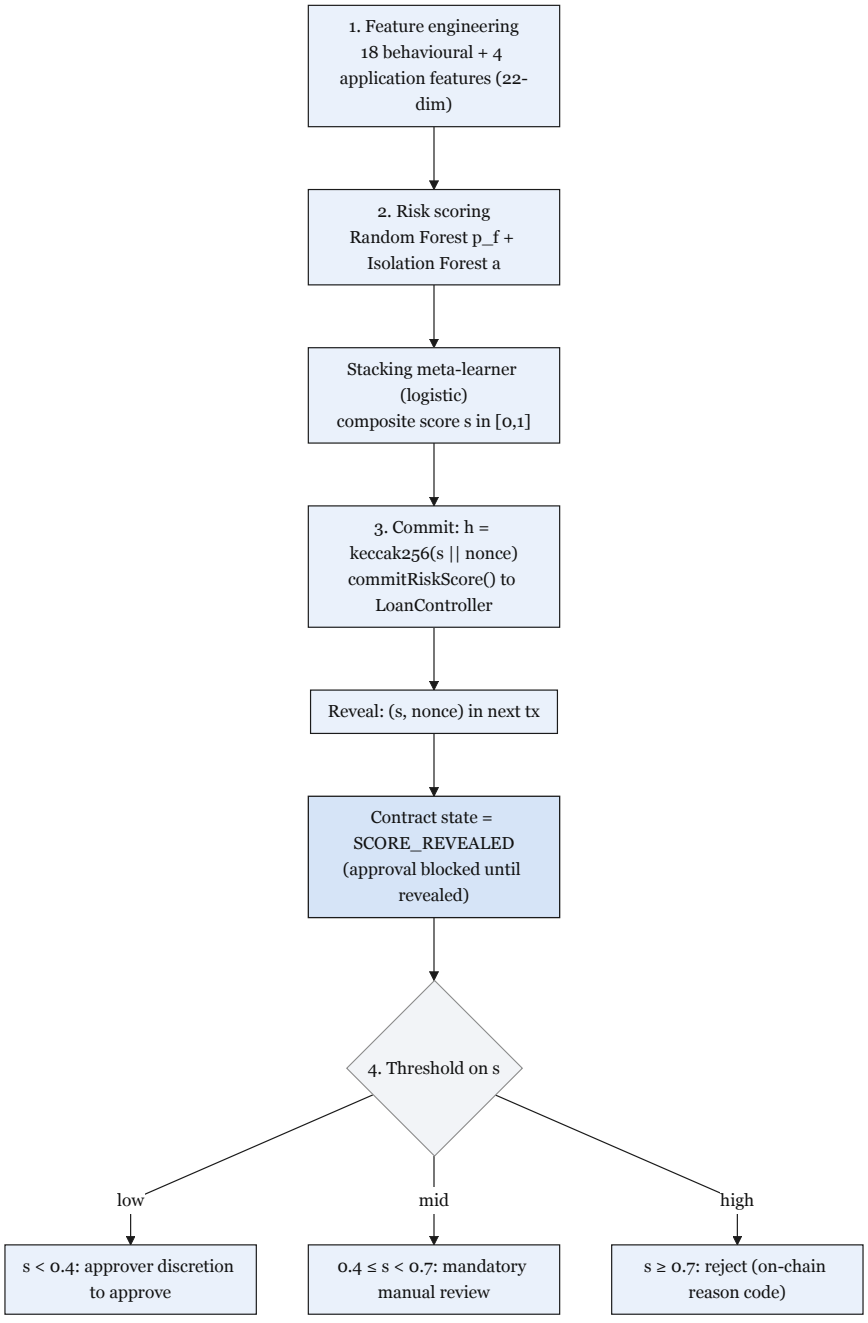


Figure 4.1: ML oracle commit–reveal gate

4.5 Conversational Agent (Phase III–IV)

The conversational agent that has been designed to assist the clients to better navigate the platform is planned to be completed by Phase III. It is an auxiliary feature that works

alongside the standard web UI for those who may not be familiar with the banking system of the project and have less domain knowledge and want to understand the platform and how to navigate with it. The schema of the full MCP agent has been shown in Table 4.4; explainability for approvers is documented in Chapter 5.

Agent six-step pipeline. Clients are expected to interact with the frontend UI like any other typical user-friendly banking website. The agent works as a guiding assistant and its tool-calling abilities use the same Express.js API endpoints; the same smart contracts are considered for functionalities. This six-step pipeline lets the Qwen3-8B model have access to the live on-chain context injection layer that helps the agent process related instant and consistent information that is relevant to how a client is interacting with the platform. The layers have been described shortly below:

1. **User enters the platform and interacts with the agent:** frontend interaction is recorded and forwarded via the Express.js backend over Server-Sent Events (SSE).
2. **Agent model with live on-chain context:** The Express.js context injection layer provides the client's information to the model (credit score, active installments, loan status, SBT risk tier, pool utilisation). The model receives this information about the user before the conversation prompt by the user, to process relevant information about the client to assist them.
3. **Q&A:** For this part of the conversation between the agent and client, the agent only processes relevant information about the client via RAG (ChromaDB) and maintains formal conversation with the client, and does not call any tools.
4. **Action request:** After a formal conversation with the client, if the client requests a state-modifying action such as requesting a loan, the agent will choose and prepare MCP write tools, assemble the full parameter set from the injected context, and show a concise summary of action on behalf of the client. Ex: *"You are requesting a loan of 150 USDC for 6 months at 8.5% APR. Your monthly installment would be 26.25 USDC. Shall I proceed?"*
5. **Human confirmation gate:** The agent will wait until a confirmation of affirmation for the action that has been prepared to initiate the loan process. The MCP agent writing tools will not be initiated until a positive response has been received from the client side. If a client responds with vague actions three times, the agent does not make the tool calls, pauses the agent service for 10 minutes, and logs this interaction to `AGENT_ACTION_LOG`.
6. **MCP write tool functionality:** If a successful conversation is established with the client, the agent takes confirmation from the client to initiate the loan request process; with a positive response the agent initiates write tools and accesses the Express.js banking API, then checks for an EIP-7702 session key—if active, it signs the on-chain transaction approval request. The agent gives a confirmation that the request has been forwarded and notifies the client that the loan application has been submitted, shows a reference number, and how long the approval might take.

MCP tool server. For the MVT scope the agent has been designed for 3–5 tools for a minimal typed tool schema. The specified 17 tools have been designed for future work, and some of these tools may be included in this prototype build if they can be delivered in the allocated time, after the MVT scope has been secured.

Table 4.4: MCP tool server (MVT subset). The full 17-tool suite is Future Work.

Tool	Type	Maps to / Description
`get_account_state`	READ	SBT tier, loan count, savings balance
`get_loan_status`	READ	Active loans + next installment
`get_requirements`	READ	Documents + KYC needed for a given loan amount
`submit_loan_-application`	WRITE	POST /api/loan/apply
`pay_installment`	WRITE	POST /api/installment/pay

4.6 Real-Time Dashboard Pipeline and Runtime Monitoring

For stability, lightweight data indexing strategies are used to reduce complexities. After the correct implementation of the core contracts and event schemas, an optional query layer is to be added for GraphQL access for better demonstration of analytics. A Node.js service is utilized for WebSocket event listener via Socket.IO. Tenderly monitoring and MVT-graded time and attempt triggers are added for monitoring and inconsistency checks.

4.7 Implementation Phase Plan and Deliverables

4.7.1 Phase I: Foundation and Infrastructure (Weeks 1–4)

Phase I focuses on the foundation layers, such as the smart contracts of World Bank, National Bank, and Local Bank, setting the right format and design of the database and schemas, and successfully deploying the smart contracts on Sepolia to show the contracts are functional. Priority follows the MoSCoW classification:

Table 4.5: Phase I task register: smart contract and database foundation.

Task	Priority	Development Task	Effort (days)
DT-I.01	Must	World Bank contract — reserve management, national bank registration	5
DT-I.02	Must	National Bank contract — borrow from World Bank, lend to Local Banks	5
DT-I.03	Must	Local Bank contract — borrow from National Bank, lend to users	5
DT-I.04	Must	Role-based access control — WorldBankAdmin, NationalBankAdmin, LocalBankAdmin, BankUser, RetailClient	3
DT-I.04a	Must	`MockUSDC.sol` — mintable ERC-20 stablecoin for testnet loan denomination (Section 3.21)	1
DT-I.05	Should	Gas cost management — initiator-pays pattern; Polygon low-fee design	2
DT-I.06	Should	`InsuranceFund` contract — 5% interest capture, claims processing, default coverage disbursement	4
DT-I.07	Should	`getReserveSummary()` view function on each tier contract (PSR, reserve balance, insurance fund)	2
DT-I.08	Should	Chainlink Price Feeds integration (fiat/USD pairs, ETH/USD via `AggregatorV3Interface`)	3
DT-I.09	Should	Chainlink Proof of Reserve — WorldBankReserve attestation	2
DT-I.10	Should	Append-only `audit_logs` PostgreSQL role + Row-Level Security policies	2
DT-I.11	Could	Polygon zkEVM Cardona migration (design-preference target) — update RPC endpoints, chain IDs, factory addresses	1
DT-I.12	Must	Phase I schema migration — **13 M1 tables** (Section 3.5.2): `INSTITUTION`, `COUNTRY`, tier subtypes, `BANK_USER`, `BORROWER` (incl. `kyc_level`), `LOAN_REQUEST`, `LOAN`, `INSTALLMENT`, `BLOCKCHAIN_EVENT_LOG`, `ASSETS`, `AUDIT_LOGS`; M2/M3/stub tables deferred	4
DT-I.13	Must	Core indexes and referential integrity constraints	2
DT-I.14	Must	React frontend shell — Vite build, routing, layout components	3
DT-I.15	Must	WalletConnect integration — wallet connection, address display, network switching	2
DT-I.16	Should	Sessions and assets tables integration in frontend (dashboard stubs)	2
DT-I.17	Should	Credit tier and interest rate schedule display	2
DT-I.18	Must	Safe 2-of-3 multisig setup and `WorldBankAdmin` role transfer (MVT item 15)	1
Phase I total estimated effort (Must-have):			*31 days*
Phase I total estimated effort (all priorities):			*51 days*

4.7.2 Phase II: Core Banking and On-Chain Lifecycle (Weeks 5–9)

For Phase II, we focus on building the React user interface to be responsive with loan requests and other banking functionalities such as loan approval, installments, registering accounts, and setting borrowing limits. Moreover, this phase builds the building blocks for Phase III that will incorporate the conversational agent and the tools that the agent has access to.

Task	Priority	Development Task	Effort (days)
DT-II.01	Must	Loan request submission with blockchain transaction	5
DT-II.02	Must	Loan approval and rejection workflow with bank approver	5
DT-II.03	Must	Installment payment system with automated schedule generation	8
DT-II.04	Must	Borrowing limit enforcement (on-chain authoritative check per client SBT)	5
DT-II.05	Should	Client–bank real-time chat system	5
DT-II.06	Should	Income verification document upload and review	5
DT-II.07	Must	Hierarchical bank registration (National → Local)	5
DT-II.08	Should	Bank user management and approver designation	3
DT-II.09	Should	Loan history and transaction tracking pages	5
DT-II.10	Could	QR code generation for wallet addresses	2
DT-II.11	Should	Responsive UI polish and error handling	2
DT-II.12	Should	Authority Brief UI (SHAP breakdown for bank approver dashboard)	5
DT-II.13	Should	Installment due-date checks: Node.js cron (MVT demo path) or Chainlink Automation upgrade when LINK funded	5
DT-II.14	Should	Credit tier schedule in Credit Passport SBT (Bronze → Diamond)	3
DT-II.15	Should	`interest_rate_tier` table integration with Kinked Rate Model	2
DT-II.16	Should	`agent_action_log` + `SESSIONS` schema migration (agent audit trail foundation)	3
Phase II total estimated effort (Must-have):			*28 days*
Phase II total estimated effort (all priorities):			*68 days*

4.7.3 Phase III: AI/ML Oracle Pipeline and Conversational Agent (Weeks 10–13)

Phase III is the continuation of Phase II, using all lending functionalities such as loan request, approval workflow, and adding the ability of the MCP agent to interact with these tasks. The ML layer is also integrated with the oracle link and a connection between the on-chain and off-chain is established. Implementation of conversational agent harnessing including MCP server, conversational UI, and confirmation gates from both client and bank authority.

Task	Priority	Development Task	Effort (days)
DT-III.01	Future Work	Chainlink Functions oracle (production upgrade): DON-based score commit when supported; commit–reveal is MVT demonstration path	6
DT-III.02	Must	Random Forest + Isolation Forest + SHAP pipeline wiring: FastAPI service computing 22 features, stacking meta-learner, calibrated composite score $s \in [0, 1]$; client-facing SHAP plain-language explanation via Authority Brief UI	7
DT-III.03	Must	MCP tool server — minimal (3–5) tool subset wired to Express.js API; one write demo E2E	4
DT-III.04	Must	Agent chat interface (Qwen3-8B + tool calling + human-gate)	8
DT-III.05	Should	Session-key management (EIP-7702) and scope enforcement	5
DT-III.06	Must	Three-tier prompt constructor in Express.js	5
DT-III.07	Must	Prompt injection scanning + confirmation audit hook (Hook 1)	5
DT-III.08	Should	Session lineage + `get_session_history` MCP tool	5
DT-III.09	Should	Context compression path + toolset scoping + Hook 2	9
DT-III.10	Should	(Optional) The Graph query layer + WebSocket event listener for Reserve Transparency Dashboard	4
DT-III.11	Should	SAR workflow: Isolation Forest $\rightarrow$ aml-alert $\rightarrow$ `freezeAccount`; compliance review queue integration	4
DT-III.12	Should	Reserve Transparency Dashboard: live queries to event-listener index (Graph optional later)	3
DT-III.13	Could	GraphSAGE GNN ablation	5
DT-III.14	Could	Federated learning aggregator	7
Phase III total estimated effort (Must-have):			*29 days*
Phase III total estimated effort (all priorities):			*77 days*

4.7.4 Phase IV: Verification, Evaluation, and Finalization (Weeks 14–16)

Implementing the Foundry fuzz suite, fully functional 300-client simulation, testnet deployment (local + remote), and the LLM agent evaluation reports, and finally paper refinements and finalization.

Task	Priority	Development Task	Effort (days)
DT-IV.01	Must	300-client scripted Foundry simulation on local Anvil/Hardhat (deterministic seed); gas-cost table and reserve-dynamics manifest published to Appendix C; optional Sepolia replay subset for explorer evidence	7
DT-IV.02	Should	LLM evaluation protocol: task-level accuracy, action accuracy, human-gate bypass test (Section 4.5; MVT item 12)	5
DT-IV.03	Future Work	Certora reserve invariant proofs: CVL specifications for Invariant 1 (reserve never under-collateralised) and Invariant 2 (no over-allocation downstream); specifications added to Appendix B	5
DT-IV.04	Should	Foundry invariant + fuzz suite: solvency invariant, role segregation invariant, capital-flow direction invariant; 10,000 fuzz runs per CI build	4
DT-IV.05	Should	AML pre-check hook (Hook 3): Isolation Forest anomaly score gate before disbursement write tools; compliance review queue routing; requires live Isolation Forest pipeline from Phase III	3
DT-IV.06	Could	Groth16 zkKYC circuit (Circom 2.0): government-ID possession proof without revealing PII; circuit design and input/output specification	5
DT-IV.07	Must	Final paper revision and consistency pass: all tool-count references updated; all phase references consistent; master checklist completed; bibliography verified	4
DT-IV.08	Must	Red-team testing and documentation: 20-payload injection red-team set; 20 bypass-attempt set; 10 session key scope enforcement scenarios; results documented in evaluation subsections	3
DT-IV.09	Must	Slither + Mythril audit run on World/National/Local Bank contracts; triage and archive report (MVT item 13)	2
DT-IV.10	Should	Tenderly alert configuration: 3 demo-critical triggers + README reproduction pass (MVT items 10, 16)	2
Phase IV total estimated effort (Must-have):			*16 days*
Phase IV total estimated effort (all priorities):			*40 days*

Phase	Focus	Weeks	Tasks	Key deliverables
I	Foundation and infrastructure	1–4	DT-I.01–18	Contract hierarchy, `MockUSDC`, core DB schema, React shell
II	Core banking and on-chain lifecycle	5–9	DT-II.01–16	Lending workflow, SBT limits, cron/Automation triggers
III	AI/ML oracle pipeline + conversational agent	10–13	DT-III.01–14	Commit–reveal oracle, ML pipeline, MCP agent (minimal toolset), dashboard
IV	Verification, evaluation, finalization	14–16	DT-IV.01–10	Foundry, simulation, evaluation pack, testnet deploy

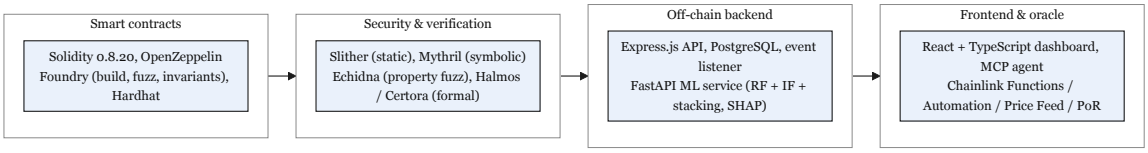


Figure 4.2: Planned development and verification toolchain

4.8 SDLC Stage Mapping

Table 4.6: SDLC stage mapping.

#	SDLC Stage	Project Activity	Deliverable
1	Planning	Feasibility studies; professor consultations; guideline review	Feasibility Report; Project Plan
2	Requirements	System analysis; use case definitions; constraints identification	Use case diagrams; UC-1 through UC-11 (eleven grouped use cases)
3	Design	Four-layer architecture; DB schema (51 entities, 3NF); smart contract interfaces	Architecture diagrams; ERD; DFD
4	Development	Phase I–IV: smart contracts, frontend, backend, AI/ML, agent	Source code; DApp prototype
5	Testing	Hardhat unit tests; Foundry fuzz; integration testing; AI/ML + agent evaluation	Test reports; model metrics
6	Deployment	Ethereum Sepolia (Phases I–IV demonstration); Certora on Sepolia (Phase IV stretch); frontend (Vercel); backend local-first (Render optional)	Testnet prototype manifest
7	Maintenance	Monitoring; model retraining; bug fixes; iteration	Updated docs; retrained models

4.9 Design Decisions and Alternatives

Table 4.8 demonstrates the chosen stack for frontend, backend, and all dependencies for the stability and completion of the project. The comparison table is shown below:

Table 4.8: Design decisions and alternatives considered.

Decision Area	1st Choice	2nd Choice	Key Criterion
Methodology	Agile / Scrum	Incremental	Evolving scope, milestones
Architecture	DApp + Off-chain AI	Hybrid with Oracle	Gas cost, ML flexibility
Frontend	React + TypeScript	Vue + TypeScript	Web3 ecosystem maturity
Smart Contract	EVM (Solidity)	Solana (Rust)	Gas, control size, free testnets
Fraud Detection	Random Forest	XGBoost	SHAP compatibility, simplicity
Anomaly Detection	Isolation Forest	Autoencoder	Unsupervised; no labelled data needed
XAI Method	SHAP	LIME	Theoretical guarantees, regulatory fit
Database	PostgreSQL	SQLite	3NF support, async queries
Hosting	Vercel + Render	Localhost only	$0 cost, publicly accessible URL
Network (primary)	Ethereum Sepolia (primary) and/or Polygon Amoy PoS (secondary)	Polygon zkEVM Cardona	Stability-first prototype vs. ZK-anchored design preference
Oracle mechanism	Commit-reveal relay (MVT)	Chainlink Functions (DON)	Always-available audited relay vs. DON-based upgrade when supported
Agent tool framework	MCP tool server	Direct Express.js API	Defined schema = safety boundary; Phase III Must
Agent session auth	Human confirmation + standard wallet signing	EIP-7702 session keys	MVT relies on explicit consent; scoped session keys are Future Work
Prompt construction	Three-tier assembly	Single ad-hoc string	Phase III Must
Session memory	Lineage model	10-turn window	Phase III Should; cross-session continuity
Tool exposure per turn	Named toolsets	Full 17-tool list	Phase III Should; attack-surface reduction
Write-tool enforcement	Confirmation audit hook	In-pipeline check	Phase III Must; agent write-path

Table 4.10: Design alternative evaluation criteria (selected vs. rejected).

Decision	Selected	Rejected	Criteria (weight)	Outcome
Methodology	Agile + incremental phases	Pure Waterfall	Adaptability (40%), milestone clarity (30%), team size (30%)	Agile wins on evolving scope
Settlement chain	Sepolia / Amoy (demo)	zkEVM Cardona (day-1)	Stability (50%), cost (30%), ZK anchor (20%)	Demo path stability-first
ML delivery	Off-chain FastAPI + commit–reveal	On-chain Chainlink Functions	Latency (35%), cost (35%), auditability (30%)	Off-chain for MVT
Fraud model	Random Forest + SHAP	XGBoost + LIME	Explainability consistency (50%), tabular fit (30%), imbalance (20%)	RF exact TreeExplainer
Database	PostgreSQL 3NF	SQLite	Concurrency (40%), window queries (35%), FK integrity (25%)	PostgreSQL for lending projections

4.9.1 Justification of Selected Technologies

Table 4.11: Selected technology justifications (1st choice).

Technology	Justification
Agile/Scrum	Supports evolving contract, UI, and ML scope within short iteration cycles.
DApp + off-chain AI	ML runs off-chain for speed and cost; critical lending decisions remain on-chain.
React + TypeScript	Strong Web3 library support (Wagmi, RainbowKit, Viem, ethers.js).
EVM (Solidity)	Mature security tooling; Sepolia/Amoy testnets; OpenZeppelin libraries.
MUI (Material Design 3)	Ready-made tables, forms, and dashboards for banking UI.
Random Forest	Exact SHAP TreeExplainer; strong on tabular lending data; handles class imbalance.
Isolation Forest	Unsupervised anomaly detection with few labeled fraud samples.
SHAP	Consistent, theory-grounded feature attribution for explainability.
PostgreSQL	ACID compliance, window queries for borrowing limits, JSON for prototyping.
Vercel + Render	Free hosting for live supervisor and examiner demos.
Sepolia / Amoy testnets	Stability-first deployment path within the thesis timeline.
Commit–reveal oracle	Simple MVT path with immutable on-chain ML score audit trail.
MCP tool server	Typed tool schema bounds agent reads and writes.
EIP-7702 session keys	Future Work: scoped session keys for agent on-chain actions.

Table 4.12: Retained alternatives (2nd choice) and why they were not selected.

Alternative	Why retained but not primary
Waterfall	Clear phases, but too rigid for research prototyping.
Hybrid on-chain oracle	Higher latency and cost; external oracle dependency.
Vue + TypeScript	Smaller Web3 ecosystem than React.
Solana (Rust)	Steeper learning curve and weaker audit tooling within 16 weeks.
Tailwind CSS	More custom UI work than MUI for banking layouts.
XGBoost	Approximate SHAP; weaker explanation consistency.
Autoencoder	Needs more data and tuning than Isolation Forest.
LIME	Less stable explanations than SHAP for audit use.
SQLite	Limited concurrency and window-query support.
Localhost-only deploy	Blocks remote supervisor and examiner access.
Polygon Amoy PoS (2nd network)	Fallback testnet if Sepolia is unstable.
Chainlink Functions	Stretch upgrade when supported; higher latency and fees.
Direct Express.js API	Weaker safety boundary than MCP tool schema.
ERC-4337 paymasters	Weaker scope control than EIP-7702 for agent actions.

4.10 Implementation Feasibility and Demonstration Scope

The full project scope—phase task registers (DT-I.01–IV.10), the MVT checklist (Table 4.1), and the module inventory in Chapter 3—remains the project record. This section states what we will actually build and demo in the 16-week window using free testnets and local tools.

Phase I targets roughly 31 person-days of Must work for a two-person team over four weeks. That is enough for core contracts, database setup, and a basic frontend, but only if Should tasks are not treated as gate blockers. Phase I originally listed about 30 Must person-days; the revised plan keeps the same scope but adds explicit room for testnet debugging.

The Phase I (G1) demo must show:

- Three tier contracts and `MockUSDC` deployed on Sepolia
- A working capital-flow demo (`allocateCapital` across tiers)
- Passing RBAC tests in Hardhat or Foundry
- A React dashboard connected via WalletConnect that reads live reserve balances
- PostgreSQL M1 tables migrated with seed data (Section 3.5.2)
- A short screen recording ($\leq$5 minutes) as the MVT #1 artifact

Production upgrades—Chainlink DON oracles, live USDC, full 51-table schema, and mainnet deployment—stay documented in Chapter 3 and Future Work. They are not required

for the graded prototype.

Chapter 5

Evaluation, Testing and Feasibility

This chapter reports **empirical and design-level evaluation** of the Crypto World Bank against the five research questions (Section 1.5): ML benchmarks, smart-contract verification, implementation evidence, economic feasibility (CO11), and examiner limitations. Build planning remains in Chapter 4.

5.1 Evaluation Methodology

5.1.1 Verification and Validation

Following the V-Model distinction taught in Software Engineering:

- **Verification** (building the product *the right way*) — unit and integration tests confirm that components meet their specifications. Contract verification uses Hardhat tests in `test/` (tier RBAC, deposit/allocation invariants, hierarchy integration). API verification uses Vitest suites in `backend/tests/`. Planned Foundry fuzz runs (Section 5.3) extend verification to stateful invariants.
- **Validation** (building the *right product*) — supervisor gate demonstrations (G1–G4), research-question evaluation (below), and BCCC fraud-detection benchmarks confirm that the prototype addresses the stated problem. Validation is intentionally separated from verification: passing all unit tests does not prove hierarchical development-finance claims; those are assessed via structured comparison to flat DeFi protocols (RQ1) and design-level banking extensions (RQ5).

This section summarizes how each research question will be evaluated in the prototype phase.

- **RQ1 (architecture encoding):** structured workflow mapping and feature comparison against flat DeFi protocols (Aave, Compound, MakerDAO), documenting institutional hierarchy, role separation, and directional capital flow (downward, same-tier, upward, syndicated/group). Success criterion: the four-tier specification encodes development-finance-style intermediation patterns absent from flat pools—not empirical validation against live World Bank or IMF operations.
- **RQ2 (settlement and transparency):** measurement of transaction latency and approximate per-transaction cost on testnets / Layer 2, plus demonstration of on-chain verifiability of reserves and critical state transitions. Figure 5.1 specifies the planned measurement points for Phase IV.
- **RQ3 (analytics support):** model evaluation using standard metrics (precision, recall, F1, AUC) plus explainability completeness (SHAP coverage, anomaly deviation reports, Authority Brief audit rows in `LOAN_RISK_ASSESSMENT`) is reported in three explicitly separated phases. *Phase (a) — BCCC baseline (pre-thesis):* measured metrics on a 50,000-address subsample of BCCC-DeFiFraudTrans-2025 with address-grouped hold-out (Table 5.2, Figure 5.2); establishes RF + IF + stacking + SHAP on real DeFi

fraud labels while acknowledging flat-pool domain gap vs. CWB. *Phase (b) — integration test:* model ingestion of procedurally generated loan-manifest features, labeled "proof-of-concept evaluation only." *Phase (c) — full BCCC + CWB mapping:* retrain on the complete 1M-row corpus, map to the 22-dimensional CWB feature vector, and update MVT item 8. GNN ablation and federated learning are likewise Future Work (Section 6).

- **RQ4 (technical viability):** end-to-end deployment verification on public testnets with integration tests covering wallet connection, loan request, approval, repayment, and role-based access control; Foundry invariant suite reports the proportion of fuzz runs that hold each invariant (Section 5.3). *Note:* at pre-thesis stage, RQ4 is evaluated through formal specification, Hardhat simulation scripts, and planned Phase IV testnet deployment. End-to-end four-tier capital flow will be evaluated and reported in the final thesis.
- **RQ5 (banking extensions):** design-level validation via specification completeness and interface contracts for deposit products and group lending, plus prototype demonstrations of selected mechanisms where implemented.

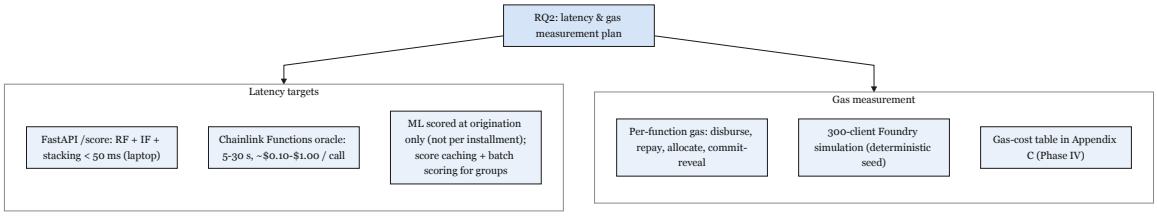


Figure 5.1: RQ2 latency and gas measurement plan

5.2 Machine Learning Evaluation

5.2.1 Datasets and Benchmark Results

RQ3 is evaluated with precision, recall, F1, and ROC-AUC on a held-out test set, plus explainability completeness (SHAP coverage and anomaly deviation lines in `LOAN_RISK_ASSESSMENT`). Pre-thesis evaluation uses a **50,000-address subsample** of BCCC-DeFiFraudTrans-2025 [99] with address-grouped hold-out (`synthetic: false`); full 1M-row retraining and 22-dimensional CWB feature mapping are scheduled for final thesis.

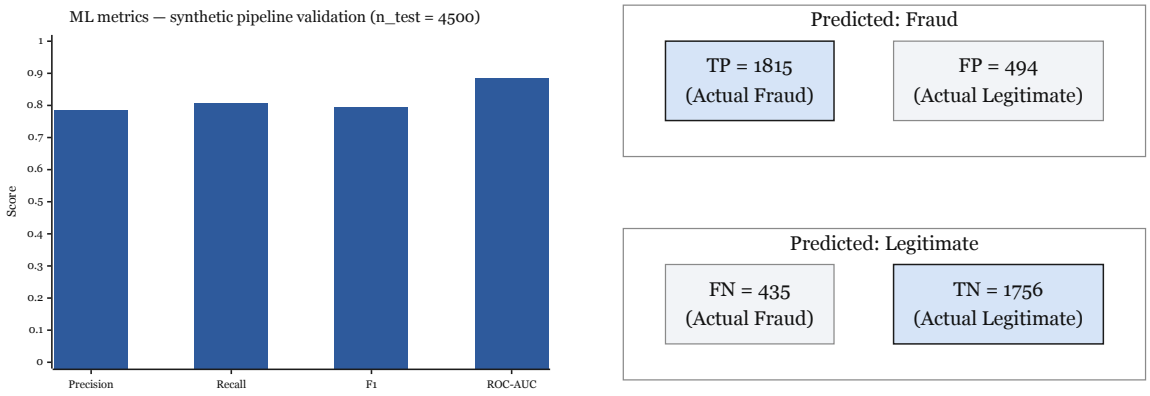
Table 5.1: Datasets used for ML risk scoring (RQ3).

Dataset	Role	Usage in this project
BCCC-DeFiFraudTrans-2025 [99]	Primary (supervised)	50k-address subsample; held-out metrics in Table 5.2; full corpus deferred to Phase III
Elliptic Bitcoin Dataset [68]	Baseline comparison	Secondary benchmark only; domain mismatch acknowledged
On-chain CWB transaction log (future)	Production retrain	Local Bank history once MVT deployment has live traffic

Table 5.2: BCCC-DeFiFraudTrans-2025 subsample benchmark on held-out test set (`evidence/mac-m4/metrics.json`).

Split	Precision	Recall	F1	ROC-AUC	n_{test}
Address-grouped hold-out	0.874	0.908	0.891	0.991	5,140

Confusion matrix (TN = 4,621, FP = 60, FN = 42, TP = 417); fraud rate ≈8.7%; 21 behavioral features; leakage column `flag` removed.



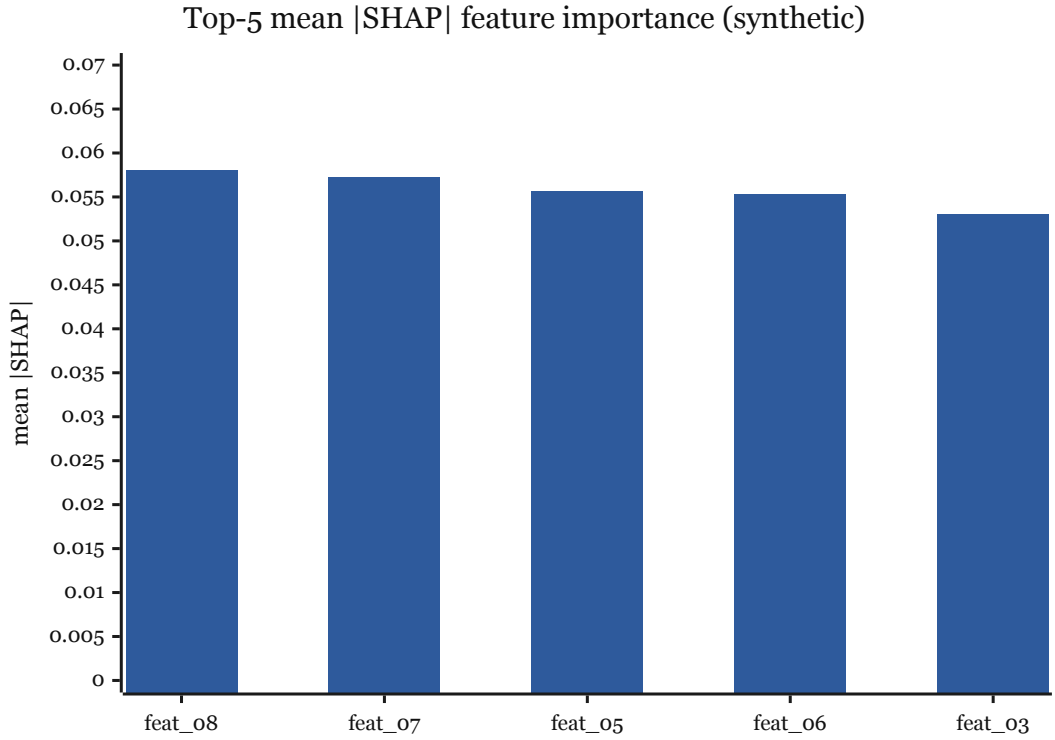


Figure 5.2: ML evaluation on BCCC subsample (held-out test: F1 = 0.891, ROC-AUC = 0.991) and global SHAP feature importance.

Domain-transfer limitation. BCCC [99] contains Ethereum DeFi fraud from flat pool protocols. CWB hierarchical lending generates different transaction patterns. BCCC provides baseline fraud-detection benchmarking only; production deployment requires CWB-specific labeled data from testnet operation (Future Work, Section 6).

5.2.2 Explainability Methodology

The FastAPI scoring service combines Random Forest fraud probability p_f (with SHAP TreeExplainer attributions) and Isolation Forest anomaly scores via a stacking meta-learner. Anomaly explainability uses feature-deviation reports (applicant value vs. training median/percentile). Every inference writes one `LOAN_RISK_ASSESSMENT` row; the contract never auto-disburses on ML output alone—an approver must sign disbursement (decision support, not decision authority).

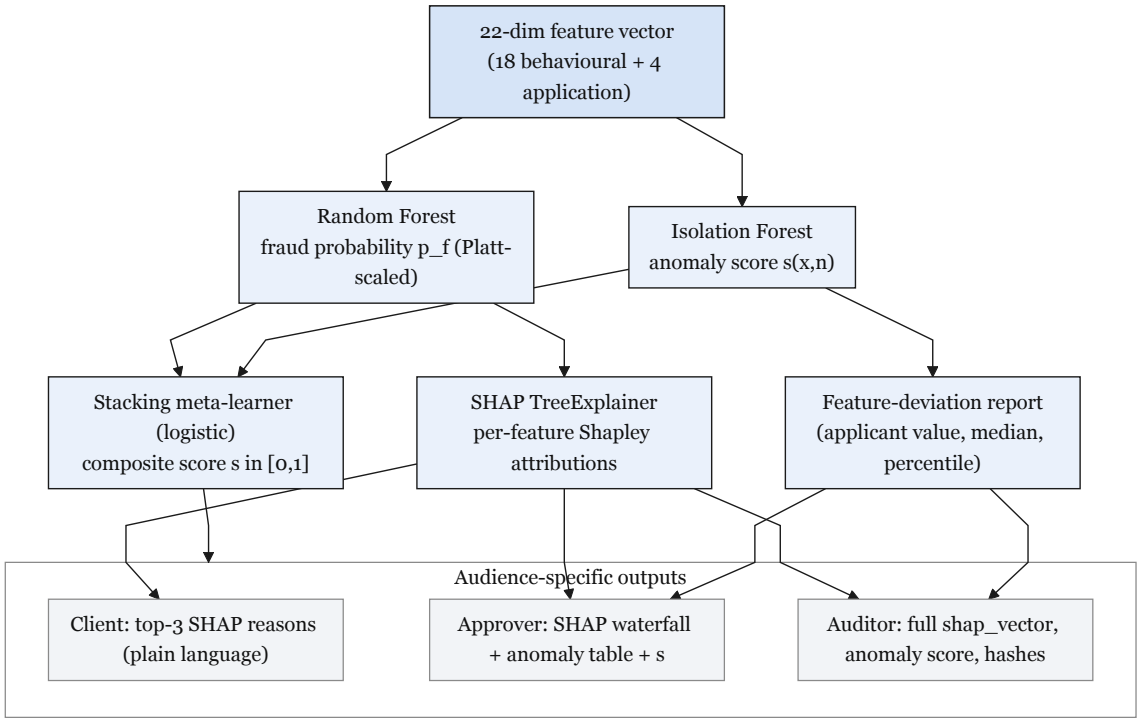


Figure 5.3: Explainability and anomaly-detection flow

5.3 Foundry Invariant and Fuzz Test Suite

The planned test stack combines a Hardhat JavaScript suite for integration tests and deployment scripts with a parallel Foundry suite for unit-level fuzzing and invariant testing because the two frameworks catch different bug classes. The industry pattern is Hardhat for integration, Foundry for invariants and stateful fuzzing.

Three invariants asserted under Foundry's stateful fuzzer.

- Solvency: $\text{totalOutstandingLoans} \leq \text{getTotalReserve}$.
- Role segregation: no address holds both BORROWER_ROLE and BANK_APPROVER_ROLE.
- Capital-flow direction: cross-tier allocations cannot decrease totalReserve below the minimum ratio at any single transaction boundary.

Each invariant is planned to be asserted under 10,000 fuzz runs; counterexamples will be saved as Foundry replay tests so any regression in a future commit fails CI deterministically.

5.4 Implementation Evidence and Traceability

Table 5.3 lists artifacts that exist at pre-thesis submission time in the GitHub repository (repository root layout per `README.md`). These items support Phase C deliverables: runnable code, measured ML benchmarks, and test evidence—not slide-only progress.

Table 5.3: Pre-thesis implementation evidence (repository root).

Component	Path	Status at pre-thesis
Smart contracts	`contracts/*.sol`	WorldBankReserve, NationalBank, LocalBank specified and compilable
Contract tests	`test/*.test.ts`	42 Hardhat unit/integration cases across four suites
Backend API	`backend/src/`	Express routes for loans, banks, auth, income
API tests	`backend/tests/`	Vitest suites for auth, banks, loans, chat, income
Frontend DApp	`frontend/src/`	React shell with Wagmi wallet connect
ML training pipeline	`Documentation/.../ML pipeline - Mac M4/`	BCCC subsample trained; `artifacts/metrics.json`
ML evidence pack	`Documentation/.../evide`	Figures + `metrics.json` copied for thesis
ML service stub	`ml-service/app/`	FastAPI `/health`, `/score` scaffold
Deploy scripts	`scripts/deploy.ts`	Sepolia/Amoy/local deploy ready

Gate G1 demonstration package (end of Phase I): Sepolia-deployed tier contracts, `npm run test:contracts` green, PostgreSQL M1 tables migrated, $\leq$5-minute screen recording of `allocateCapital`.

5.4.1 Requirements Traceability Matrix

Table 5.4 links representative requirements (Section 3.2) to design models, implementation modules, and verification tests. Full MVT item mapping remains in Table 4.1.

Table 5.4: Requirements traceability matrix (excerpt).

Req.	Use case / diagram	Module	Test / evidence	MVT item
FR-01	Figure 3.27	`WorldBankReserve`	`hierarchy.test.`	#1
FR-02	Figures 3.19, 3.22	`LocalBank.sol`, backend loans route	`LocalBank.test.ts`, `loans.test.ts`	#2–3
FR-04	Table 3.15	`BORROWING_LIMIT` + SBT	Planned Foundry invariant	#5
FR-05	Figure 4.1	`ml-service/`, Mac M4 pipeline	Table 5.2	#4, #8
FR-07	Figures 3.3–3.4	Event listener (planned)	PostgreSQL projection tests	DT-I.12–13
FR-08	Figure 3.25	MCP agent (Phase III)	Section 4.5	#11–12
NFR-02	Table 3.26	OpenZeppelin RBAC	`WorldBankReserv`	#13–14

5.5 Discussion and Examiner Limitations

A structured literature scan (June 2026) identified six risks examiners are likely to probe. This section records them explicitly so the thesis does not over-claim.

G1 — Compound novelty vs. partial counterexamples. Maple Finance, Goldfinch, Morpho V2, and Aave Arc provide partial institutional-credit precedents on individual axes [51, 67, 71]. The defensible claim is the *compound* architecture (four-tier reserve enforcement + explainable oracle ML + programmable mutual liability), not "no prior four-tier DeFi."

G2 — BCCC domain transfer. Models trained on BCCC flat-pool transactions may not generalize to hierarchical CWB flows (Section 5.2.1). At pre-thesis stage, a *50,000-address BCCC subsample* provides baseline fraud-detection metrics (F1 = 0.891); full-corpus training and 22-dimensional CWB feature mapping remain for final thesis. Mitigation: acknowledge the gap, benchmark on BCCC as baseline only, collect CWB testnet labels for retraining, and run feature-distribution comparison as Future Work.

G3 — Testnet scope as academic evidence. Working testnet deployment + benchmark tables + documented specifications suffice for workshop venues (AFT, FMBC, IEEE Blockchain workshops). Full research papers additionally require controlled experiments, baselines, threat models, and reproducible public repos. This pre-thesis delivers specification and workload planning; MVT deployment provides the empirical layer.

G4 — Group lending external validity. On-chain mutual liability is cryptographic cosigning, not social solidarity [72, 73]. Repayment-rate claims must not cite BRAC/Grameen historical data as direct validation of smart-contract behaviour.

G5 — Oracle trust and economic viability. Score-at-origination-only mitigates latency/cost; oracle collusion and model staleness remain open (Section 5.2.1). Certora reserve-invariant proofs are Future Work (Section 6); Foundry fuzz tests should list what is asserted, assumed, and out of scope [77].

G6 — Preferred-creditor status and sovereign willingness to repay. The default cascade in `InterBankLendingPool` and related reserve buffers resolves *technical default*—insufficient escrowed collateral, missed installment flows, or reserve shortfall within the protocol. It does **not** create sovereign or institutional willingness to repay. Real multilateral development lending relies partly on preferred-creditor treatment, a diplomatic and reputational mechanism outside any smart contract's reach. CWB therefore does not claim to replicate treaty-backed enforcement or compel repayment from an unwilling institutional participant; that remains a legal and governance-layer question rather than a protocol property.

5.6 Economic Feasibility Analysis

This section provides the economic analysis required by the rubric (CO11): zero-cost prototype costs, gas economics, and an illustrative pilot NPV. Broader sustainability constraints are stated in Section 1.9.

5.6.1 Prototype Cost Model

Table 5.5: Economic feasibility — zero-cost prototype.

Cost Category	Estimate	Notes
Blockchain deployment	$0	Public testnets — no real cryptocurrency required
Frontend hosting	$0	Vercel free tier or localhost for demo
Backend hosting	$0	Render free tier or localhost
AI/ML training	$0	Local machine or Google Colab free tier
Development tools	$0	Hardhat, VS Code, Git — all open-source
Total prototype	**$0**	Entire prototype operates at zero financial cost

5.6.2 Gas Cost and Break-Even

On Polygon zkEVM Cardona (design preference), a full retail loan lifecycle involves roughly 27–32 on-chain state changes. At ≈30 Gwei and ETH at $2,500 (planning assumption), total gas is approximately $0.011—well under 0.01% of interest on a $25,000 USDC loan at 8% APR. On Ethereum mainnet the same lifecycle would cost $2.40–$4.80, reinforcing the Layer 2 design choice [37]. At 8% default with $25,000 average loan size, break-even requires approximately 200 active loans to cover expected default losses under $500/month infrastructure costs.

5.6.3 Net Present Value and Return on Investment

Table 5.6 presents a three-year illustrative NPV and ROI for a post-prototype pilot (not the zero-cost student build). Discount rate $r = 10\%$; cash flows in USDC thousands.

Table 5.6: Illustrative three-year NPV and ROI at pilot scale (300 active loans, $25,000 average).

Year	Benefits ($k)	Costs ($k)	Net ($k)	PV factor @ 10%
0 (setup)	0	6.0	−6.0	1.000
1	48.0	6.0	42.0	0.909
2	72.0	8.0	64.0	0.826
3	96.0	10.0	86.0	0.751
NPV (sum of discounted net)				**$149.6k**
ROI (3-yr benefits − costs) / costs				**1,247%**
Break-even loans (base case)				**≈300**

Year 0 covers audit and infrastructure setup; benefits assume 8% APR interest spread on 300 loans with 8% default. The student prototype itself operates at $0 direct cost (Table 5.5).

Chapter 6

Conclusion

The Crypto World Bank addresses hierarchical development-finance coordination by combining a four-tier institutional reserve hierarchy, explainable oracle-mediated ML, and programmable mutual liability—capabilities absent from flat DeFi lending protocols managing over $55 billion in TVL [8]. At pre-thesis stage, the deliverable is formal specification plus measured ML benchmarks and contract tests; full testnet deployment is scoped to the MVT checklist (Section 4.1).

This thesis makes four contributions: (1) a *compound* four-tier hierarchical DeFi architecture (reserve enforcement + explainable ML + mutual liability); (2) on-chain programmable joint-liability group lending with over-indebtedness controls (Section 3.13); (3) oracle-mediated AI/ML integration with SHAP and anomaly reports (Section 5.2.2), subject to the BCCC domain-transfer gap (Section 5.2.1); and (4) a specified compliance-aware identity pathway (zkKYC, W3C DID/VC) deferred to Future Work. Examiner limitations G1–G6 are consolidated in Section 5.5.

With specified architecture (Chapter 3), empirical evaluation and economic feasibility (Chapter 5), and a phased implementation roadmap, CWB is a structurally distinct contribution to blockchain-based development finance at the *specification and prototype* stage.

MVT completion (final thesis Must).

1. End-to-end hierarchical lending on the chosen prototype testnet (World Bank → National → Local → client).
2. Stablecoin-first deployment with USDC default at retail tiers (Section 1.9.3).
3. Phase II modules: `SavingsVault`, `FixedDeposit`, `GroupLendingPool`, `InterBankLendingPool`, `UpwardDepositFacility`.
4. ML pipeline integration: commit–reveal oracle, Authority Brief, full BCCC retrain (Sections 5.2.2, 5.2.1).
5. MCP conversational agent with human confirmation gate (Section 4.5).
6. Foundry fuzz / invariant suite (Section 5.3).
7. Runtime monitoring: event-listener index and reserve dashboard (Section 4.6).

Post-MVT extensions (GNN ablation, federated learning, zkKYC circuits, Certora proofs, retail payments) are documented in the architecture chapters but are outside the graded MVT minimum.

Appendix A

Database Schema Reference

This appendix records the physical-schema details moved from Chapter 3 to keep the main architecture chapter focused on conceptual design.

A.1 Full Relational ERD

Figure A.1 is the complete logical data model: 51 documented entities plus two Phase-IV configuration tables (`CHAIN_REGISTRY`, `SESSION_KEY_PERMISSION`), with crow's-foot cardinalities, migration-tier colour coding, and a notation key. It is placed here rather than in Chapter 3 because the diagram requires a full landscape page at high magnification to remain legible.

Figure A.1: Full relational ERD: 51 documented entities plus two Phase-IV configuration tables (`CHAIN_REGISTRY`, `SESSION_KEY_PERMISSION`), grouped by migration tier (M1 core, M2, M3, stub, future work).

A.2 Indexing Strategy

B-tree indexes (the PostgreSQL default) support efficient retrieval on frequently queried columns:

Table A.1: Indexing strategy: B-tree indexes for time-window and high-frequency queries.

Index	Table / Column(s)	Rationale
idx_loan_borrower	LOAN_REQUEST(borrower_id)	High-frequency lookup for borrower loan history.
idx_loan_status	LOAN_REQUEST(status, oracle_state)	Efficient filtering of application lifecycle and oracle pipeline states.
idx_installment_due	INSTALLMENT(due_date, status) WHERE status = 'PENDING'	Partial-index range query for pending overdue installment detection.
idx_txn_created	TRANSACTION(created_at)	Time-window aggregation for 6-month and 1-year borrowing limits.
idx_txn_borrower	TRANSACTION(borrower_id)	Per-borrower transaction history retrieval.
idx_market_symbol	MARKET_DATA(symbol, recorded_at)	Composite index for price feed time-series queries.
idx_loan_bank_status	LOAN(local_bank_id, status, created_at DESC)	Loan lifecycle queries by bank and status.
idx_loan_borrower_active	LOAN(borrower_id, status) WHERE status = 'ACTIVE'	Partial index for per-borrower open-loan count.
idx_agent_session_date	AGENT_ACTION_LOG(session_id, created_at DESC)	Per-session agent action history retrieval (join to SESSIONS for borrower).
idx_agent_status	AGENT_ACTION_LOG(status) WHERE status = 'PENDING'	Partial index for the agent status-monitoring loop.
idx_risk_score	LOAN_RISK_ASSESSMENT(anomaly_score DESC) WHERE anomaly_score > 0.5	Partial index for AML alert queue (SAR workflow).
idx_ib_loan_status	INTERBANK_LOAN(status, lender_institution_id)	IBLP default cascade and maturity monitoring.
idx_institution_type	INSTITUTION(institution_type, status)	Tier-scoped institution lookups for multi-entity operations.
idx_txn_institution	TRANSACTION(origin_institution_id, created_at DESC)	Institution-to-institution ledger history retrieval.
idx_chainlink_feed	CHAINLINK_PRICE_ROUND(feed_address, updated_at DESC)	Latest round lookup per Chainlink feed for LTV staleness checks.
idx_session_parent	SESSIONS(parent_session_id) WHERE parent_session_id IS NOT NULL	Partial index for session lineage chain traversal.
idx_collateral_borrower	COLLATERAL_POSITION(borrower_id, loan_id)	Health-factor lookup per borrower and per loan.
idx_event_log_txhash	BLOCKCHAIN_EVENT_LOG(tx_hash)	Deduplication guard: event listener checks this before INSERT.
idx_event_log_block	BLOCKCHAIN_EVENT_LOG(block_number, chain_id)	Catchup-sync replay from a given block.

A.3 Functional Dependencies

Table A.3: Representative functional dependencies.

Relation	Functional Dependency	Notes
LOAN_REQUEST	`request_id` → all attributes	Application row keyed by request identifier.
LOAN_REQUEST	`borrower_id, local_bank_id` → `status, oracle_state, amount, ...`	Lifecycle `status` and oracle pipeline `oracle_state` are separate columns.
LOAN	`loan_id` → all attributes	Disbursement record keyed by loan identifier.
LOAN	`loan_request_id` → `loan_id`	1:1 link from approved request to disbursed loan.
BORROWING_LIMIT	`borrower_id` → `six_month_limit, one_year_limit, on_chain_enforced_limit, last_synced_block, ...`	1:1 with BORROWER; event-listener owned projection.
CREDIT_PASSPORT	`passport_id` → all attributes; `borrower_id` → `passport_id`	Read-model projection of on-chain SBT; event-listener owned.
BORROWER	`wallet_address` → `borrower_id`, all attributes; `registered_local_bank_id` → home Local Bank; `kyc_level` → borrowing-cap tier (Section 3.8)	Unique candidate key in the prototype; Phase IV may use (`wallet_address, chain_id`).
INSTITUTION	`institution_id` → `institution_type, name, wallet_address, contract_address, status`	Supertype for cross-tier FK anchors.
NATIONAL_BANK	`country_code` → `institution_id`	One national reserve per nation (`UNIQUE`).
MODEL_REGISTRY	`model_id` → `model_name, version, training_date, metrics, is_active`	Model lineage for explainability audit.
LOAN_RISK_ASSESSMENT	`assessment_id` → all attributes; `model_id` → `fraud_probability, anomaly_score, shap_vector`	Per-loan ML inference; FK to `MODEL_REGISTRY`.
INSTALLMENT	`loan_id, installment_number` → `amount, due_date, status, payment_event_log_id`	Composite primary key.
SESSIONS	`session_id` → `borrower_id, wallet_address, session_key_hash, session_key_scope, session_key_expires_at, expires_at, revoked`	Retail-client agent sessions only at MVT; bank-authority sessions are Future Work.

A.4 Materialized Views and Agent Query Optimisation

The MCP tool `get_account_state` joins `BORROWER`, `CREDIT_PASSPORT`, `COLLATERAL_POSITION`, active `LOAN` rows, pending `INSTALLMENT` rows, `SAVINGS_ACCOUNT`, and `BORROWING_LIMIT`. To avoid six-to-seven table joins on every agent call, the schema defines a materialized view refreshed by the event listener after any event that modifies component tables:

```
CREATE MATERIALIZED VIEW mv_client_dashboard AS
SELECT
    b.borrower_id,
    b.wallet_address,
    b.kyc_level,
    cp.credit_score,
    cp.risk_tier,
    bl.six_month_remaining,
    bl.one_year_remaining,
    count(l.loan_id) FILTER (WHERE l.status = 'ACTIVE') AS active_loan_count,
    max(i.due_date) FILTER (WHERE i.status = 'PENDING') AS next_installment_due,
    sum(sa.balance) AS total_savings_balance,
    sum(col.locked_amount) AS total_collateral_locked
FROM borrower b
LEFT JOIN credit_passport cp USING (borrower_id)
LEFT JOIN borrowing_limit bl USING (borrower_id)
LEFT JOIN loan l USING (borrower_id)
LEFT JOIN installment i USING (loan_id)
LEFT JOIN savings_account sa USING (borrower_id)
LEFT JOIN collateral_position col USING (borrower_id)
GROUP BY b.borrower_id, b.wallet_address, b.kyc_level,
         cp.credit_score, cp.risk_tier,
         bl.six_month_remaining, bl.one_year_remaining;

CREATE UNIQUE INDEX ON mv_client_dashboard (borrower_id);
```

Refresh strategy: `REFRESH MATERIALIZED VIEW CONCURRENTLY mv_client_dashboard` triggered by the event listener after processing events on projection or lending tables (Phase III).

A.5 Representative SQL Queries

The following queries illustrate Database Design coursework competencies (joins, aggregates, subqueries, and views) on the prototype schema. They map to FR-02, FR-04, and FR-07 in Table 3.1.

Q1 — Active loans per Local Bank with borrower KYC level (INNER JOIN + aggregate).

```
SELECT lb.name AS local_bank,
       COUNT(l.loan_id) AS active_loans,
       AVG(l.principal_amount) AS avg_principal
```

```
FROM loan l
JOIN local_bank lb ON l.local_bank_id = lb.institution_id
JOIN borrower b ON l.borrower_id = b.borrower_id
WHERE l.status = 'ACTIVE'
GROUP BY lb.institution_id, lb.name
ORDER BY active_loans DESC;
```

Q2 — Overdue installments with borrower contact hash (JOIN + filter).

```
SELECT i.loan_id, i.installment_number, i.due_date, i.amount,
       b.wallet_address, b.kyc_level
FROM installment i
JOIN loan l ON i.loan_id = l.loan_id
JOIN borrower b ON l.borrower_id = b.borrower_id
WHERE i.status = 'PENDING' AND i.due_date < CURRENT_DATE
ORDER BY i.due_date ASC;
```

Q3 — Borrowers exceeding six-month rolling limit (subquery).

```
SELECT b.borrower_id, b.wallet_address,
       bl.six_month_limit,
       (SELECT COALESCE(SUM(t.amount), 0)
        FROM transaction t
        WHERE t.borrower_id = b.borrower_id
          AND t.transaction_type = 'LOAN_DISBURSEMENT'
          AND t.created_at >= NOW() - INTERVAL '6 months') AS disbursed_6mo
FROM borrower b
JOIN borrowing_limit bl ON b.borrower_id = bl.borrower_id
WHERE (SELECT COALESCE(SUM(t.amount), 0) FROM transaction t
       WHERE t.borrower_id = b.borrower_id
         AND t.transaction_type = 'LOAN_DISBURSEMENT'
         AND t.created_at >= NOW() - INTERVAL '6 months')
      > bl.six_month_limit;
```

Q4 — High-risk assessments awaiting approver review (JOIN + ORDER BY).

```
SELECT lr.request_id, lra.composite_score, lra.anomaly_score,
       lra.created_at, mr.model_name, mr.version
FROM loan_request lr
JOIN loan_risk_assessment lra ON lr.request_id = lra.loan_request_id
JOIN model_registry mr ON lra.model_id = mr.model_id
WHERE lr.status = 'UNDER_REVIEW'
  AND lra.composite_score BETWEEN 0.4 AND 0.7
ORDER BY lra.composite_score DESC;
```

Q5 — Client dashboard via materialized view (simplified SELECT).

```
SELECT borrower_id, wallet_address, credit_score, risk_tier,
       active_loan_count, next_installment_due, total_savings_balance
```

```
FROM mv_client_dashboard
WHERE borrower_id = $1;
```

Appendix B

Agent Harness Reference

This appendix records the conversational agent harness (Phase III–IV Must/Should deliverables). Section 4.5 in Chapter 4 summarises the six-step pipeline, MCP tool schema, and Authority Brief example. The choice of a locally-hosted 8B-parameter model over a cloud-hosted API (Section C.1) is consistent with the broader 2026 industry shift toward on-device inference for compliance-sensitive financial applications, in which major hardware and platform vendors have positioned on-premise deployment as the only viable path for sectors handling sensitive financial records [67].

B.1 Per-User Context and Prompt Assembly

Per-user personalisation — shared model, per-user context. Every client receives a personalised experience without deploying a separate model per user. Personalisation lives entirely in the per-user context namespace injected into every system prompt by the Express.js context injection layer:

```
{
  "borrower_id": "550e8400-e29b-41d4-a716-446655440000",
  "wallet_address": "0xABC...",
  "language": "en",
  "credit_tier": "Silver",
  "credit_score": 512,
  "active_loans": [
    {
      "loan_id": "L-4821",
      "amount": 100,
      "next_due": "2026-06-15",
      "installment": 17.5
    }
  ],
  "savings_balance": 250.0,
  "pool_utilisation": 0.67,
  "kyc_tier": 1,
  "conversation_history": [ "...last 10 turns..." ]
}
```

Because the on-chain state is re-injected at every turn, the agent always operates on current account data and cannot hallucinate the client's balance or loan status.

Three-tier system prompt assembly. The agent's system prompt is assembled by a dedicated prompt constructor in the Express.js backend, organised into three explicit tiers rather than a single ad-hoc string:

Stable tier. Agent persona, active toolset schema, compliance anchors (EU AI Act Art. 86, FATF R.16, zero-bypass rule). Prefix-cache target; unchanged within a session.
Context tier. Interest rate schedule (Table 3.22), KYC matrix, pool policy, active skill procedure from `AGENT_SKILLS`. Updates on governance votes only.
Volatile tier. Per-user on-chain context namespace, parent-session compression summary, last N conversation turns. Rebuilt every turn from `eth_call` state.

B.2 Session Lineage, Compression, and Toolset Scoping

Session lineage and cross-session memory. The `SESSIONS` table carries a `parent_session_id` self-referential FK. When a conversation reaches a token threshold, the compression path closes the session with `end_reason = 'compression'` and opens a child session seeded with the compression summary. The read tool `get_session_history` exposes ranked excerpts from the client's conversation ancestry.

Context compression strategy. When the assembled prompt approaches the 32K-token window, the backend retains head (H=500) and tail (T=1,000) verbatim, summarises the middle via a secondary Qwen3-8B call, stores `SESSIONS.compression_summary`, and opens a child session. Trigger: 28,000 tokens (87.5% of window).

Toolset	Tools visible
`read_only`	Full-suite taxonomy (Future Work): all 9 read tools; the MVT deploys only the subset needed for the selected 3–5 tools.
`loan_actions`	`read_only` + loan, installment, and group write tools.
`account_management`	`read_only` + deposit, KYC, and reminder write tools.

B.3 Lifecycle Hooks and Optional Capabilities

Lifecycle hook middleware. Write-tool POST requests pass through a hook stack: (1) confirmation audit hook (Phase II) verifies a logged confirmation turn or returns HTTP 403; (2) session-key scope pre-check (Phase III); (3) AML pre-check against `LOAN_RISK_ASSESSMENT` and `SECURITY_EVENT_LOG` (Phase IV).

Agent capabilities.

- **Proactive installment reminders** via `get_installment_schedule` and `pay_installment`.
- **Credit Passport coaching** via `get_credit_score` and Table 3.22.
- **Loan calculator** with live FX from `get_market_data`.
- **Group lending coordination** and **KYC tier upgrade guidance** via write/read tools.
- **Cross-session context retrieval** via `get_session_history`.

Appendix C

Technology Stack

C.1 In-product assistant: local large language model (LLM) integration (prototype)

Purpose. The in-product assistant is a **planned Phase III Must deliverable** (Section 4.5), providing a plain-language interface alongside the React dashboard. It combines a locally hosted, privacy-preserving large language model (Qwen3-8B, Apache 2.0 licence) with a Model Context Protocol (MCP) tool server exposing a minimal MVT subset of **3–5 tools** (Section 4.5). The full **17-tool** suite (9 read tools, including a session history search tool, and 8 write tools requiring human confirmation) remains a specified Future Work expansion rather than the graded prototype baseline. The agent can both answer questions—via retrieval-augmented generation (RAG) from policy documents—and execute on-chain banking operations via the MVT write-tool subset, subject to an explicit human confirmation gate before any state-modifying action is taken. Local hosting keeps conversation data on institutional infrastructure, supporting data-sovereignty requirements in regulated pilots. A client who prefers the React UI experiences no reduction in available functionality, since every operation the agent can perform is equally available through the standard dashboard.

Model and runtime (local development). The inference endpoint follows the **OpenAI Chat Completions** contract (`/v1/chat/completions`) with `stream: true`, proxied from the project backend to a local service such as `http://127.0.0.1:1234`. During early development, the assistant used a locally hosted **Gemma-4-E4B** checkpoint (Google mixture-of-experts model, ∼26B total / 4B active parameters) via LM Studio in GGUF form (e.g., `Q4_K_M` quantization) for a laptop-friendly footprint. For the final thesis evaluation, the assistant is replaced by **Qwen3-8B** (Alibaba Cloud Qwen team, released April 28 2025, Apache 2.0 licence) [67], a dense 8.19B-parameter causal language model with a 32K-token native context window (extendable to 131K tokens via YaRN scaling) and support for 100+ languages and dialects. A distinguishing architectural feature of Qwen3-8B is its *switchable reasoning mode*: the model operates in **thinking mode** (chain-of-thought wrapped in `<think>` tokens, suited to multi-step compliance or policy questions) and **non-thinking mode** (direct, low-latency responses for general navigation queries), selectable per-turn with `/think` or `/no_think` in the system prompt. For Q&A and retrieval-augmented generation (RAG) queries, non-thinking mode is the default; thinking mode is activated only for the red-team regulatory-compliance prompts in the evaluation protocol of Section 4.5. For agent tool-calling, non-thinking mode is the default for read tools and confirmation summaries; thinking mode is activated for complex multi-step operations such as EMI calculation or group signature coordination. The per-user context namespace (see Section 4.4) is prepended to every system prompt as a structured JSON block, so the model has full account context before reading the user's message. The API model identifier is environment-driven; the integration layer remains identical for both models.

End-to-end behavior. The user interface assembles a short transcript (user/assistant messages) and posts it to the backend streaming route. Before the user message reaches the model, the context assembly layer: (a) selects the active toolset based on intent classification, (b) applies prompt injection scanning to all user-sourced fields, and (c) assembles the three-tier system prompt (Stable + Context + Volatile), injecting the compression summary from the parent session into the Volatile tier if this is a continuation session. The active toolset name and any injection scan flag are recorded in `AGENT_ACTION_LOG` for every turn. The backend then opens a streaming request to the local model server, converts upstream token events into **server-sent events (SSE)** for the browser, and the UI incrementally appends text. The message renderer uses Markdown (including GitHub-flavored extensions), math (KaTeX) for `$$...$$` blocks, and line-break rules appropriate for chat transcripts.

For write-tool requests, the agent assembles the full parameter set from the user's request and the injected on-chain state, then presents a confirmation summary. Only after the user sends explicit affirmative consent does the agent call the corresponding MCP write tool via the Express.js banking API layer. The EIP-7702 session key (if active) signs the resulting on-chain transaction within its approved scope. Every executed write tool is logged to `agent_action_log` with the confirmation message ID as the audit reference.

Live on-chain context injection. The Express.js backend injects the client's full on-chain state as a structured JSON context block into every system prompt before forwarding the user's message to the model. The per-user context namespace is:

```
{
  "tier": "volatile",
  "borrower_id": "550e8400-e29b-41d4-a716-446655440000",
  "wallet_address": "0xABC...",
  "language": "en",
  "credit_tier": "Silver",
  "credit_score": 512,
  "active_loans": [
    {
      "loan_id": "L-4821",
      "amount": 100,
      "next_due": "2026-06-15",
      "installment": 17.5
    }
  ],
  "savings_balance": 250.0,
  "pool_utilisation": 0.67,
  "kyc_tier": 1,
  "session_id": "sess_20260602_xyz",
  "parent_session_id": "sess_20260515_abc",
  "compression_summary": "Client asked about Gold tier eligibility.
  Was told 2 more on-time repayments required. No action taken.",
  "conversation_history": [ "...last 10 turns..." ]
}
```

The `compression_summary` field is injected only when this is a child session (i.e., when `parent_session_id` is non-null). It provides the prior-session context without exceeding the Volatile tier token budget. This context block is injected only when the user is authenticated and the `eth_call` round-trip completes within a 200 ms timeout (falling back to a static-context response otherwise). The model uses the injected context to ground its answers in the user's actual account data rather than generic policy text, materially reducing hallucination risk for account-specific queries.

Security and limitations (prototype stance). The agent's safety posture is enforced at four levels. (1) **Tool-schema boundary:** the agent interacts with CWB exclusively through the MVT MCP tool schema (**3–5 tools**) and cannot execute arbitrary code, make unapproved API calls, or read data outside that schema. The full 17-tool suite remains a specified but undeployed Future Work expansion. (2) **Human confirmation gate:** every write tool requires explicit affirmative consent in the conversation history; the confirmation turn ID is logged as the audit reference. (3) **Session key scope:** the EIP-7702 session key is scoped to specific tools, value-capped, time-bound to 24 hours, and revocable at any time. A session that attempts to call an out-of-scope tool is rejected at the key level, not just at the application level. (4) **Prompt injection scanning and lifecycle hook middleware:** user-sourced content is scanned for injection patterns before entering the prompt (Layer 2 control, Section 4.4), and every write-tool HTTP POST is intercepted by a middleware stack that enforces the confirmation invariant at the application layer independently of the model's output. Together, these four levels form a defence-in-depth posture in which no single point of failure can circumvent a critical safety constraint. Hallucination risk is bounded by: (a) the injected on-chain state grounding account-specific answers, and (b) the refusal layer for regulatory and legal queries (100% target on the red-team set, Section 4.5).

Architecture diagram. Figure C.1 shows the compact end-to-end request path; Figure C.2 expands the same flow with component boundaries and authentication context.

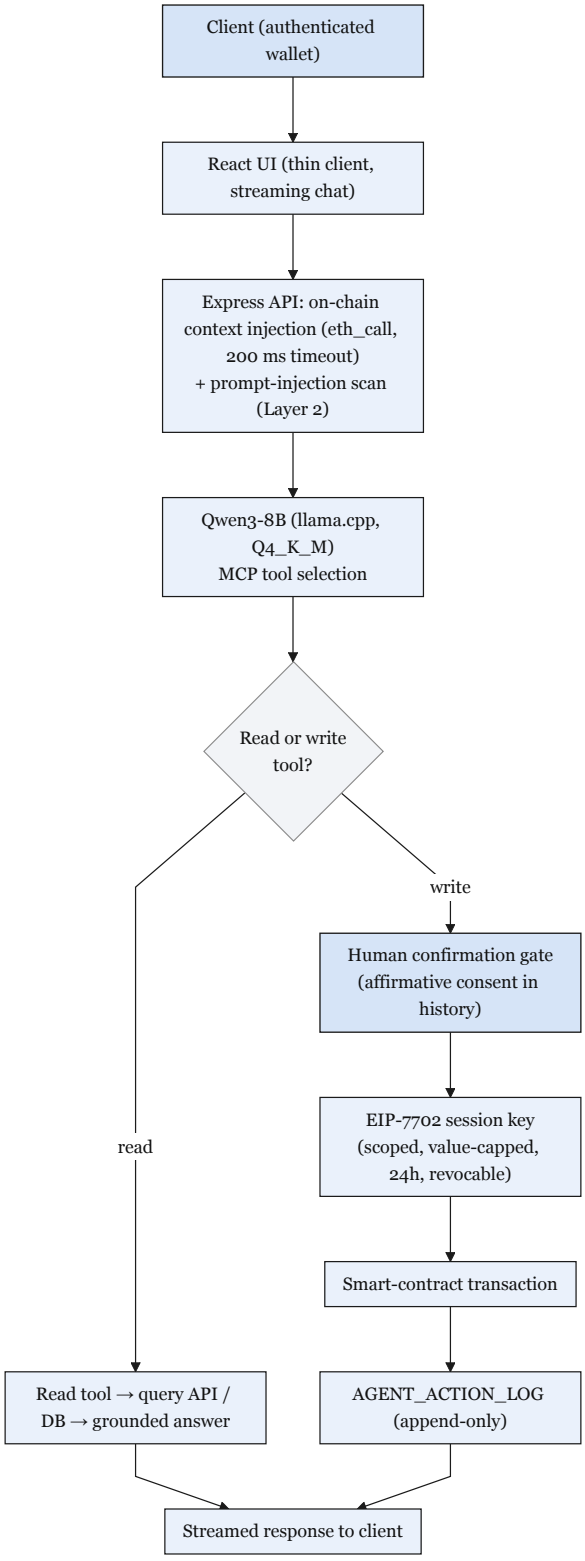


Figure C.1: AI banking agent request path

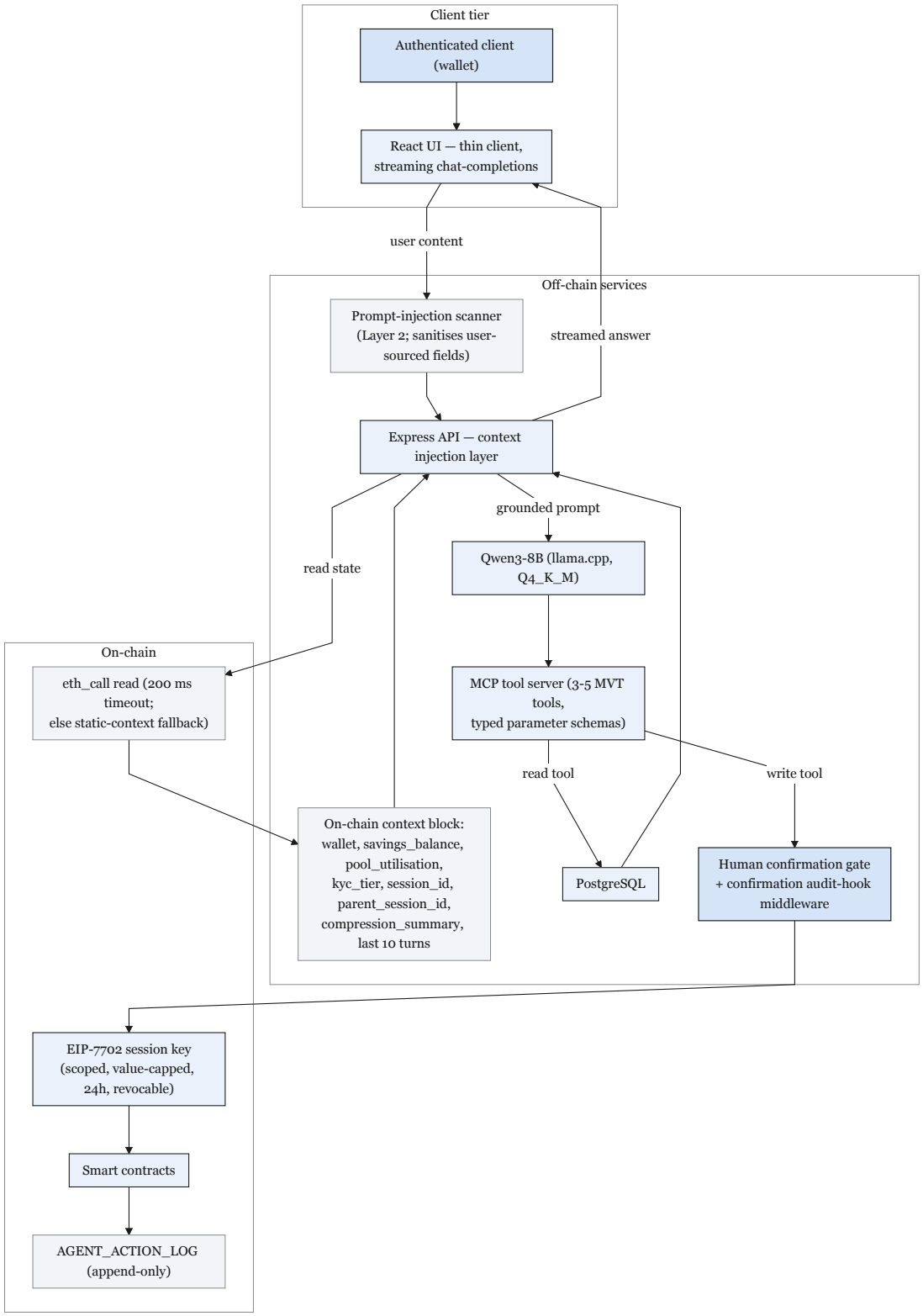


Figure C.2: AI agent data flow

Implementation analysis (brief). The integration is successful for the project prototype because it reuses a stable transport shape (chat-completions with streaming) and isolates the model vendor behind a single API route, allowing the user interface to remain a thin client. The main practical engineering challenges in local development were: (1)

process/port hygiene to keep the API bound to a predictable port, (2) **CORS and same-origin** considerations when the UI and API run on different localhost ports, mitigated with a Vite dev proxy to `/api` and with permissive dev CORS policy on the API, and (3) **RPC provider selection** for wallet reads in the browser, where some public endpoints may fail browser CORS; pinning explicit public RPC endpoints avoids noisy failures unrelated to the chat feature.

Relationship to the earlier rule-based assistant (legacy). The repository also retains early keyword-routed `/api/chatbot` handlers used for initial prototyping; the LLM path is additive. This separation prevents regressions in deterministic demo endpoints while the conversational assistant evolves.

Table C.1: Technology stack summary.

Layer	Technology	Version / Notes
Smart Contract	Solidity, OpenZeppelin	0.8.20; Ownable, ReentrancyGuard
Frontend	React, TypeScript	Material UI; Design 3
Wallet Integration	Wagmi, RainbowKit, Viem	EIP-1193 compliant
Build and Test	Hardhat	Automated test suite; deployment scripts
Backend API	Express.js, Node.js	REST API; middleware architecture
Database	PostgreSQL (designed)	51 entities (31 core + 6 extension + 14 stubs), 3NF; relational integrity
Target Network	Ethereum Sepolia (primary) and/or Polygon Amoy PoS (secondary single-chain prototype)	Stable public testnets for delivery; Polygon zkEVM retained as evaluated design preference
AI Agent Engine	Qwen3-8B (llama.cpp, Q4_K_M)	MCP tool server; per-user context namespace; human confirmation gate
MCP Tool Server	Minimal MVT tool subset (3–5 tools); prompt injection scanner; confirmation audit hook middleware	Exposes a small read+write surface to the agent with typed parameter schemas
Session Keys	EIP-7702 (Future Work)	Scoped, time-bound, value-capped agent wallet authorisation
Oracle Network	Commit–reveal (MVT) + Chainlink Functions (stretch)	Commit–reveal enforces auditable score commitment; DON upgrade when supported
Price Feeds	Chainlink AggregatorV3Interface	Fiat/USD pairs (e.g. EUR/USD) and ETH/USD; 8-decimal precision
Automation	Chainlink Automation	Trustless installment due-date checking; replaces centralised cron job
Proof of Reserve	Chainlink PoR	Cryptographically verifiable WorldBankReserve balance
Event Indexing	PostgreSQL event listener	Stability-first indexing baseline (GraphQL over The Graph optional later)

This appendix table summarizes assistant integration components and configuration aspects used in the prototype (UI, API proxying, and local model server). The purpose is to make the implementation reproducible and to clarify which parts are prototype conveniences versus production requirements.

Appendix D

Smart Contract Capabilities

The complete banking architecture is designed around seventeen modular contracts. The four-contract lending core (`WorldBankReserve`, `NationalBank`, `LocalBank`, `LoanController`) is specified at pre-thesis stage; the remaining thirteen modules are planned across Phases II and III of the final thesis phase.

Specified core contracts (Phase I target):

- **World Bank Reserve Contract:** Reserve custody, deposit handling, national bank registration, capital allocation to national banks, system pause and unpause, emergency withdrawal, and system statistics.
- **National Bank Contract:** Local bank registration, borrowing from the World Bank reserve, capital allocation to local banks, and network status queries.
- **Local Bank Contract:** Client registration, deposit routing, approver management, user account management, and a `freezeAccount(clientId)` function gated by `onlyApprover` modifier, used by the SAR workflow (Section 3.14) to suspend loan and payment operations for a flagged client pending compliance review. Owns a `LoanController` instance for loan-application processing, approval / rejection workflows, and installment-state enforcement.
- **LoanController Contract:** On-chain loan state machine (request, approve, reject, disburse, installment payment); specified as a Phase I deploy shell with functional logic completed in Phase II (DT-II.01–03).

Planned contracts – Banking product suite (Phase II):

- **SavingsVault Contract:** Standard savings account management, variable yield accrual, withdrawal processing, and deposit-to-lending pool integration.
- **FixedDeposit Contract:** Term deposit creation, lock period enforcement, agreed APY accrual, early withdrawal penalty calculation, and maturity release (Stub until contract ships).
- **GroupLendingPool Contract:** Group formation, multi-signature consent recording, shared collateral management, per-member disbursement, mutual liability enforcement, and group credit history recording.
- **InsuranceFund Contract:** Captures 5% of all interest collected across the lending hierarchy, processes default coverage disbursement claims, and publishes its reserve balance to Chainlink Proof of Reserve for external verifiability (Phase I Should, commonly deferred to Phase II).
- **CurrentAccount Contract:** Transactional account management, atomic peer transfers, recurring payment scheduling, and payroll deposit handling (Stub until contract ships).

Planned contracts – Multi-entity and cross-tier operations (Phases II and III):

- **InterBankLendingPool Contract** (Section 3.11.1): one instance per tier (`IBLP_NB`, `IBLP_LB`). Short-tenor (1 / 7 / 30 day) same-tier borrowing with a utilization-kinked rate model bounded by the tier-above downward rate; default cascade through reserve buffer → InsuranceFund → parent-tier backstop.
- **UpwardDepositFacility Contract** (Section 3.11.2): role-restricted extension of SavingsVault accounting that lets a lower-tier bank deposit excess reserves into its parent tier, earning a strictly lower yield than the downward borrowing rate; withdrawal gate enforces depositor reserve-ratio safety.
- **FXModule Contract** (Phase III): oracle-priced currency conversion, spread calculation, dual-currency lending denomination support, and conversion audit trail.
- **LiquidationEngine Contract** (Phase III, Section 3.9.2): health-factor monitoring; pays third-party liquidators a bonus when $HF < 1.0$.
- **SyndicatedLoan Contract** (Section 3.11.3): syndicate proposal, subscription window, supermajority on-chain consent vote (75%-by-capital-share), atomic disbursement, pro-rata interest and recovery distribution; supports cross-tier syndication (mixed NB / LB co-funders).
- **TranchedPool Contract** (Section 3.11.4): senior / junior tranching with a subordination-ratio policy, waterfall interest and principal payments, first-loss absorption by the junior tranche; suitable for risk-segmented institutional and impact-aligned capital.
- **TreasurySwap Contract** (Phase III, Section 3.11.5): oracle-priced atomic asset swap restricted to wallets holding a banking role; cross-tier asset exchange (e.g., NB ETH treasury → USDC) with a tighter spread than retail FX; post-swap reserve-ratio invariant enforced before state change.
- **NettingEngine Contract** (Future Work, Section 6): multilateral netting with partial settlement and challenge-period dispute resolution—not specified at implementation depth in this report.

Appendix C: Planned Testnet Deployment Manifest

The following table is the **consolidated planned deployment manifest template** for the prototype public testnet deployment (Ethereum Sepolia as primary and/or Polygon Amoy PoS as secondary). Polygon zkEVM remains an evaluated option in the platform comparison section; it is not the default narrative deployment chain for gates, MVT items, or phase deliverables. No contracts are claimed as live-deployed at pre-thesis submission. Addresses will be recorded in the project repository under `/deployments/testnet/` as phases complete and can be verified on the relevant block explorers.

Table D.1: Planned testnet deployment manifest (consolidated Phase IV template; Phase I–III rows populated incrementally on a single chosen prototype testnet).

Contract	Network	Address / manifest path
WorldBankReserve	Prototype testnet	`deployments/testnet/WorldBankReserve.json`
NationalBank (Scaffold)	Prototype testnet	`deployments/testnet/NationalBank.json`
LocalBank (Scaffold)	Prototype testnet	`deployments/testnet/LocalBank.json`
LoanController	Prototype testnet	`deployments/testnet/LoanController.json`
MockUSDC	Prototype testnet	`deployments/testnet/MockUSDC.json`
SavingsVault	Prototype testnet	`deployments/testnet/SavingsVault.json`
GroupLendingPool	Prototype testnet	`deployments/testnet/GroupLendingPool.json`
InterBankLendingPool	Prototype testnet	`deployments/testnet/InterBankLendingPool.json`
UpwardDepositFacility	Prototype testnet	`deployments/testnet/UpwardDepositFacility.json`
LiquidationEngine	Prototype testnet	`deployments/testnet/LiquidationEngine.json`

Note: Exact hex addresses will be populated after Phase IV testnet deployment. The repository manifest is the authoritative record. This appendix documents the intended contract inventory only; it does not assert live deployment at pre-thesis stage.

Appendix D: WorldBankReserve Contract Interface

The following Solidity interface documents the public API of the `WorldBankReserve` contract—the formally specified Tier 1 contract (Phase I deployment target). Three design annotations follow the listing:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/security/ReentrancyGuard.sol";
import "@openzeppelin/contracts/access/AccessControl.sol";

/// @title WorldBankReserve
/// @notice Tier 1 reserve contract: manages global capital allocation
///         to registered National Bank addresses.
contract WorldBankReserve is ReentrancyGuard, AccessControl {

    bytes32 public constant NATIONAL_BANK_ROLE = keccak256("NATIONAL_BANK_ROLE");
    bytes32 public constant AUDITOR_ROLE       = keccak256("AUDITOR_ROLE");

    uint256 public totalReserve;
    uint256 public minimumReserveRatio;   // e.g. 1000 = 10.00%
    mapping(address => uint256) public allocatedTo; // NationalBank => amount

    event CapitalAllocated(address indexed nationalBank, uint256 amount);
    event RepaymentReceived(address indexed nationalBank, uint256 amount);
    event ReserveRatioUpdated(uint256 newRatio);

    /// @notice Allocate capital downward to a registered National Bank (CEI
        pattern)
    function allocateCapital(address nationalBank, uint256 amount)
        external onlyRole(DEFAULT_ADMIN_ROLE) nonReentrant;

    /// @notice Record repayment from a National Bank (CEI pattern)
    function recordRepayment(uint256 amount)
        external onlyRole(NATIONAL_BANK_ROLE) nonReentrant;

    /// @notice Query current utilization ratio (scaled 1e4)
    function utilizationRate() external view returns (uint256);

    /// @notice Returns true if reserve ratio constraint is satisfied
    function isReserveAdequate() external view returns (bool);

    /// @notice Returns a four-value reserve summary for Proof of Reserve
    ///         verification and the Reserve Transparency Dashboard.
    /// @return totalDeposited  Gross deposits ever made to this reserve tier
    /// @return totalLoaned     Outstanding principal lent to National Banks
    /// @return reserveRatio    (totalDeposited - totalLoaned) * 1e4 /
        totalDeposited
    ///                         (e.g. 5000 = 50.00%)
    /// @return insuranceFundBalance  Current balance of the InsuranceFund contract
    function getReserveSummary() external view returns (
        uint256 totalDeposited,
        uint256 totalLoaned,
        uint256 reserveRatio,
        uint256 insuranceFundBalance
    );
}
```

Listing D.1: WorldBankReserve.sol – Tier 1 public interface. Full implementation in project repository.

Design annotation 1 — `nonReentrant` on state-changing functions. The

`nonReentrant` modifier from OpenZeppelin's `ReentrancyGuard` sets a boolean lock before execution and clears it after, preventing recursive calls. It is applied to `allocateCapital` and `recordRepayment` because both involve ETH transfers that could trigger attacker-controlled fallback functions. The full Checks-Effects-Interactions analysis is in Section 3.9.1.

Design annotation 2 — `AccessControl` mapping to RBAC design. OpenZeppelin's `AccessControl` maps each on-chain role (e.g., `NATIONAL_BANK_ROLE`) to a `bytes32` keccak256 hash stored in a role-to-address mapping. Role assignment is controlled by the `DEFAULT_ADMIN_ROLE` (the World Bank admin). This directly implements the four-tier RBAC hierarchy described in Section 3.1: only addresses granted `NATIONAL_BANK_ROLE` can call `recordRepayment`, and only the World Bank admin can call `allocateCapital`.

Design annotation 3 — Events enable off-chain indexing for the analytics layer. `CapitalAllocated`, `RepaymentReceived`, and `ReserveRatioUpdated` are emitted at every significant state transition. The off-chain Express.js event listener subscribes to these events and writes them to PostgreSQL, enabling the AI/ML monitoring layer to construct loan lifecycle timelines, compute utilization windows, and feed the Random Forest fraud detector without requiring costly repeated `SLOAD` calls.

Appendix E: Internal Industry Research Notes

This appendix catalogues supplementary industry-research documents prepared for the Crypto World Bank project. They informed Chapter 3 (actor taxonomy), Chapter 5 (economic feasibility), and the no-native-token design constraint. All files reside in the project repository under `Documentation/2nd Phase (Bokhtiar)/Improvements/`.

Table D.2: Internal industry research corpus.

ID	Document	Thesis use
IR-1	`Binance_FTX_Full_Report.md`	CEX vs. CWB comparison; FTT/BNB token lesson; SAFU and Binance Earn analogues; PoR and commingling safeguards
IR-2	`blockchain platforms research.csv`	Five-exchange matrix (Binance, Bybit, Coinbase, OKX, Kraken)
IR-3	`Binance Business Analysis.md`	Exchange revenue taxonomy (fees, listing, OTC)
IR-4	`binance_software_engineering_-architecture.md`	Nine-actor taxonomy (Section 3.8); use-case and sequence-diagram conventions
IR-5	`binance_architecture_-research.md`	Off-chain PostgreSQL/Redis patterns (background; not cited in body text)

These documents are project-internal research notes, not peer-reviewed publications. They are listed here for reproducibility and examiner traceability; the body text cites them via Appendix D (entries IR-1–IR-4), not numbered bibliography entries.

Appendix F: UI Prototype Wireframes (Figma)

This appendix holds high-fidelity **Figma prototype exports** for the two primary user interfaces specified in Section 3.12.6. Keeping visual mockups at the end of the report follows the convention of separating *design specification* (Chapter 3) from *visual deliverables* (appendix), and matches System Analysis and Design coursework where UI prototypes are often submitted as supplementary figures.

Retail loan application flow. Screens cover wallet connect, KYC status, loan amount and term entry, APR/installment preview, and post-submission tracking. Export as PDF or PNG from Figma and replace the placeholder below.

Figure D.1: Retail loan application prototype (Figma).

Authority Brief approver dashboard. Screens cover the composite risk score, SHAP feature breakdown, anomaly deviation table, and approve/decline actions for bank approvers. Export as PDF or PNG from Figma and replace the placeholder below.

Figure D.2: Authority Brief approver dashboard prototype (Figma).

Optional additions. Further Figma boards (onboarding funnel, reserve dashboard, mobile navigation) may be appended here without changing the main chapter structure.

References

Reference verification (29 June 2026). Web-accessible bibliography entries were checked individually for link availability and title–URL alignment. Credibility fixes in this revision: correspondent-banking costs and delays on CPMI [15] and FSB [23] (not vendor product pages); Ozili eNaira adoption study [100]; BCCC dataset [99]; Chaliasos ICSE security-tool evaluation [91]; IEEE microloan full metadata [97]; and Appendix D for project-internal industry notes. Entries without access dates are stable DOI, arXiv, or publisher records; dynamic market statistics (DefiLlama, Fensory) should be re-checked before final thesis submission. The bibliography contains 100 numbered entries.

[1] S. M. Werner, D. Perez, L. Gudgeon, A. Klages-Mundt, D. Harz, and W. J. Knottenbelt. 2022. SoK: Decentralized Finance (DeFi). *arXiv preprint arXiv:2101.08778*. https://arxiv.org/abs/2101.08778

[2] M. Bastankhah, V. Nadkarni, C. Jin, S. Kulkarni, and P. Viswanath. 2024. Thinking Fast and Slow: Data-Driven Adaptive DeFi Borrow-Lending Protocol. In *Proc. 6th Conf. Advances in Financial Technologies (AFT)* (LIPIcs, Vol. 316). Article 27, 23 pages. https://doi.org/10.4230/LIPIcs.AFT.2024.27

[3] G. Palaiokrassas, S. Scherrers, I. Ofeidis, and L. Tassiulas. 2024. Leveraging Machine Learning for Multichain DeFi Fraud Detection. In *Proc. IEEE Int. Conf. Blockchain and Cryptocurrency (ICBC)*. https://doi.org/10.1109/ICBC59979.2024.10634350

[4] D. Adom, E. O. Acheampong, and M. Boateng. 2022. LIME and SHAP: A Comparison of Model-Agnostic Approaches to Explainability in Loan Approval Systems. *International Journal of Advanced Computer Science and Applications* 13, 11. https://thesai.org/Downloads/Volume13No11/Paper_90-LIME_and_SHAP_A_Comparison.pdf

[5] B. W. Tan. 2023. Central Bank Digital Currency and Financial Inclusion. *IMF Working Paper WP/23/69*. https://www.imf.org/en/Publications/WP/Issues/2023/03/27/Central-Bank-Digital-Currency-and-Financial-Inclusion-531273

[6] F. T. Liu, K. M. Ting, and Z.-H. Zhou. 2008. Isolation Forest. In *Proc. IEEE Int. Conf. Data Mining (ICDM)*, pages 413–422. https://doi.org/10.1109/ICDM.2008.17

[7] N. Atzei, M. Bartoletti, and T. Cimoli. 2017. A Survey of Attacks on Ethereum Smart Contracts (SoK). In *Proc. Int. Conf. Principles of Security and Trust (POST)*, pages 164–186. https://doi.org/10.1007/978-3-662-54455-6_8

[8] DefiLlama. 2024. Lending Protocols — DeFi TVL and Protocol Rankings. https://defillama.com/protocols/Lending

[9] World Bank. 2021. The Global Findex Database 2021: Financial Inclusion, Digital Payments, and Resilience. https://www.worldbank.org/en/publication/globalfindex

[10] Aave. 2024. Aave Protocol Documentation. https://docs.aave.com/

[11] Compound. 2024. Compound Protocol Documentation. https://docs.compound.finance/

[12] World Bank. 2022. World Development Report 2022: Finance for an Equitable Recovery. https://www.worldbank.org/en/publication/wdr2022

[13] International Finance Corporation (IFC). 2017. MSME Finance Gap: Assessment of the Shortfalls and Opportunities in Financing Micro, Small, and Medium Enterprises in Emerging Markets. World Bank. https://openknowledge.worldbank.org/entities/publication/ff4c9839-21ac-5676-a23a-7cf6f745df0c

[14] Bank for International Settlements. 2024. DeFi Lending: Intermediation Without Information?. BIS Working Paper No. 1183. https://www.bis.org/publ/work1183.pdf

[15] Committee on Payments and Market Infrastructures and Bank for International Settlements. 2016. Correspondent Banking — Technical Report. CPMI Papers No. 147. https://www.bis.org/cpmi/publ/d147.pdf

[16] Ripple Labs. 2025. RLUSD Stablecoin Product Overview. Ripple Documentation. https://ripple.com/solutions/stablecoin/

[17] World Bank. 2024. Remittance Prices Worldwide: Making Markets More Transparent. Migration and Development Brief 40. https://remittanceprices.worldbank.org/

[18] DefiLlama. 2026. Aave V3 TVL, Fees, Revenue & Income Statement. March. https://defillama.com/protocol/aave-v3

[19] DefiLlama. 2026. Compound V3 TVL, Fees, Revenue & Income Statement. March. https://defillama.com/protocol/compound-v3

[20] Fensory. 2026. Sky Protocol Projects $611M Revenue in 2026 as USDS Supply Targets $20.6 Billion. February. https://www.fensory.com/intelligence/rwa/sky-protocol-tokenization-regatta-solana-february-2026

[21] Fensory. 2026. Maple Finance — Institutional DeFi Lending Protocol. https://www.fensory.com/insights/protocols/maple-finance

[22] Fensory. 2026. DeFi Credit Platforms Hit $2.4B Milestone. March. https://www.fensory.com/intelligence/rwa/defi-private-credit-platforms-2-4-billion-loans-ethereum

[23] Financial Stability Board. 2020. Enhancing Cross-border Payments: Stage 3 Roadmap. https://www.fsb.org/2020/10/enhancing-cross-border-payments-stage-3-roadmap/

[24] GlobeNewswire. 2025. R3's Corda Leads Tokenized RWA Market with Over $10 Billion in On-chain Assets. February. https://www.globenewswire.com/news-release/2025/02/13/3025637/0/en/R3-s-Corda-leads-tokenized-RWA-market-with-over-10-billion-in-on-chain-assets-and-unrivalled-industry-adoption.html

[25] World Bank. 2025. World Bank Group Tracks Project Funds with New Blockchain Tool. Press Release, September. https://www.worldbank.org/en/news/press-release/2025/09/26/world-bank-group-tracks-project-funds-with-new-blockchain-tool

[26] JPMorgan. 2025. Kinexys Digital Payments: Real-Time Multicurrency Payments. https://www.jpmorgan.com/kinexys/index

[27] World Economic Forum. 2022. Why Decentralized Finance Is a Leapfrog Technology for the 1.1 Billion People Who Are Unbanked. September. https://www.weforum.org/stories/2022/09/decentralized-finance-a-leapfrog-technology-for-the-unbanked/

[28] Opera Newsroom. 2026. 160M CELO Allocation Proposal to Grow Opera from Distribution Partner into Key Network Stakeholder. March. https://press.opera.com/2026/03/19/opera-celo-partnership-2026/

[29] Morpho. 2026. Network Data. March. https://data.morpho.org/

[30] Stellar. 2025. End of Year 2025 Report — Execution at Scale. https://www.stellar.org/blog/foundation-news/2025-year-in-review

[31] Y. Chen, S. Bin, and H. He. 2024. Design and Implementation of a Multi-Chain Lending Model in Blockchain. In *Proc. 3rd Int. Conf. Artificial Intelligence and Computer Information Technology (AICIT)*, pages 97–102.

[32] S. Yang and W. Cui. 2023. An Evaluation System for DeFi Lending Protocols. *arXiv preprint arXiv:2303.01022*. https://arxiv.org/abs/2303.01022

[33] J. Hartmann and O. Hasan. 2021. A Social-Capital Based Approach to Blockchain-Enabled Peer-to-Peer Lending. In *Proc. IEEE Int. Conf. Blockchain Computing and Applications (BCCA)*, pages 105–110. https://hal.science/hal-03371872

[34] P. T. Hasan, H. K. T. Akhter, M. R. Tahsinur, A. B. Haque, A. K. M. N. Islam, and R. M. Rahman. 2022. Blockchain and Machine Learning for Fraud Detection: A Privacy-Preserving and Adaptive Incentive Based Approach. *IEEE Access*, vol. 10, pages 87115–87131. https://ieeexplore.ieee.org/document/9857827

[35] Bank for International Settlements et al. 2020. Central Bank Digital Currencies: Foundational Principles and Core Features. BIS and Central Banks Joint Report. https://www.bis.org/publ/othp33.htm

[36] M. Asaduzzaman, F. Hasib, and Z. B. Hafiz. 2020. Towards Using Blockchain Technology for Microcredit Industry in Bangladesh. In *Proc. 23rd Int. Conf. Computer and Information Technology (ICCIT)*, pages 1–6. https://doi.org/10.1109/ICCIT51783.2020.9392730

[37] H. Kim and D. Kim. 2024. Optimal Gas Fee Minimization in DeFi: Enhancing Efficiency and Security on the Ethereum Blockchain. *IEEE Access*, vol. 12, pages 173810–173823. https://ieeexplore.ieee.org/document/10750187

[38] X. Yuan. 2024. SHAP-based Interpretable Models for Credit Default Assessment Using Machine Learning. In *Proc. 14th Int. Conf. Software Technology and Engineering (ICSTE)*, pages 213–217. https://doi.org/10.1109/ICSTE63875.2024.00044

[39] T. Dao, T. Trinh, and V. Pham. 2025. Optimizing Credit Scoring Models for Decentralized Financial Applications. In *Information and Communication Technology*, Springer. https://doi.org/10.1007/978-981-96-4282-3_36

[40] A. Beniiche. 2020. A Study of Blockchain Oracles. *arXiv preprint arXiv:2004.07140*. https://arxiv.org/pdf/2004.07140

[41] A. Pasdar, Y. C. Lee, and Z. Dong. 2023. Connect API with Blockchain: A Survey on Blockchain Oracle Implementation. *ACM Computing Surveys*, 55, 10. https://doi.org/10.1145/3567582

[42] L. Gudgeon, S. M. Werner, D. Perez, and W. J. Knottenbelt. 2020. DeFi Protocols for Loanable Funds: Interest Rates, Liquidity and Market Efficiency. In *Proc. 2nd ACM Conf. Advances in Financial Technologies (AFT)*, pages 92–112. https://doi.org/10.1145/3419614.3423254

[43] K. W. Wu. 2024. Strengthening DeFi Security: A Static Analysis Approach to Flash Loan Vulnerabilities. *arXiv preprint arXiv:2411.01230*.

[44] Y. Li et al. 2025. A Comprehensive Study of Exploitable Patterns in Smart Contracts: From Vulnerability to Defense. *arXiv preprint arXiv:2504.21480*. https://arxiv.org/abs/2504.21480

[45] F. Piper, K. Wolf, and J. Heiss. 2025. Privacy-Preserving On-chain Permissioning for KYC-Compliant Decentralized Applications. TU Berlin, *arXiv preprint arXiv:2510.05807*.

[46] N. Decker. 2025. Zero-Knowledge Proofs: Cryptographic Model for Financial Compliance and Global Banking Security. *SSRN Working Paper No. 5170068*.

[47] Grameen Bank. 2024. Grameen Bank at a Glance. Grameen Communications. https://www.grameen-info.org/

[48] A. Carstens et al. 2021. Stablecoins: Risks, Potential and Regulation. *BIS Working Papers No. 905*, Bank for International Settlements. https://www.bis.org/publ/work905.pdf

[49] J. A. Ocampo and K. Gallagher. 2024. The Role of Multilateral Development Banks and Development Assistance in the Provision of Global Public Goods. UNDP Background Paper. https://hdr.undp.org/system/files/documents/background-paper-document/2024jaocampokdgonzaleztheroleofmultilateraldevlmntbanks.pdf

[50] F. A. Aponte-Novoa, A. L. S. Orozco, R. Villanueva-Polanco, and P. Wightman. 2021. The 51% Attack on Blockchains: A Mining Behavior Study. *IEEE Access*, vol. 9, pages 140549–140564. https://doi.org/10.1109/ACCESS.2021.3119291

[51] Sygnum Bank. 2026. Institutional DeFi in 2025: The Disconnect Between Infrastructure and Allocation. Sygnum Blog, May 2025. https://www.sygnum.com/blog/2025/05/30/institutional-defi-in-2025-the-disconnect-between-infrastructure-and-allocation/

[52] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, et al. 2021. The PRISMA 2020 statement: an updated guideline for reporting systematic reviews. *BMJ*, vol. 372, n71. https://doi.org/10.1136/bmj.n71

[53] D. Cheng, X. Wang, Y. Zhang, and L. Zhang. 2022. Graph Neural Network for Fraud Detection via Spatial-Temporal Attention. *IEEE Trans. Knowledge and Data Engineering*, 34, 8, pages 3800–3813. https://ieeexplore.ieee.org/document/9356317

[54] D. Wang, B. Wu, X. Yuan, L. Wu, Y. Zhou, and H. Cui. 2024. DeFiGuard: A Price Manipulation Detection Service in DeFi Using Graph Neural Networks. *arXiv preprint arXiv:2406.11157*. https://arxiv.org/abs/2406.11157

[55] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas. 2017. Communication-Efficient Learning of Deep Networks from Decentralized Data. In *Proc. Int. Conf. Artificial Intelligence and Statistics (AISTATS)*.

[56] Z. Chen, J. Chen, M. Sra, et al. 2025. Standard Benchmarks Fail — Auditing LLM Agents in Finance Must Prioritize Risk. *arXiv preprint arXiv:2502.15865*. https://arxiv.org/abs/2502.15865

[57] MicroSave Consulting. 2025. Shaping a Sustainable Microfinance Sector in Bangladesh. MSC Signature Project, September. https://www.microsave.net/signature_projects/shaping-a-sustainable-microfinance-sector-in-bangladesh-through-mscs-transformation-feasibility-study/

[58] European Parliament and Council of the European Union. 2024. Regulation (EU) 2023/1114 of 31 May 2023 on Markets in Crypto-Assets (MiCA). *Official Journal of the European Union*, L 150/40, June 2023; fully applicable from 30 December. https://eur-lex.europa.eu/eli/reg/2023/1114/oj

[59] European Banking Authority. 2024. Guidelines on the Minimum Content of the Governance Arrangements for Issuers of Asset-Referenced Tokens under MiCAR. EBA/GL//06, June. https://www.eba.europa.eu/activities/single-rulebook/regulatory-activities/asset-referenced-and-e-money-tokens-micar/guidelines-internal-governance-arrangements-issuers-arts-under-micar

[60] United States Congress. 2025. Guiding and Establishing National Innovation for U.S. Stablecoins (GENIUS) Act. Pub. L. No. 119-27, July 18. https://www.congress.gov/bill/119th-congress/senate-bill/919

[61] Bank for International Settlements Innovation Hub. 2024. Project mBridge: Connecting Economies through CBDC — Reaches Minimum Viable Product. BIS Innovation Hub Report, June. https://www.bis.org/about/bisih/topics/cbdc/mcbdc_bridge.htm

[62] Bank for International Settlements and Central Banks of France, Japan, Korea, Mexico, Switzerland, the UK, and the US. 2024. Project Agora: Tokenization of Cross-Border Payments using Unified Ledgers. BIS Innovation Hub Working Paper, April. https://www.bis.org/about/bisih/topics/fmis/agora.htm

[63] Bangladesh Bank. 2025. FE Circular No. 06: Electronic Communication for Letters of Credit and Trade Finance Instruments. Foreign Exchange Policy Department, January. https://www.bb.org.bd/mediaroom/circulars/fepd/jan172025fepd06.pdf

[64] Bangladesh Bank. 2025. Financial Inclusion in Bangladesh: Progress and Gender-Disaggregated Outlook 2024–25. Special Publication SP-02, Financial Inclusion Department, June. https://www.bb.org.bd/pub/special/SP2025-02.pdf

[65] BRAC. 2025. BRAC Microfinance Annual Performance Report 2025: Women-Centric Lending Outcomes. BRAC Research and Evaluation Division. https://www.brac.net/program/microfinance/

[66] Ethereum Improvement Proposals. 2025. EIP-4626: Tokenized Vault Standard. Ethereum Foundation, April 2022. ; "EIP-7540: Asynchronous ERC-4626 Tokenized Vaults," Ethereum Foundation, 2023. ; "EIP-3643: T-REX – Token for Regulated EXchanges," Ethereum Foundation, 2021. See also: Spectral Finance, "On-Chain Credit Scoring for DeFi Lending," 2023. ; RociFi Labs, "Undercollateralized DeFi Lending via On-Chain Credit Score," 2023. ; Qwen Team (Alibaba Cloud), "Qwen3 Technical Report," *arXiv preprint arXiv:2505.09388*. https://eips.ethereum.org/EIPS/eip-4626

[67] Ethereum Improvement Proposals. 2022–2025. EIP-4626, EIP-7540, EIP-3643; Spectral Finance and RociFi documentation; Qwen Team. 2025. Qwen3 Technical Report (*arXiv:2505.09388*). https://eips.ethereum.org/EIPS/eip-4626

[68] Elliptic. 2019. The Elliptic Data Set. Kaggle. https://www.kaggle.com/datasets/ellipticco/elliptic-data-set

[69] M. Alizadeh, M. Gholampour, A. Imani, and J. Schulz. 2026. Simple Prompt Injection Attacks Can Leak Personal Data Observed by LLM Agents During Task Execution. *arXiv preprint arXiv:2506.01055*, University of Zurich / IPM, June.

[70] OWASP Foundation. 2026. OWASP Top 10 for Large Language Model Applications and Agents, 2026 Edition. OWASP Foundation. https://owasp.org/www-project-top-10-for-large-language-model-applications/

[71] E. Gamma, R. Helm, R. Johnson, and J. Vlissides. 1994. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

[72] G. Aspris and J. Svec. 2025. Institutionalizing Risk Curation in Decentralized Credit. *arXiv preprint arXiv:2512.11976*. https://arxiv.org/abs/2512.11976

[73] S. Jaffer. 1999. Microfinance and the Mechanics of Solidarity Lending. Harvard Law School, *SSRN Working Paper No. 162548*. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=162548

[74] R. Sherratt. 2016. From Solidarity to Coercion: The Dynamics of Group Liability. *Oxford Economic Papers*, 68, 4, pages 955–974. https://doi.org/10.1093/oep/gpw018

[75] H. Fazliani, A. Pasdar, and Y. C. Lee. 2025. Ensemble-Based Semi-Supervised Learning for Illicit Account Detection in Ethereum DeFi Transactions. *arXiv preprint arXiv:2412.02408*, rev. October. https://arxiv.org/abs/2412.02408

[76] H. Alhasawi, A. Almutairi, and S. Alharbi. 2025. FedFraud: A Federated Approach to Scalable and Trustworthy Financial Fraud Detection. *Security and Privacy*, Wiley. https://doi.org/10.1002/spy2.70012

[77] European Parliament and Council of the European Union. 2026. Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (AI Act). *Official Journal of the European Union*, L 2024/1689, July 2024; high-risk credit-scoring provisions applicable from August. https://eur-lex.europa.eu/eli/reg/2024/1689/oj

[78] L. Dong, Y. Qiu, and F. Xu. 2023. Blockchain-Enabled Deep-Tier Supply Chain Finance. *Manufacturing & Service Operations Management*, 25, 6, pages 2021–2037. https://doi.org/10.1287/msom.2022.1123

[79] U. Bindseil. 2020. Tiered CBDC and the Financial System. *ECB Working Paper Series*, no. 2351, January. https://www.ecb.europa.eu/pub/pdf/scpwps/ecb.wp2351~c8c18bbd60.en.pdf

[80] J. Kiff, J. Alwazir, S. Davidovic, A. Farias, A. Khan, T. Khiaonarong, M. Malaika, H. Monroe, N. Sugimoto, H. Tourpe, and P. Zhou. 2020. A Survey of Research on Retail Central Bank Digital Currency. *IMF Working Paper WP/20/104*, June. https://www.imf.org/en/Publications/WP/Issues/2020/06/26/A-Survey-of-Research-on-Retail-Central-Bank-Digital-Currency-49069

[81] R. Auer and R. Böhme. 2020. The Technology of Retail Central Bank Digital Currency. *BIS Quarterly Review*, March; see also BIS Working Paper No. 948. https://www.bis.org/publ/qtrpdf/r_qt2003j.htm

[82] L. Ante. 2025. From Adoption to Continuance: Stablecoins in Cross-Border Remittances and the Role of Digital and Financial Literacy. *Telematics and Informatics*, vol. 97, art. 102230. https://doi.org/10.1016/j.tele.2024.102230

[83] Y. Li, Y. Xiang, Q. Wang, T. H. Yuen, A. Deppeler, and J. Yu. 2026. SoK: Stablecoins in Retail Payments. *arXiv preprint arXiv:2601.00196*. https://arxiv.org/abs/2601.00196

[84] W. Du, C. Huang, and D. Scharfstein. 2026. Competing Rails for Cross-Border Payments: Banks, Fintechs, and Stablecoins. Harvard Business School Working Paper, February. https://www.hbs.edu/faculty/Pages/item.aspx?num=66992

[85] M. Fröhlich, J. A. Vega Vermehren, F. Alt, and A. Schmidt. 2022. Implementation and Evaluation of a Point-Of-Sale Payment System Using Bitcoin Lightning. In *Proc. NordiCHI*. http://www.medien.ifi.lmu.de/pubdb/publications/pub/froehlich2022nordichi/froehlich2022nordichi.pdf

[86] K. Zhang, T.-M. Choi, S.-H. Chung, Y. Dai, and X. Wen. 2024. Blockchain Adoption in Retail Operations: Stablecoins and Traceability. *European Journal of Operational Research*, 315, 1, pages 147–160. https://doi.org/10.1016/j.ejor.2023.11.026

[87] J. Lin, M. Liu, S. Li, and X. Wang. 2025. SecurePay: Enabling Secure and Fast Payment Processing for Platform Economy. In *Proc. IEEE/ACM Int. Symp. Quality of Service (IWQoS)*; *arXiv:2505.17466*. https://arxiv.org/abs/2505.17466

[88] Committee on Payments and Market Infrastructures. 2023. Considerations for the Use of Stablecoin Arrangements in Cross-Border Payments. Bank for International Settlements, July. https://www.bis.org/cpmi/publ/d220.pdf

[89] A. Kosse, J. Lisack, and N. Boakye-Agyeman. 2023. The Impact of Stablecoins on the International Monetary and Financial System. *BIS Papers*, no. 170, February. https://www.bis.org/publ/bppdf/bispap170.pdf

[90] E. Cerutti, M. Firat, and H. Pérez-Saiz. 2025. Estimating the Impact of Digital Money on Cross-Border Flows: Scenario Analysis Covering the Intensive Margin. *IMF Fintech Notes* 2025/002, February. https://www.imf.org/-/media/files/publications/ftn063/2025/english/ftnea2025002.pdf

[91] S. Chaliasos, M. Charalambous, L. Zhou, R. Galanopoulou, A. Gervais, D. Mitropoulos, and B. Livshits. 2024. Smart Contract and DeFi Security: Insights from Tool Evaluations and Practitioner Surveys. In *Proc. IEEE/ACM 46th Int. Conf. Software Engineering (ICSE)*, pages 160:1–160:13. https://doi.org/10.1145/3597503.3623302

[92] K. Qin, L. Zhou, B. Livshits, and A. Gervais. 2021. Attacking the DeFi Ecosystem with Flash Loans for Fun and Profit. In *Financial Cryptography and Data Security (FC)*, pages 3–32. https://doi.org/10.1007/978-3-662-63514-3_1

[93] T. Mackinga, T. Nadahalli, and R. Wattenhofer. 2024. SoK: Attacks on DAOs. *arXiv preprint arXiv:2406.15071*. https://arxiv.org/abs/2406.15071

[94] E. G. Weyl, P. Ohlhaver, and V. Buterin. 2022. Decentralized Society: Finding Web3's Soul. *SSRN Working Paper No. 4105763*. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4105763

[95] SmartContractSecurity. 2020. SWC Registry: Smart Contract Weakness Classification and Test Cases. https://swcregistry.io/

[96] G. Caldarelli. 2025. Can Artificial Intelligence Solve the Blockchain Oracle Problem? Unpacking the Challenges and Possibilities. *Frontiers in Blockchain*, vol. 8, art. 1682623. https://doi.org/10.3389/fbloc.2025.1682623

[97] M. Gurupriya, R. Gayathri, C. Yadav, and U. Umamaheshwar. 2025. Blockchain-Powered Platform for Microloans and Lending Solutions. In *Proc. Int. Conf. on Intelligent Computing, Instrumentation and Control Technologies (ICICT)*, pages 1–6. https://doi.org/10.1109/ICTEST64710.2025.11042350

[98] V. Buterin, Y. Weiss, K. Gazso, N. Patel, D. Tirosh, S. Nacson, and T. Hess. 2021. ERC-4337: Account Abstraction Using Alt Mempool. Ethereum Improvement Proposals. https://eips.ethereum.org/EIPS/eip-4337

[99] Behaviour-Centric Cybersecurity Center (BCCC), York University. 2025. BCCC-DeFiFraudTrans-2025: A Labelled Ethereum DeFi Fraud Transaction Dataset. Accessed June 29, 2026. https://www.yorku.ca/research/bccc/ucs-technical/cybersecurity-datasets-cds/defi-fraud-transactions-bccc-defifraudtrans-2025/

[100] P. K. Ozili. 2024. Central Bank Digital Currency Adoption Challenges. *Journal of Central Banking Theory and Practice*, 13, 1, pages 5–24. https://doi.org/10.2478/jcbtp-2024-0001